

UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato finale in **Impianti di Elaborazione**

Elaborato d'esame

Anno Accademico 2025/2026

Candidati

Ruggiero Bruno	**M63001704**
Matarazzo Alessio	**M63001708**
Vitale Luciano	**M63001774**

Indice

1 Capacity Test **4**
- 1.1 Setup . . . 4
- 1.2 Cattura dei parametri . . . 5
- 1.3 Visualizzazione dei parametri . . . 6
- 1.4 Analisi del bottleneck . . . 7
- 1.5 Fairness index . . . 8

2 PCA e Clustering **10**
- 2.1 Raccolta dei dati . . . 10
- 2.2 Principal component analysis . . . 10
- 2.3 Clustering . . . 11
- 2.4 Clustering senza PCA . . . 12

3 Workload Characterization **13**
- 3.1 Setup Sperimentale . . . 14
- 3.2 Fase 1: Caratterizzazione del Workload Reale . . . 15
  - 3.2.1 Analisi Metriche Alto Livello (HL) . . . 15
  - 3.2.2 Analisi Metriche Basso Livello (LL) . . . 19
- 3.3 Fase 2: Generazione Workload Sintetico . . . 21
- 3.4 Fase 3: Validazione (Data Validation) . . . 22
  - 3.4.1 Verifica della Normalità (Shapiro-Wilk) . . . 23
  - 3.4.2 Test di Validazione sui Centroidi . . . 24

4 Regression **26**
- 4.1 Exp1 . . . 26
  - 4.1.1 Test di normalità . . . 30
  - 4.1.2 Test di Mann-Kendal . . . 31
  - 4.1.3 Theil - Sen . . . 34
- 4.2 Exp2 . . . 35
  - 4.2.1 Analisi ByteRec . . . 35
  - 4.2.2 Analisi ByteSent . . . 39
  - 4.2.3 Analisi Nmail . . . 42
- 4.3 VMres1 . . . 45
  - 4.3.1 Stima lineare . . . 45
  - 4.3.2 Controllo della normalità . . . 45
  - 4.3.3 Test di Mann-Kendall . . . 46
  - 4.3.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G . . . 47
- 4.4 VMres2 . . . 47
  - 4.4.1 Stima lineare . . . 47
  - 4.4.2 Controllo della normalità . . . 48
  - 4.4.3 Test di Mann-Kendall . . . 49
  - 4.4.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G . . . 50

4.5 VMres3 . . . 50
4.5.1 Stima lineare . . . 50
4.5.2 Controllo della normalità . . . 51
4.5.3 Test di Mann-Kendall . . . 52
4.5.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G . . . 53
4.6 OS . . . 54
4.6.1 os1 . . . 54
4.6.2 os2 . . . 61
4.6.3 os3 . . . 68

5 Design Of Experiments 75
5.1 Response Time . . . 75
5.1.1 Stima del modello e analisi della varianza . . . 75
5.1.2 Normalità e omoschedasticità dei residui . . . 77
5.1.3 Test di significatività . . . 78
5.2 Throughput . . . 81
5.2.1 Stima del modello e analisi della varianza . . . 82
5.2.2 Normalità e omoschedasticità . . . 83
5.2.3 Test di significatività . . . 84
5.3 Latency . . . 85
5.3.1 Stima del modello e analisi della varianza . . . 86
5.3.2 Normalità e omoschedasticità . . . 87
5.3.3 Test di significatività . . . 89

6 Relay Diagram Block 90
6.1 Esercizio 1 . . . 90
6.2 Esercizio 2 . . . 92
6.3 Esercizio 3 . . . 95
6.4 Esercizio 4 . . . 98
6.5 Esercizio 5 . . . 101

7 Field Failure Data Analysis 104
7.1 Sistema Mercury . . . 104
7.1.1 Descrizione del Dataset e Architettura . . . 104
7.1.2 Analisi Globale del Sistema . . . 105
7.1.3 Analisi esplorativa per Categoria e Nodo (Local Analysis) . . . 108
7.2 Sistema BlueGene/L . . . 111
7.2.1 Descrizione del Dataset e Architettura . . . 111
7.2.2 Analisi Globale del Sistema . . . 112
7.2.3 Analisi Esplorativa per Computational card e Nodo . . . 114
7.3 Conclusioni e Confronto . . . 116

8 Benchmarking **117**
8.1 Raccolta Dati e calcolo dei campioni minimi 117
8.2 Test di normalità sui residui 118
8.3 Test di ipotesi 119

1 Capacity Test

In questo test vogliamo effettuare **Performance Analysis** di un web server, andandone a valutare tre parametri:

- Throughput
- Tempo di risposta
- Potenza

Il nostro obiettivo è individuare due capacità, dette *Knee Capacity* e *Usable Capacity*. La **Knee Capacity** è il punto di massima efficienza del sistema, in cui il sistema risponderà con un alto throughput e un basso tempo di risposta. Oltre questo punto, il sistema, nonostante continui a funzionare, inizia a perdere di efficienza, fino a raggiungere il punto di **Usable Capacity**, ovvero il carico massimo di throughput che il sistema riesce a reggere prima di degradare le performance.

1.1 Setup

La prima cosa da fare è andare a creare il web server. Come web server utilizzeremo una macchina virtuale gestita con **Oracle VirutalBox**. Il sistema operativo guest sarà **Ubuntu 24.04 LTS**. Alla VM abbiamo dedicato 1 GB di RAM e un core di CPU. Sulla VM allochiamo 5 diverse risorse di tipo testuale, ognuno con una file size diversa:

- *small1.txt*: 20.3 KiB
- *small2.txt*: 183.1 KiB
- *medium.txt*: 732.4 KiB
- *large1.txt*: 2.1 MiB
- *large2.txt*: 8.6 MiB

Lato client per generare il traffico di richieste HTTP abbiamo utilizzato **JMeter**. Il nostro obiettivo è effettuare tre catture per ogni valore di throughput, andando a richiedere per ogni cattura i file in maniera casuale. Per fare ciò creiamo un thread group di 50 utenti che invia richieste per un totale di 5 minuti (300 s) con un ramp up time di 10 s, il ramp up time è lo slot temporale che thred utilizzano per svegliarsi.



Figura 1: Configurazione thread group

Al suo interno andiamo a istanziare un **Random Controller**, in modo tale che ogni richiesta viene effettuata su una risorsa casuale. All'interno del Random Controller andiamo a specificare:

- il **tipo** di richieste da fare, ovvero HTTP;
- le **risorse** per cui fare richiesta, ovvero le cinque risorse del web server;
- un **Constant Throughput Timer**, ovvero il timer che determina il numero di richieste da inviare idealmente al minuto, da configurare col throughput desiderato.

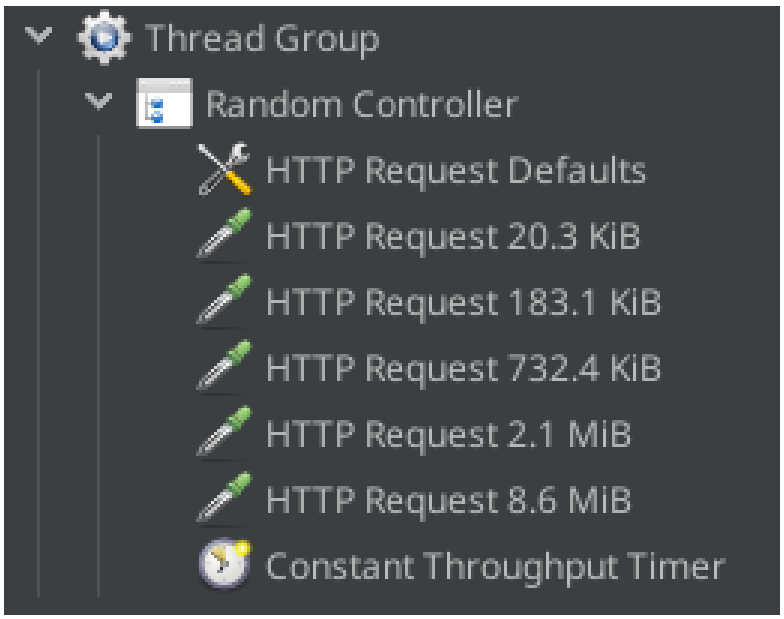


Figura 2: Struttura Random Controller

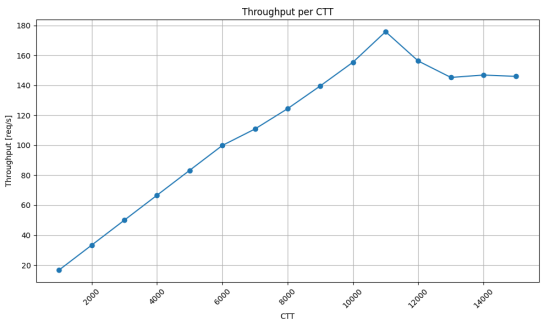
1.2 Cattura dei parametri

Dopo che abbiamo configurato il tutto, andiamo a eseguire le catture. In particolare, andiamo a strutturare le catture nella seguente maniera: eseguiamo tre

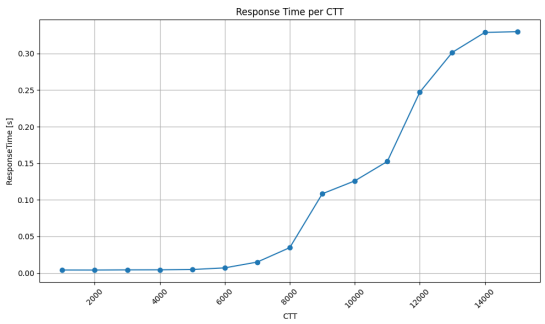
catture per valore di CTT, per valori di CTT da 1000 a 15000 a intervalli di 1000. A ogni cattura andiamo a memorizzare l'output in un file CSV contenente tutte le richieste inviate, l'esito e il tempo di risposta. Dopodiché per quanto riguarda il tempo di risposta andiamo a mediare tra tutti i tempi sotto la voce *elapsed*. Per il throughput, invece, andiamo innanzitutto a definire l'intervallo di tempo tra il timestamp più grande e il timestamp più piccolo, e dopo dividiamo il numero di righe (ovvero il numero di richieste) per il tempo calcolato precedentemente. Grazie a questo CSV riusciamo a ottenere tutti i parametri di alto livello che ci servono.
In parallelo alle catture, lato server eseguiamo il comando *vmstat* per catturare i parametri di basso livello.

1.3 Visualizzazione dei parametri

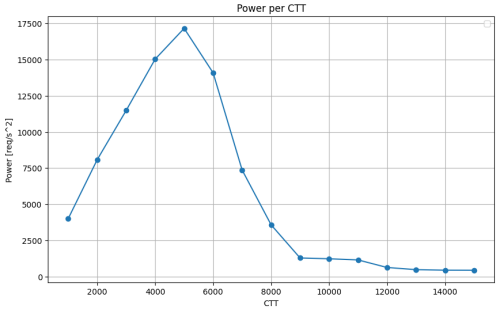
Dopo aver catturato i parametri andiamo a filtrare i parametri, calcolare la media e fare il grafico. Partendo dai parametri di alto livello il risultato è il seguente:



(a) Throughput



(b) Response time



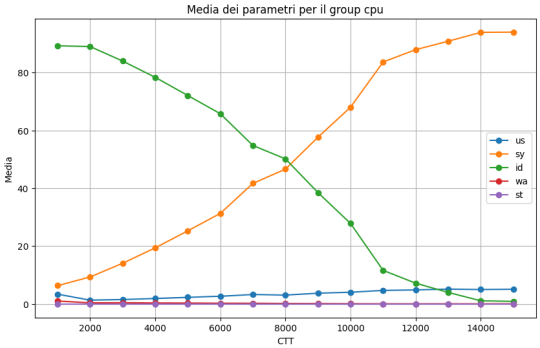
(c) Potenza

Figura 3: Parametri di alto livello

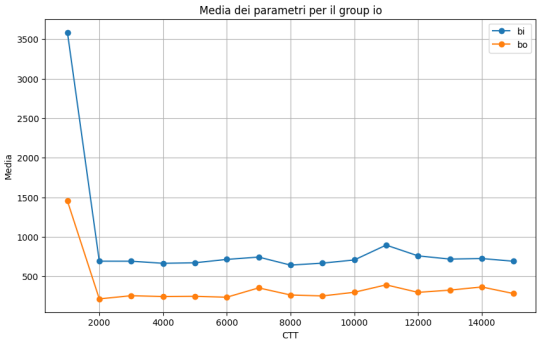
Dai grafici si riescono a evidenziare quali sono le capacità che ci interessano: la *Knee Capacity* è data dal punto di massima potenza, ovvero il punto in cui il rapporto tra throughput e tempo di risposta è maggiore. In particolare, dal grafico è facile notare che la *Knee Capacity* è situata a 5000 richieste al minuto. Per quanto riguarda la *Usable Capacity*, andiamo a individuare il massimo nel grafico del throughtput, massimo che è situato a 11000 richieste al minuto.

1.4 Analisi del bottleneck

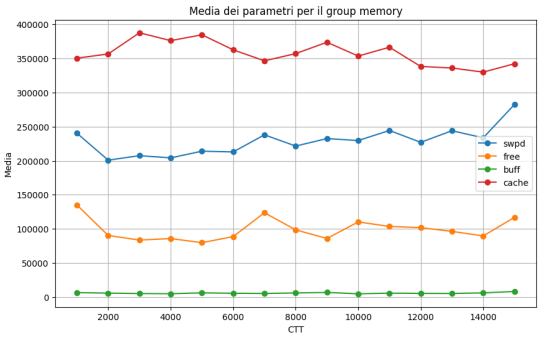
Andiamo ad analizzare i grafici dei parametri di basso livello per individuare la causa del bottleneck del sistema:



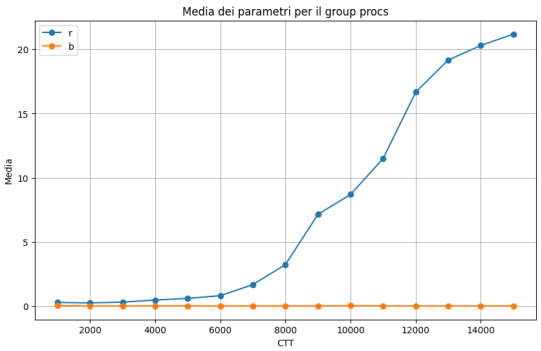
(a) CPU



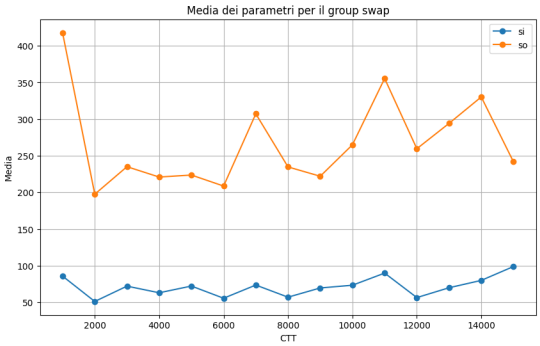
(b) I/O



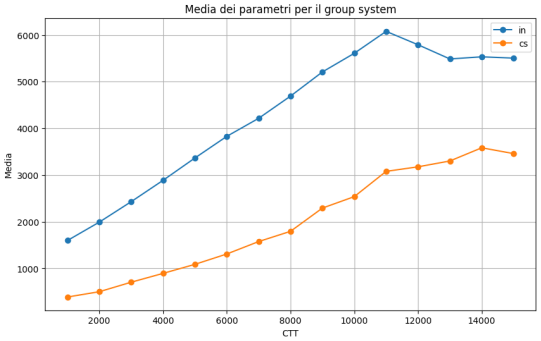
(c) Memoria



(d) Processi



(e) Swap



(f) Sistema

Figura 4: Parametri di basso livello

I parametri di memoria e I/O rimangono pressoché costanti. Aumentano, come da aspettativa, i parametri di sistema (interruzioni e context switch) e i processi. Dalle immagini si evince chiaramente come all'aumentare delle richieste in ingresso

cambia il rapporto tra kernel time e idle time. Inoltre, all'aumentare del throughput di richieste in ingresso, aumenta lo swapping della memoria. Se volessimo definire un bottleneck per il sistema, sarebbero sicuramente la CPU, che è sempre più impegnata, e la memoria, la quale necessita di fare più swapping.

1.5 Fairness index

Il **fairness index** è un parametro che indica se gli utenti vengono serviti in egual misura oppure no. Per calcolarlo andiamo a utilizzare solamente tre delle cinque risorse che abbiamo utilizzato precedentemente:

- *small.txt*: 20.3 KiB
- *medium.txt*: 732.4 KiB
- *large.txt*: 8.6 MiB

Dopodiché definiamo tre flussi di richieste, ognuno che chiederà una risorsa diversa. Questi tre flussi invieranno richieste contemporaneamente, in modo tale da osservare come il server gestirà più flussi di richieste. Effettueremo tre catture, ognuna di 5 minuti, strutturati nella seguente maniera:

- Numero di utenti **costante**, CTT **decrescente** al crescere della dimensione dei file
    1. risorsa piccola: 4000 CTT e 20 utenti
    2. risorsa media: 2000 CTT e 20 utenti
    3. risorsa grande: 1200 CTT e 20 utenti
- Numero di utenti **costante**, CTT **crescente** al crescere della dimensione dei file
    1. risorsa piccola: 1200 CTT e 20 utenti
    2. risorsa media: 2000 CTT e 20 utenti
    3. risorsa grande: 4000 CTT e 20 utenti
- Numero di utenti **crescente** e CTT **decrescente** al crescere della dimensione dei file
    1. risorsa piccola: 1200 CTT e 10 utenti
    2. risorsa media: 2000 CTT e 20 utenti
    3. risorsa grande: 4000 CTT e 30 utenti

La formula con cui calcoliamo il fairness index è la seguente:

$$f\left(x_1, x_2, ..., x_n\right) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}$$

dove x_i è il **throughput normalizzato**, calcolato come:

$$x_{size} = \frac{Measured\ Throughput}{Fair\ Throughput}$$

Abbiamo ottenuto i seguenti risultati:

Test	$x_{piccola}$	x_{media}	x_{grande}	Fairness
1	1.0008	1.0018	1.0032	0.9999
2	1.0032	1.0019	0.8839	0.9969
3	1.0003	1.0019	1.0047	0.9999

Tabella 1: Fairness index

Dalla tabella si può osservare che il fairness index è molto vicino a 1, per cui il sistema serve gli utenti in maniera molto bilanciata, senza favorire nessuno rispetto ad altri.

2 PCA e Clustering

Lo scopo di questo esercizio è quello di effettuare una raccolta di dati monitorando le prestazione del proprio calcolatore ed effettuare l'operazione di Principal Component Analysis e Clustering per ottenerne un workload sintetico.

2.1 Raccolta dei dati

I dati sono stati raccolti con il software messo a disposizione da windows, ovvero Perfmon. La cattura è stata lasciata attiva per 1 ora dividendo il tempo in 3 sezioni che vengono differenziate dalla combinazione di tipologie di stress di ogni componente(memoria, Disco e CPU). Per stressare la CPU è stato utilizzato il Simulatore Nbody con 5000 pianeti, mentre per Memoria e Disco è stato utilizzato Stress-ng con il seguente comando.

stress-ng –vm 1 –vm-bytes 40% –hdd 4 –hdd-bytes 2G

ovvero sto dicendo di stressare la ram al 40% ed effettuare delle scritture e letture sul disco di 2 Giga a volta. Con lo strumento Perfmon sono quindi stati recuperate numerose informazioni a partire da diverse parti del sistema come disco memoria e processore, in particolare le colonne uscite fuori dalla raccolta dati sono 27.

2.2 Principal component analysis

Dopo aver raccolto i nostri dati il prossimo passo è effettuare una trasformazione decorrelante(PCA) che ci permettesse di spiegare esplicitamente la varianza persa se volessimo togliere delle colonne dal dataset. Prima di fare ciò abbiamo effettuato un primo filtraggio sulle colonne che non fornivano alcun significato, studiandone la distribuzione abbiamo eliminando le colonne contenenti tutti 0 e le colonne costanti.

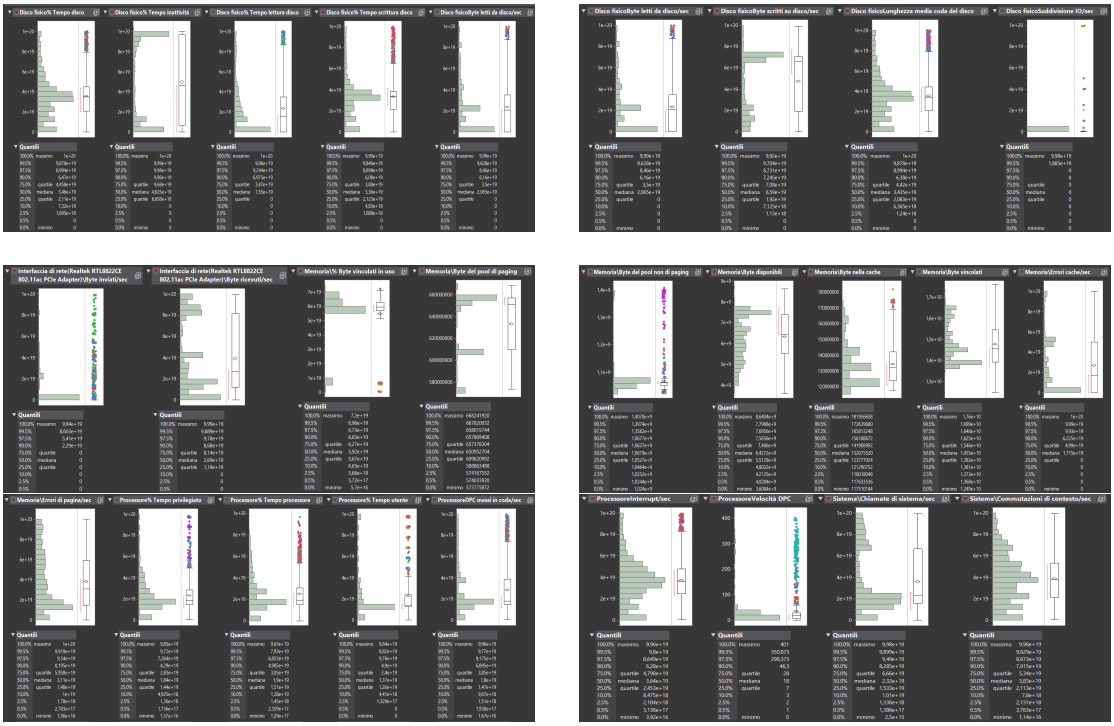


Figura 5: Distribuzione dei dati iniziali

Dopo aver effettuato a monte un primo filtraggio abbiamo provato ad effettuare la PCA e ci siamo presi un range di componenti principali che andassero da 17 a 19, con una varianza mantenuta che va dal **84.4%** al **89.6%**.

2.3 Clustering

Per ogni numero di componenti principali scelte è stata poi fatta una clusterizzazione gerarchica per ridurre ulteriormente la mole di dati, l'obbiettivo è dividere il dataset in cluster e andare a selezionare solo una riga per cluster che rappresenti bene l'intero insieme. La clusterizzazione è avvenuta seguendo un range di cluster che va da 5 a 100 con uno scarto di 10 per volta. Successivamente è stato tracciato un grafico che rappresentasse la varianza persa combinando le scelte delle componenti principali con le scelte della clusterizzazione, seguendo la formula:

$$Var_{LossTot} = Var_{LossPCA} + Var_{LossCluster} * Var_{KeptPCA}$$

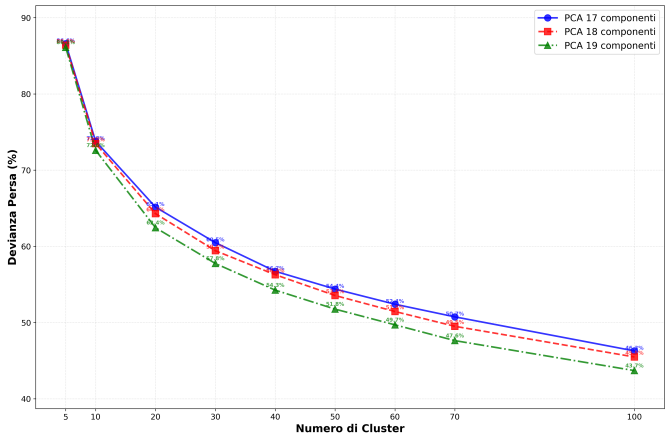


Figura 6: Devianza persa Totale

Dal grafico abbiamo ricavato che il punto di knee che ci dava il giusto compromesso tra informazione mantenuta e sintesi del workload fosse quello con 19 componenti principali e 20 cluster, andando a perdere circa il **62.4%** di varianza.

2.4 Clustering senza PCA

Un'ulteriore analisi è stata fatta cercando una clusterizzazione ottimale senza effettuare la PCA, sebbene non consigliabile come pratica, il nostro risultato non si è discostato troppo dall'analisi con PCA poichè le informazioni sulle colonne non avevano un alto grado di correlazione già in principio.

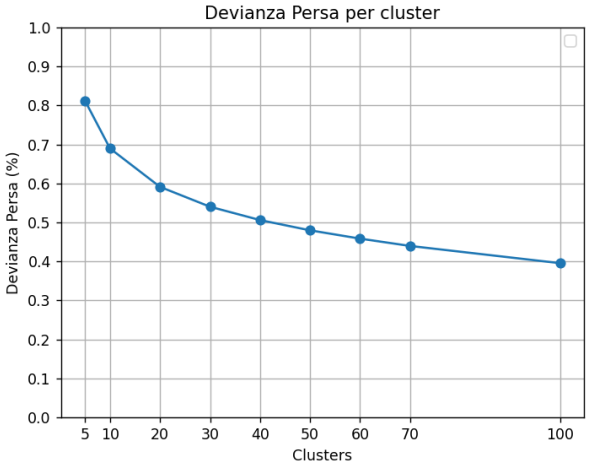


Figura 7: Devianza persa no PCA

Osservando il grafico è possibile notare che mantenendo lo stesso numero di cluster è possibile mantenere solo il **3,4%(59% di varianza persa rispetto a 62,4%)** in piu di varianza, con lo svantaggio però di dover mantenere tutte e 27 le colonne di partenza.

3 Workload Characterization

In questo capitolo viene descritto il processo di Workload Characterization. L'obiettivo è caratterizzare un carico reale complesso per ottenerne uno sintetico, statisticamente equivalente.

Caratterizzare un workload significa creare un modello statistico compatto e rappresentativo del carico reale. Il modello deve infatti catturare sia il comportamento statico che dinamico del carico, deve essere facilmente riproducibile e spiegare la varianza che perdiamo nell'usarlo.

Ci aspettiamo quindi che, pur usando un workload sintetico, gli effetti sul sistema sotto esame (SUT - System Under Test) siano statisticamente gli stessi

Il processo metodologico seguito è schematizzato nella Figura 8 e si articola in tre fasi distinte.

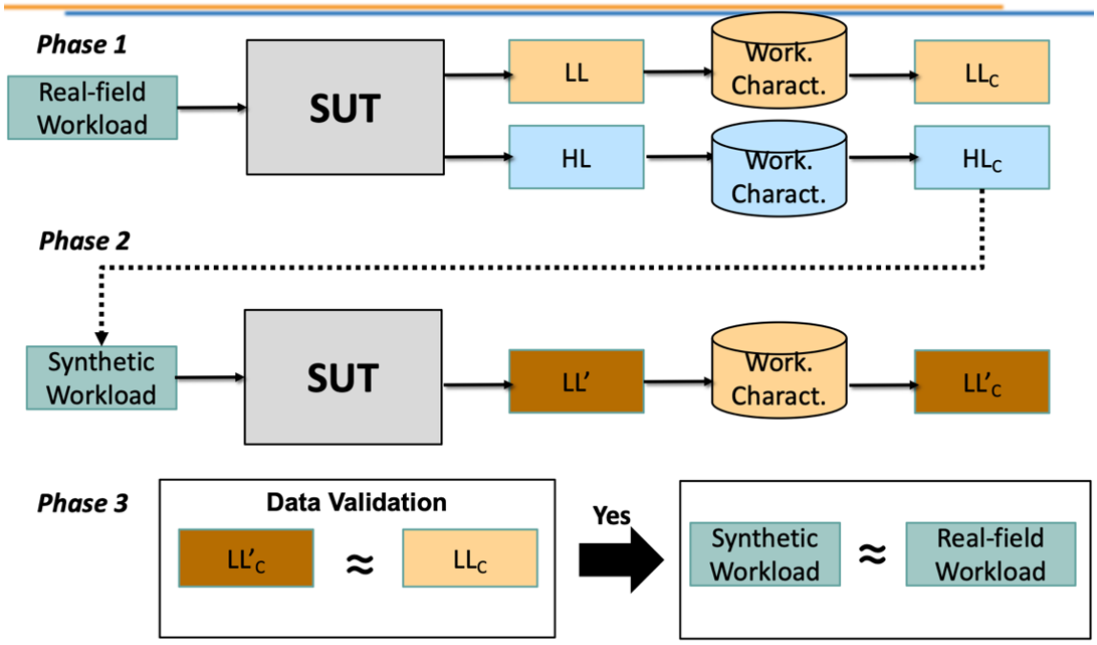


Figura 8: Processo di Workload Characterization a tre fasi

Fase 1: Analisi e Modellazione (Real-field Workload): In questa fase, il carico reale viene applicato al sistema sotto test. Durante l'esecuzione, vengono raccolte due tipologie di metriche:

- Metriche di Alto Livello (HL): Descrivono il comportamento dell'utente o del generatore di traffico (es. tipologia di richiesta HTTP, tempo di risposta, dimensione dei dati scambiati).
- Metriche di Basso Livello (LL): Descrivono l'impatto del carico sulle risorse fisiche del server (es. utilizzo CPU, consumo Memoria, I/O disco, context switches).

Sui dati raccolti verrà poi applicata la PCA e il Clustering. Il risultato è un modello caratterizzato del carico reale (HL_C e LL_C).

Fase 2: Generazione del Workload Sintetico Utilizzando il modello ottenuto dalla caratterizzazione HL, viene generato un Workload Sintetico. Vengono riprodotte le caratteristiche statistiche fondamentali del carico reale attraverso un numero ridotto di richieste. Il carico sintetico viene quindi applicato nuovamente al SUT, misurando le nuove metriche di basso livello (LL'). Anche su questi nuovi dati viene applicata la stessa pipeline di analisi (PCA e Clustering) per ottenere la caratterizzazione sintetica (LL'_C).

Fase 3: Validazione (Data Validation) L'ultima fase è critica e serve a confermare la qualità del modello. Si confrontano i parametri di basso livello risultanti del carico reale (LL_C) con quelli del carico sintetico (LL'_C) attraverso test statistici. Se i due dataset risultano statisticamente equivalenti (ovvero $LL'_C \approx LL_C$), allora il workload sintetico è validato e può essere utilizzato come sostituto affidabile del carico reale per future analisi sul sistema.

3.1 Setup Sperimentale

Per l'acquisizione dei dati è stato utilizzato lo stesso ambiente del Capacity Test: Web Server su macchina virtuale con cattura dei parametri LL tramite vmstat, Apache JMeter per la generazione del carico con cattura dei parametri HL tramite SimpleDataWriter

Il piano di test JMeter è stato configurato con 3 Thread Groups eseguiti in maniera sequenziale. I gruppi, da 50 thread ognuno, hanno una durata di 300 secondi e un ramp-up time di 10 secondi. Le risorse vengono richieste mediante una logica dettata dal Random Controller.

Di seguito l'elenco delle risorse associate a ciascun gruppo con la relativa dimensione:

- **Group-1**
  - File10MB-1 (HTML): 15.64 MB
  - File8MB-1 (HTML): 12.49 MB
  - CityBW.png: 1.86 MB
  - City.png: 1.80 MB
  - autumn.jpeg: 19.34 KB
  - turbo.jpeg: 8.70 KB
  - forest.jpeg: 6.93 KB
  - heart.jpeg: 5.60 KB
- **Group-2**
  - File10MB-2 (HTML): 10.49 MB

  - File3.4MB-2 (HTML): 3.53 MB
  - File1.1MB-2 (HTML): 1.19 MB
  - File391KB-2 (HTML): 399.78 KB
  - File132KB-2 (HTML): 134.74 KB
  - File45KB-2 (HTML): 45.55 KB
  - File15KB-2 (HTML): 15.55 KB
  - File5KB-2 (HTML): 5.45 KB

- **Group-3**
  - File5.9MB-3 (HTML): 6.08 MB
  - File2MB-3 (HTML): 2.05 MB
  - File673KB-3 (HTML): 688.96 KB
  - File227KB-3 (HTML): 232.04 KB
  - File77KB-3 (HTML): 78.29 KB
  - File26KB-3 (HTML): 26.56 KB
  - File9KB-3 (HTML): 9.16 KB

3.2 Fase 1: Caratterizzazione del Workload Reale

3.2.1 Analisi Metriche Alto Livello (HL)

Sono state raccolte un totale di **12.165 osservazioni** di richieste HTTP. I dati grezzi forniti da JMeter includevano un ampio set di metriche, le cui distribuzioni originali sono riportate in Figura.



Figura 9: Distribuzioni HL: TimeStamp, Elapsed, Label, ResponseCode

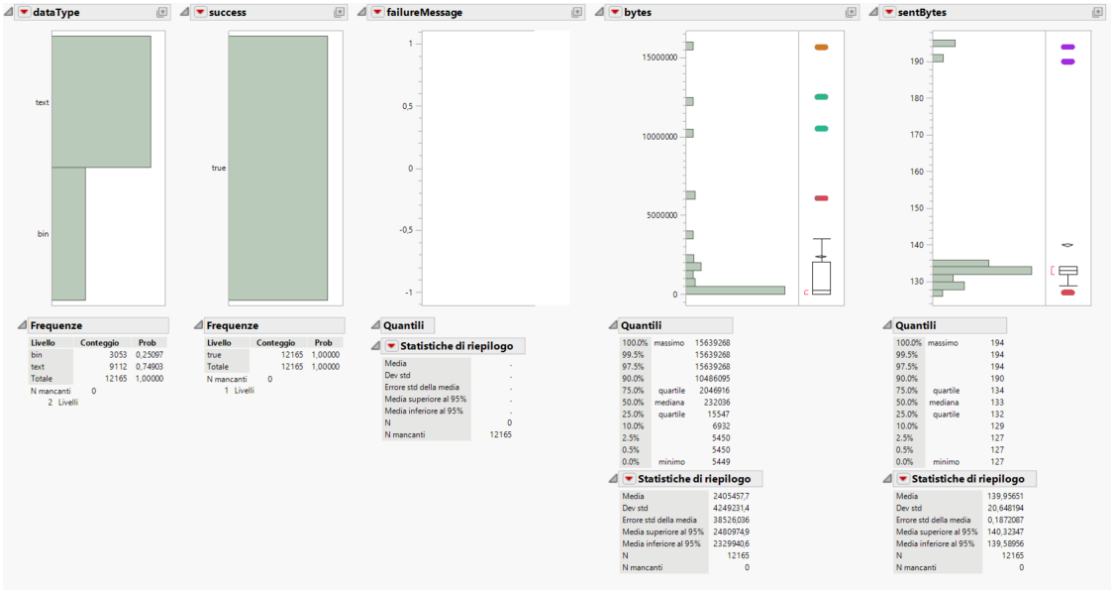


Figura 10: Distribuzioni HL: DataType, Success, Bytes, SentBytes

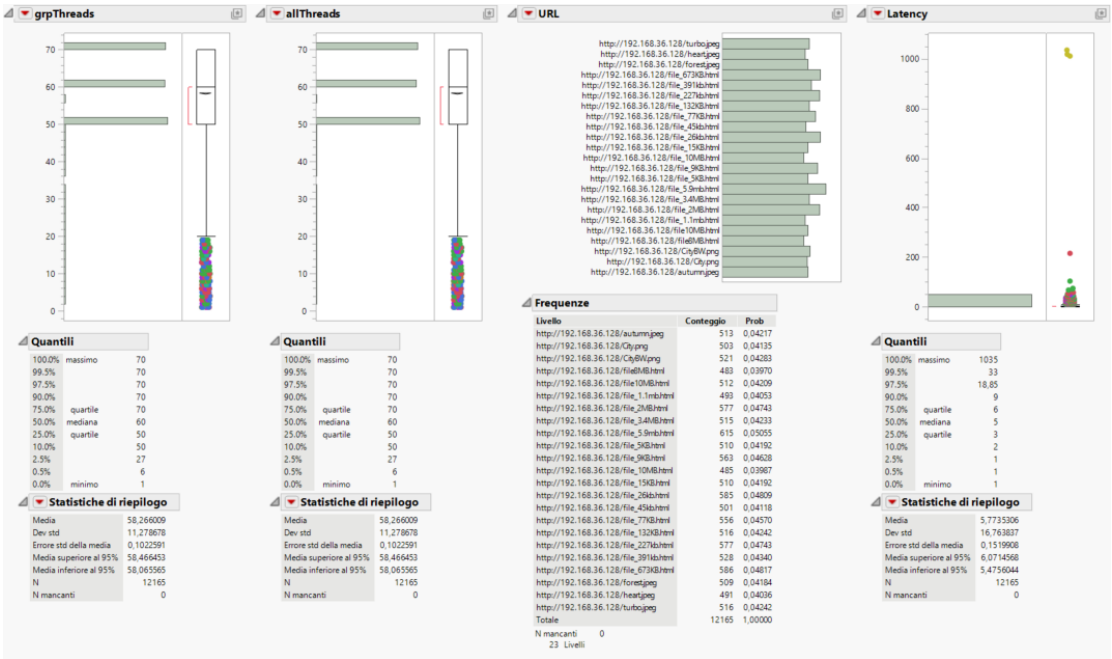


Figura 11: Distribuzioni HL: Threads, URL, Latency



Figura 12: Distribuzioni HL: IdleTime, Connect Time

Prima di procedere con l'analisi multivariata, è stata effettuata una fase di **preprocessing e filtraggio** per selezionare solo le variabili significative. Dall'analisi degli istogrammi sopra riportati, sono state scartate le seguenti metriche:

- **Costanti:** *responseCode* (sempre 200), *responseMessage*, *success* (sempre true) e *IdleTime* (sempre 0). Avendo varianza nulla, non apportano informazione.
- **Categoriche/Ridondanti:** *URL* (ridondante rispetto alla Label) e *dataType*.
- **Metadati:** *timeStamp*, in quanto indice temporale non utile alla caratterizzazione statistica del carico.

Le metriche numeriche mantenute per l'analisi sono dunque: **elapsed, bytes, sentBytes, Latency, grpThreads, Connect**.

Per ridurre la dimensionalità del problema e rimuovere le correlazioni tra queste metriche, è stata applicata la Principal Component Analysis. Dall'analisi degli autovalori, mostrata in Figura 13, si osserva che le prime **4 componenti principali** sono sufficienti a spiegare circa il **97.9%** della varianza cumulativa del dataset.

Autovalori

Numero	Autovalore	Percentuale	20 40 60 80	Percentuale cumulativa
1	2,329858	38,831		38,831
2	1,744102	29,068		67,899
3	0,917699	15,295		83,194
4	0,882106	14,702		97,896
5	0,094586	1,576		99,473
6	0,031649	0,527		100,000

Figura 13: Autovalori PCA e varianza spiegata (HL)

Successivamente, sulle 4 componenti principali individuate, è stato applicato un **clustering gerarchico** utilizzando la **distanza di Ward** come metrica di aggregazione. Per determinare il numero ottimale di cluster (k), è stata effettuata un'analisi basata sul compromesso tra numero di cluster e informazione persa.

Come visibile nel grafico in Figura 14, la curva della devianza persa inizia ad appiattirsi in modo significativo intorno a $k = 7$. È stata pertanto selezionata una configurazione a **7 Cluster**, che corrisponde a una devianza persa totale di circa il **35%**.

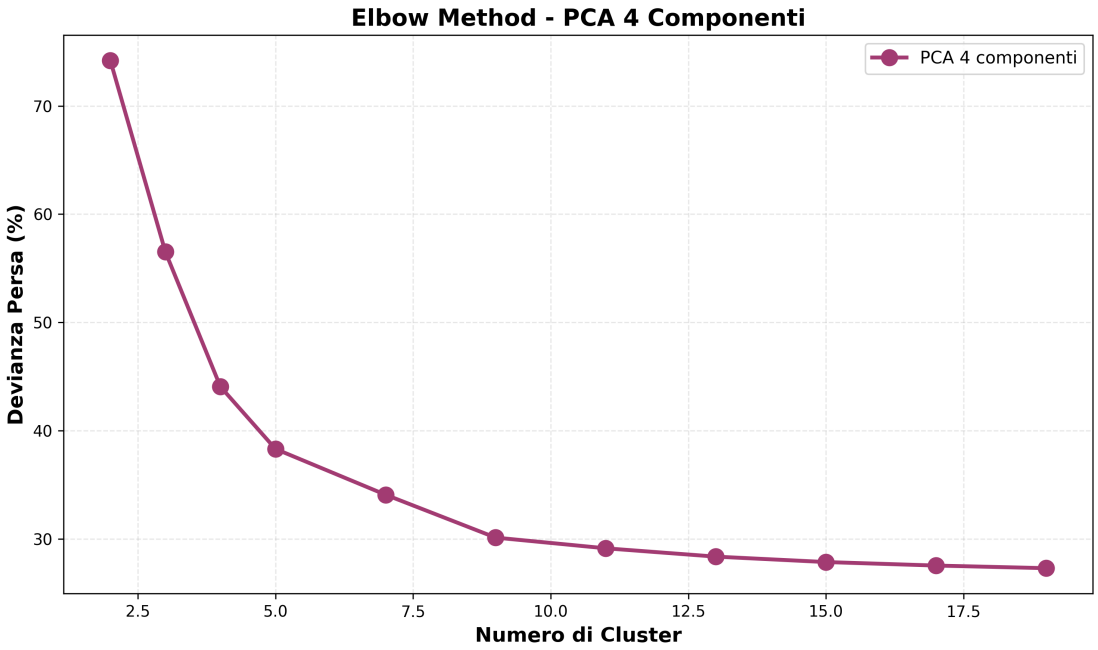


Figura 14: Devianza persa vs Numero di Cluster (PCA 4)

3.2.2 Analisi Metriche Basso Livello (LL)

Parallelamente all'analisi di alto livello, sono state monitorate le prestazioni della macchina server tramite il tool `vmstat`. I dati grezzi includono metriche relative a CPU, Memoria, I/O e Processi. Le distribuzioni delle metriche osservate sono riportate nelle figure seguenti.



Figura 15: Distribuzioni LL: CPU e Interupts (Int, Csw, User, Sys, Idle, Wait)

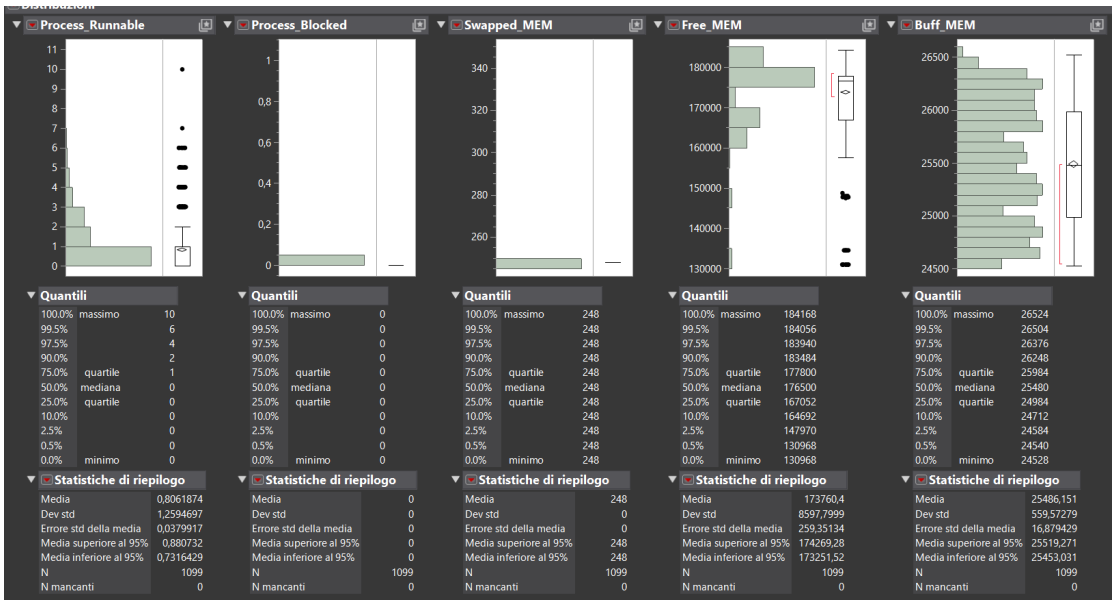


Figura 16: Distribuzioni LL: Memoria e Processi (Run, Block, Swap, Free, Buff)

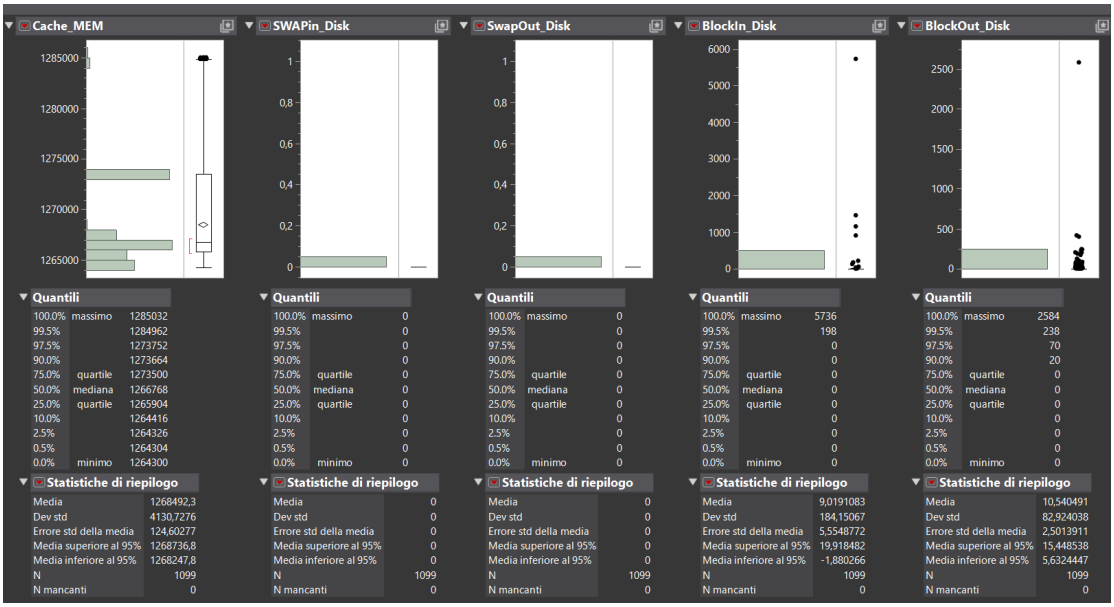


Figura 17: Distribuzioni LL: I/O Disco e Swap (Cache, SwapIn/Out, BlockIn/Out)

Analogamente a quanto fatto per l'HL, è stata effettuata una fase di **preprocessing e filtraggio**. Dall'analisi degli istogrammi riportati sopra, sono state scartate le metriche con **varianza nulla o trascurabile**, ovvero quelle relative allo swap su disco (*SwapIn, SwapOut*), alla memoria swappata (*Swapped_MEM*) e ai processi bloccati (*Process_Blocked*), in quanto presentano valori costanti vicini allo zero.

Le metriche mantenute per la caratterizzazione includono l'intero set dei descrittori di CPU (*User, System, Idle, Wait*), Memoria (*Free, Buff, Cache*), I/O disco (*BlockIn, BlockOut*) e attività di sistema (*Interrupt, Context Switch*).

Sul dataset filtrato è stata applicata la Principal Component Analysis. La scelta si è posta tra l'utilizzo di **6 o 7 componenti principali**, che spiegano rispettivamente circa l'89.8% e il 94.9% della varianza totale.

Autovalori

Numero	Autovalore	Percentuale	Percentuale cumulativa
1	3,559632	29,664	29,664
2	2,493041	20,775	50,439
3	1,965408	16,378	66,817
4	0,997236	8,310	75,128
5	0,946671	7,889	83,017
6	0,821833	6,849	89,865
7	0,614581	5,122	94,987
8	0,356544	2,971	97,958
9	0,150778	1,256	99,214
10	0,087469	0,729	99,943
11	0,006737	0,056	99,999
12	0,000070	0,001	100,000

Figura 18: Autovalori PCA e varianza spiegata (LL)

La decisione definitiva è stata presa analizzando l'impatto di questa scelta sulla qualità del clustering successivo.

In linea con l'approccio utilizzato per le metriche di alto livello è stato applicato un Clustering Gerarchico sulle componenti principali.

Anche qui per determinare il numero ottimale di cluster è stata effettuata un'analisi della devianza persa.

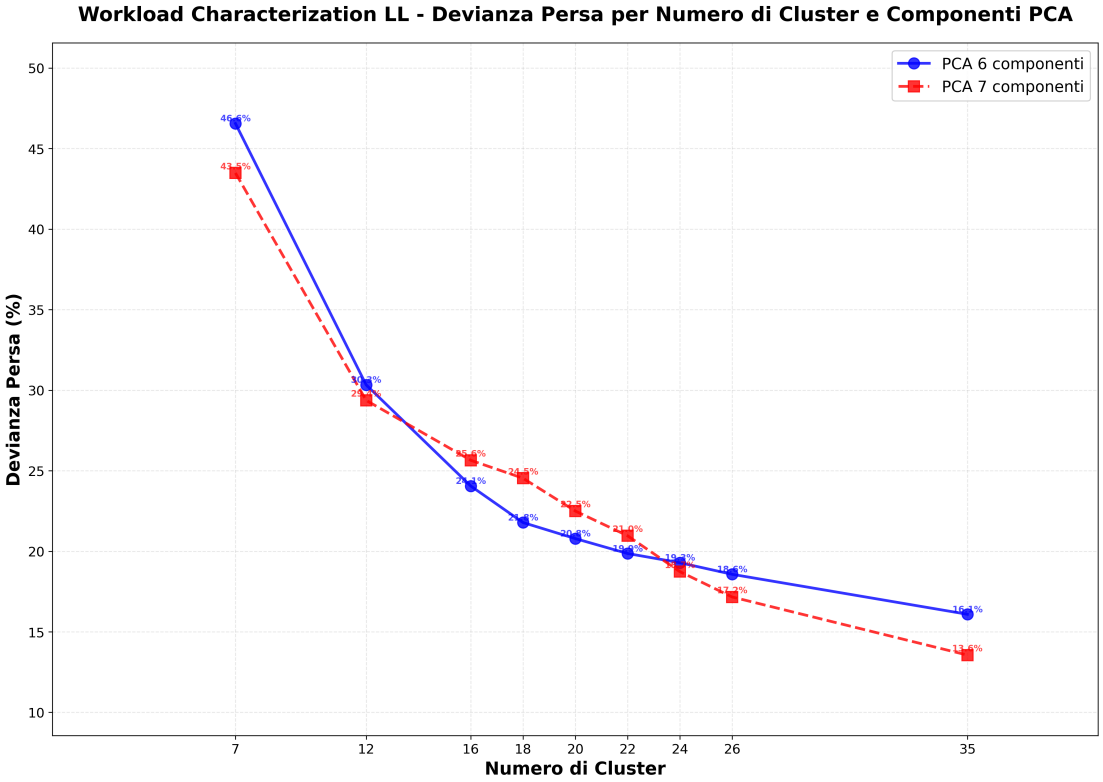


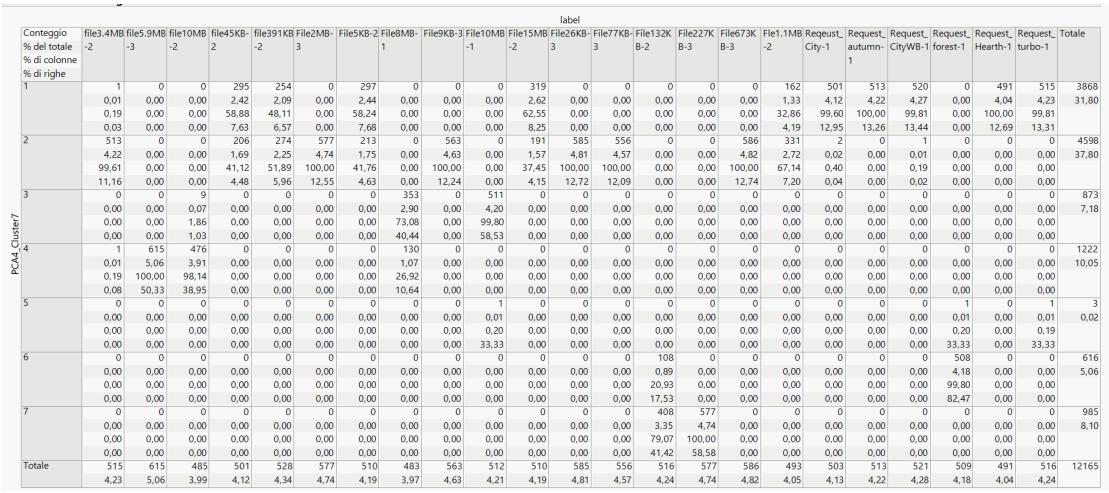
Figura 19: Analisi della Devianza Persa: confronto PCA 6 vs PCA 7 componenti

Osservando la curva relativa alla PCA a 6 componenti, si nota che il punto $k = 18$ è il miglior compromesso copn una devianza persa intorno al **21.8%** Pertanto, il modello finale per il Basso Livello è costituito da **6 Componenti Principali e 18 Cluster**.

3.3 Fase 2: Generazione Workload Sintetico

Sulla base dei risultati ottenuti dalla caratterizzazione HL, è stato generato il workload sintetico. L'obiettivo di questa fase è tradurre i cluster individuati in un set di richieste rappresentative da sottoporre al server.

Per fare ciò, abbiamo analizzato la tabella di contingenza (Figura 20) che incrocia i 7 cluster ottenuti con le etichette (*Label*) delle risorse originali. Per ogni cluster è stata selezionata la richiesta HTTP con la frequenza maggiore.



PCA4_Cluster7	Conteggio % del totale % di colonne % di righe	label file3.4MB-2	file5.9MB-3	file10MB-2	file45KB-2	file391KB-2	File2MB-3	File5KB-2	File8MB-1	File9KB-3	File10MB-1	File15MB-2	File26KB-3	File77KB-3	File132K B-2	File227K B-3	File673K B-3	Fle1.1MB-2	Reqeust_ City-1	Request_ autumn-1	Request_ CityWB-1	Request_ forest-1	Request_ Hearth-1	Request_ turbo-1	Totale
1		1	0	0	295	254	0	297	0	0	0	319	0	0	0	0	0	162	501	513	520	0	491	515	3868
		0,01	0,00	0,00	2,42	2,09	0,00	2,44	0,00	0,00	0,00	2,62	0,00	0,00	0,00	0,00	0,00	1,33	4,12	4,22	4,27	0,00	4,04	4,23	31,80
		0,19	0,00	0,00	58,88	48,11	0,00	58,24	0,00	0,00	0,00	62,55	0,00	0,00	0,00	0,00	0,00	32,86	99,60	100,00	99,81	0,00	100,00	99,81	
		0,03	0,00	0,00	7,63	6,57	0,00	7,68	0,00	0,00	0,00	8,25	0,00	0,00	0,00	0,00	0,00	4,19	12,95	13,26	13,44	0,00	12,69	13,31	
2		513	0	0	206	274	577	213	0	563	0	191	585	556	0	0	586	331	2	0	1	0	0	0	4598
		4,22	0,00	0,00	1,69	2,25	4,74	1,75	0,00	4,63	0,00	1,57	4,81	4,57	0,00	0,00	4,82	2,72	0,02	0,00	0,01	0,00	0,00	0,00	37,80
		99,61	0,00	0,00	41,12	51,89	100,00	41,76	0,00	100,00	0,00	37,45	100,00	100,00	0,00	0,00	100,00	67,14	0,40	0,00	0,19	0,00	0,00	0,00	
		11,16	0,00	0,00	4,48	5,96	12,55	4,63	0,00	12,24	0,00	4,15	12,72	12,09	0,00	0,00	12,74	7,20	0,04	0,00	0,02	0,00	0,00	0,00	
3		0	0	9	0	0	0	0	353	0	511	0	0	0	0	0	0	0	0	0	0	0	0	0	873
		0,00	0,00	0,07	0,00	0,00	0,00	0,00	2,90	0,00	4,20	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	7,18
		0,00	0,00	1,86	0,00	0,00	0,00	0,00	73,08	0,00	99,80	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
		0,00	0,00	1,03	0,00	0,00	0,00	0,00	40,44	0,00	58,53	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
4		1	615	476	0	0	0	0	130	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1222
		0,01	5,06	3,91	0,00	0,00	0,00	0,00	1,07	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	10,05
		0,19	100,00	98,14	0,00	0,00	0,00	0,00	26,92	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
		0,08	50,33	38,95	0,00	0,00	0,00	0,00	10,64	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
5		0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	3
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,01	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,01	0,00	0,01	0,02
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,20	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,20	0,00	0,19	
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	33,33	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	33,33	0,00	33,33	
6		0	0	0	0	0	0	0	0	0	0	0	0	0	108	0	0	0	0	0	0	508	0	0	616
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,89	0,00	0,00	0,00	0,00	0,00	0,00	4,18	0,00	0,00	5,06
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	20,93	0,00	0,00	0,00	0,00	0,00	0,00	99,80	0,00	0,00	
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	17,53	0,00	0,00	0,00	0,00	0,00	0,00	82,47	0,00	0,00	
7		0	0	0	0	0	0	0	0	0	0	0	0	0	408	577	0	0	0	0	0	0	0	0	985
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	3,35	4,74	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	8,10
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	79,07	100,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
		0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	41,42	58,58	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00	
Totale		515	615	485	501	528	577	510	483	563	512	510	585	556	516	577	586	493	503	513	521	509	491	516	12165
		4,23	5,06	3,99	4,12	4,34	4,74	4,19	3,97	4,63	4,21	4,19	4,81	4,57	4,24	4,74	4,82	4,05	4,13	4,22	4,28	4,18	4,04	4,24	

Figura 20: Distribuzione delle Risorse (Label) all'interno dei 7 Cluster

Dall'analisi della tabella, è emerso che il Cluster 5 contiene solamente 3 osservazioni su un totale di 12.165, andando a esplorare dove si potessero trovare questi outliers abbiamo visto che erano dovuti ad una situazione di basso carico e, pertanto, il Cluster 5 è stato escluso dalla generazione del workload sintetico.

Le risorse selezionate per il workload sintetico, basate sui restanti 6 cluster significativi, sono le seguenti:

- **Cluster 1:** CityBW.png 1.86 MB (Thread Group 1)
- **Cluster 2:** File673KB (Thread Group 3)
- **Cluster 3:** File10MB-1 (Thread Group 1)
- **Cluster 4:** File10MB-2 (Thread Group 2)
- **Cluster 6:** Request_forest-1 6.93KB (Thread Group 1)
- **Cluster 7:** File227KB-3 (Thread Group 3)

È stato quindi creato un **nuovo Test Plan su JMeter** contenente esclusivamente queste 6 richieste. Durante l'esecuzione abbiamo misurato solo i parametri di basso livello sulla macchina che ospita il server, ottenendo così il workload di basso livello LL'. Su questi dati abbiamo eseguito analisi e pre-filetring, applicando poi le stesse configurazioni di PCA e clustering utilizzate per LL. Il risultato di questa fase è stato il workload caratterizzato LL'_C.

3.4 Fase 3: Validazione (Data Validation)

L'ultima fase del processo di Workload Characterization consiste nel verificare se il workload sintetico generato (LL') è statisticamente equivalente al workload reale di partenza (LL). La validazione è stata condotta attraverso il test di ipotesi:

- H_0 (Ipotesi Nulla): Le distribuzioni del workload sintetico e reale provengono dalla stessa popolazione.

- H_1 (Ipotesi Alternativa): Le distribuzioni sono statisticamente differenti.

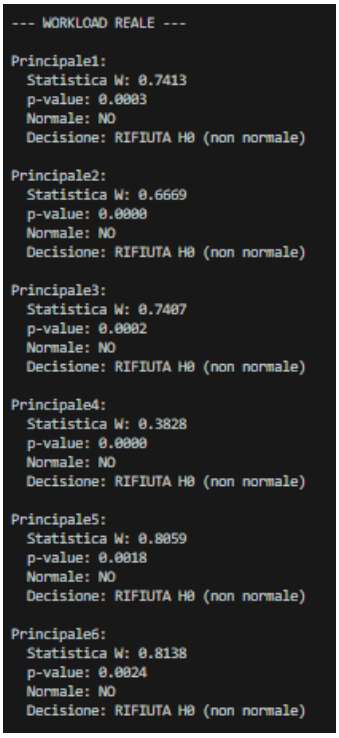
Il livello di significatività scelto per i test è $\alpha = 0.05$.

3.4.1 Verifica della Normalità (Shapiro-Wilk)

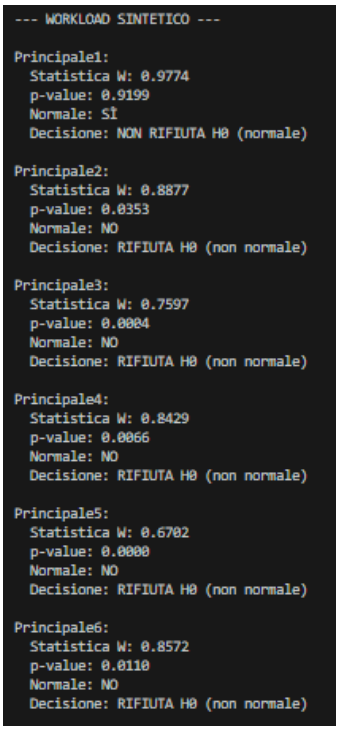
Prima di selezionare il test di confronto adeguato, è stato necessario verificare la normalità delle distribuzioni delle Componenti Principali. È stato utilizzato il test di Shapiro-Wilk.

I risultati, mostrati in Figura 21, indicano che:

- Nel **Workload Reale**, nessuna delle 6 componenti segue una distribuzione normale (p-value < 0.05).
- Nel **Workload Sintetico**, solo la prima componente mostra normalità, mentre le altre no.



(a) Test Normalità - Reale



(b) Test Normalità - Sintetico

Figura 21: Esiti del test di Shapiro-Wilk: Rifiuto della normalità per la maggior parte delle componenti

Data l'assenza di normalità nei dati, non è stato possibile applicare test parametrici. Si è optato dunque per il test non parametrico di Wilcoxon Rank-Sum.

3.4.2 Test di Validazione sui Centroidi

Il confronto è stato effettuato sui **centroidi** dei 18 cluster identificati.

Sono stati confrontati due dataset:

- **Dataset A:** 18 centroidi derivati dal Workload Reale.
- **Dataset B:** 18 centroidi derivati dal Workload Sintetico.

I risultati visivi del confronto sono riportati nei boxplot in Figura 22, che mostrano una notevole sovrapposizione delle distribuzioni mediane per tutte le 6 componenti principali.

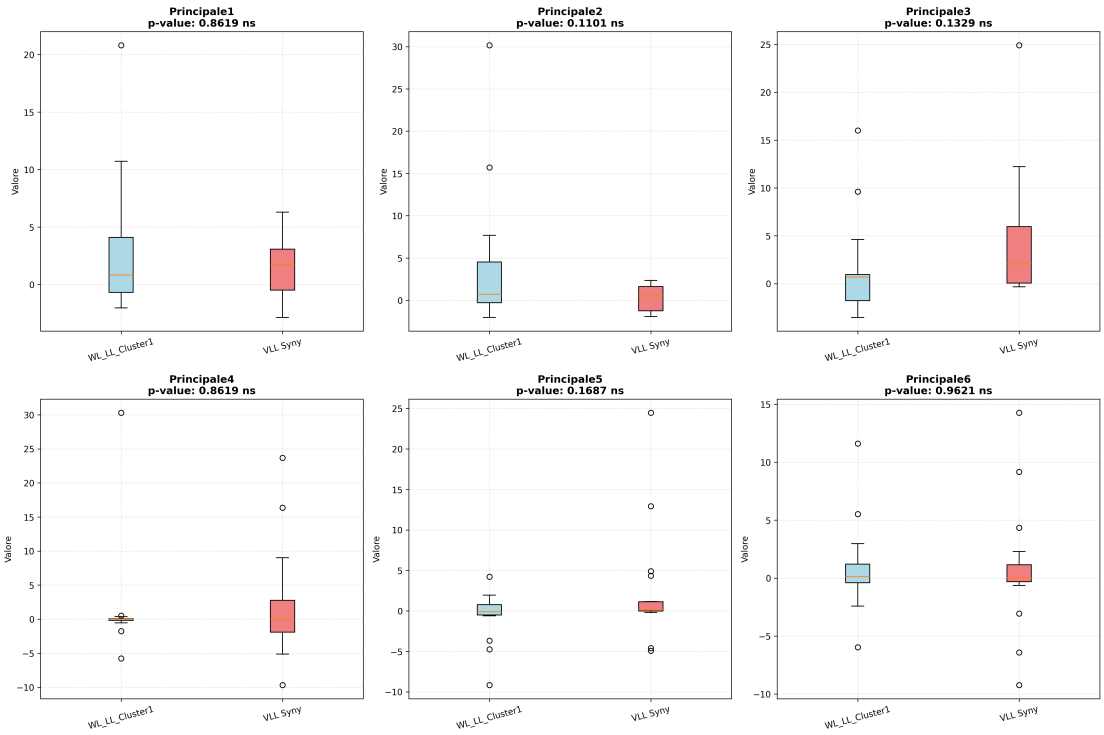


Figura 22: Boxplot comparativi delle distribuzioni dei centroidi (Reale vs Sintetico)

L'esito del test di Wilcoxon è riassunto nella Tabella 2 e visualizzato nel grafico a barre in Figura 23.

Componente	**P-Value**	**Soglia** ($\alpha = 0.05$)	**Esito** (H_0)
Principale 1	0.8619	> 0.05	**NON Rifiutata**
Principale 2	0.1101	> 0.05	**NON Rifiutata**
Principale 3	0.1329	> 0.05	**NON Rifiutata**
Principale 4	0.8619	> 0.05	**NON Rifiutata**
Principale 5	0.1687	> 0.05	**NON Rifiutata**
Principale 6	0.9621	> 0.05	**NON Rifiutata**

Tabella 2: Risultati puntuali del Test di Wilcoxon sui Centroidi

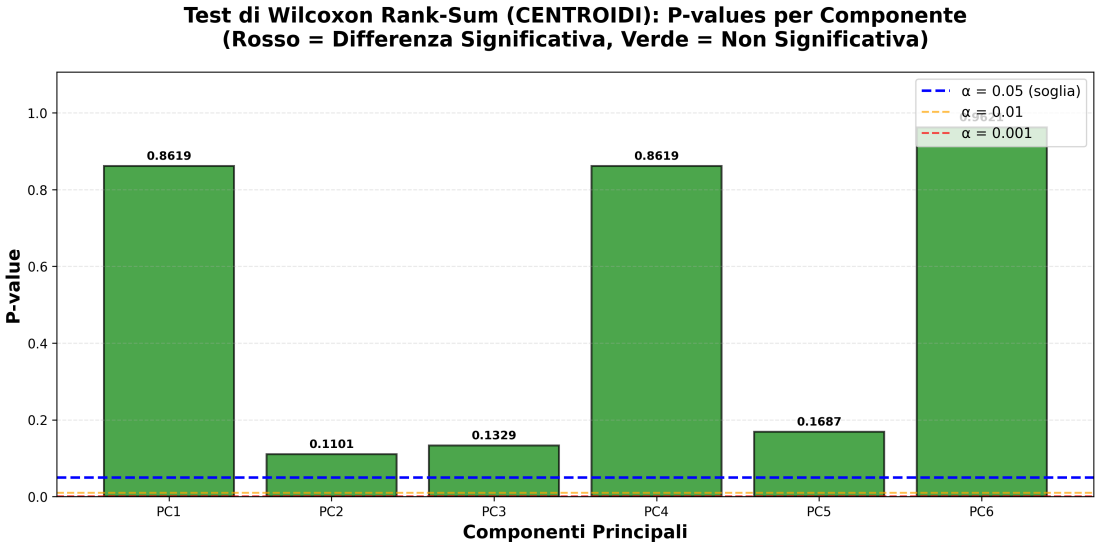


Figura 23: P-values del test di Wilcoxon per componente (La linea blu indica la soglia di significatività)

Conclusioni Come evidenziato dai risultati, per tutte le 6 componenti principali il p-value calcolato è superiore al livello di significatività $\alpha = 0.05$. Ciò significa che non c'è evidenza statistica sufficiente per dire che i due workload siano diversi. Possiamo quindi affermare, con un livello di confidenza del 95%, che **il workload sintetico generato è statisticamente rappresentativo del workload reale**.

4 Regression

I seguenti esercizi rappresentano ognuno l'analisi di una distribuzione di valori monitorati su dei sistemi campione, su ognuo di questi verranno effettuati dei test per tovare un buon modello di regressione che aiuterà a verificare e stimare la presenza di eventuali trend.

4.1 Exp1

Per analizzare la distribuzione di queste componenti è stato necessario effettuare una stima lineare su ogni variabile, andando quindi a cercare almeno visivamente se fossero presenti eventuali trend.

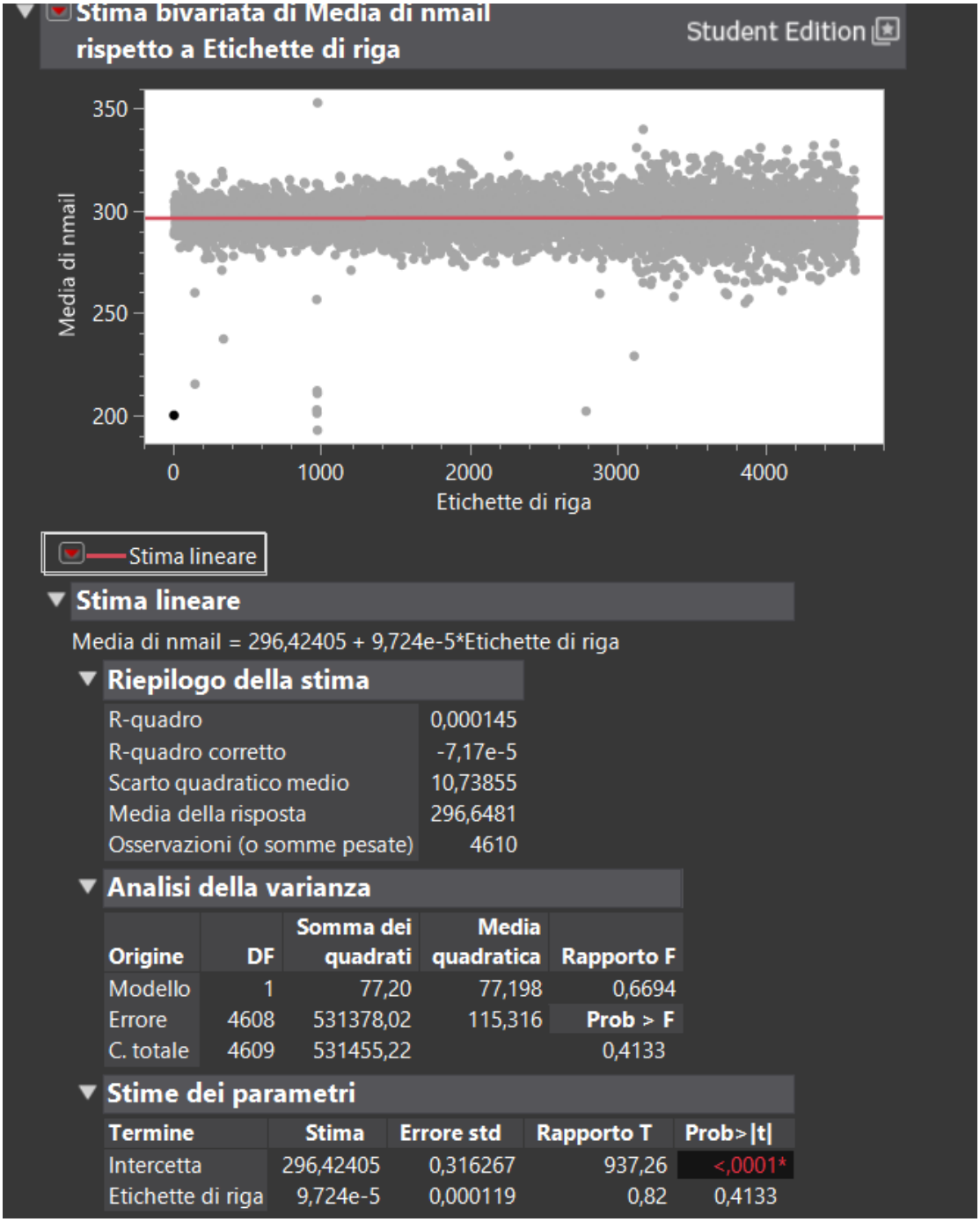


Figura 24: Stima lineare Nmail

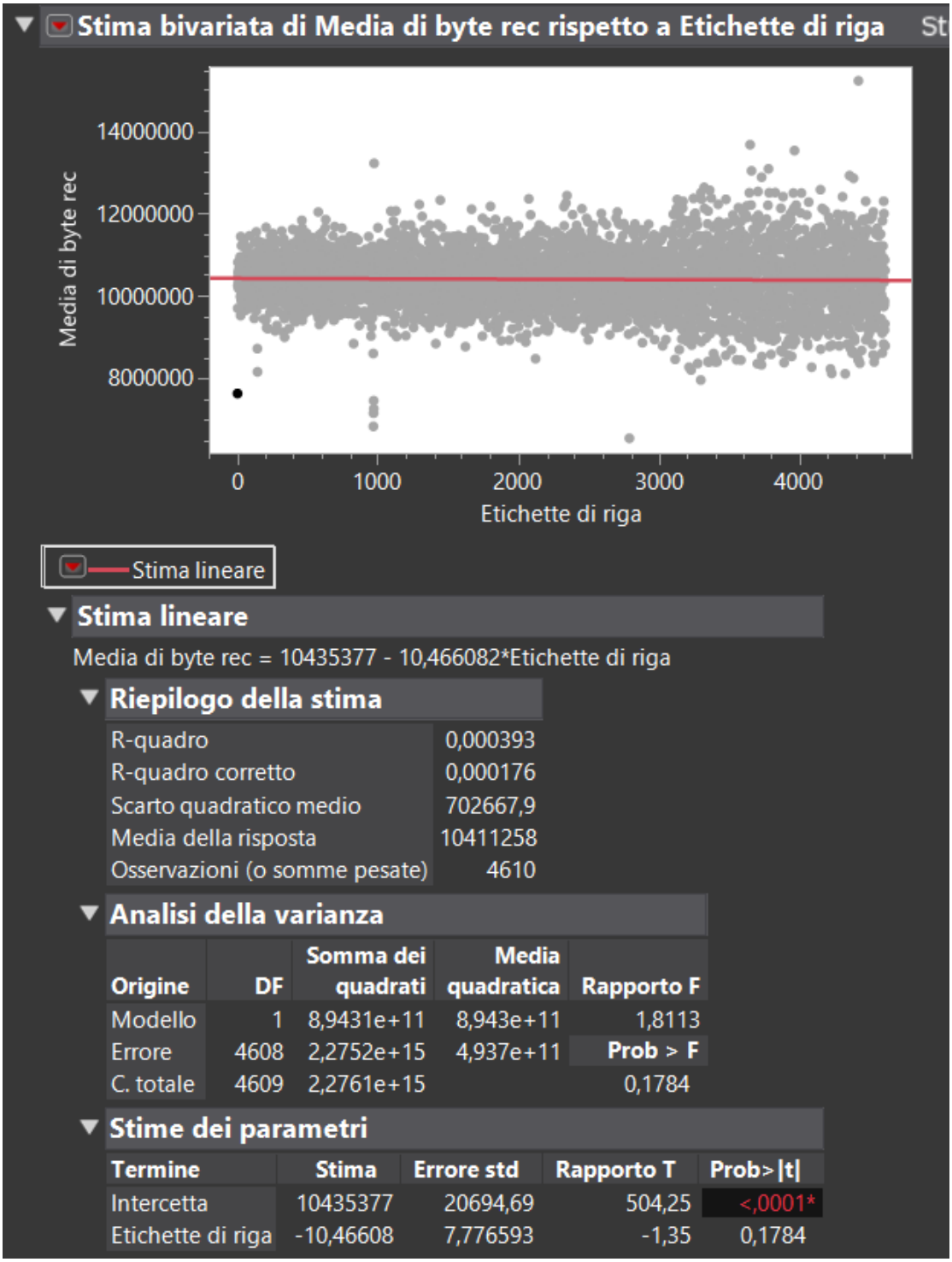


Figura 25: Stima lineare ByteRec

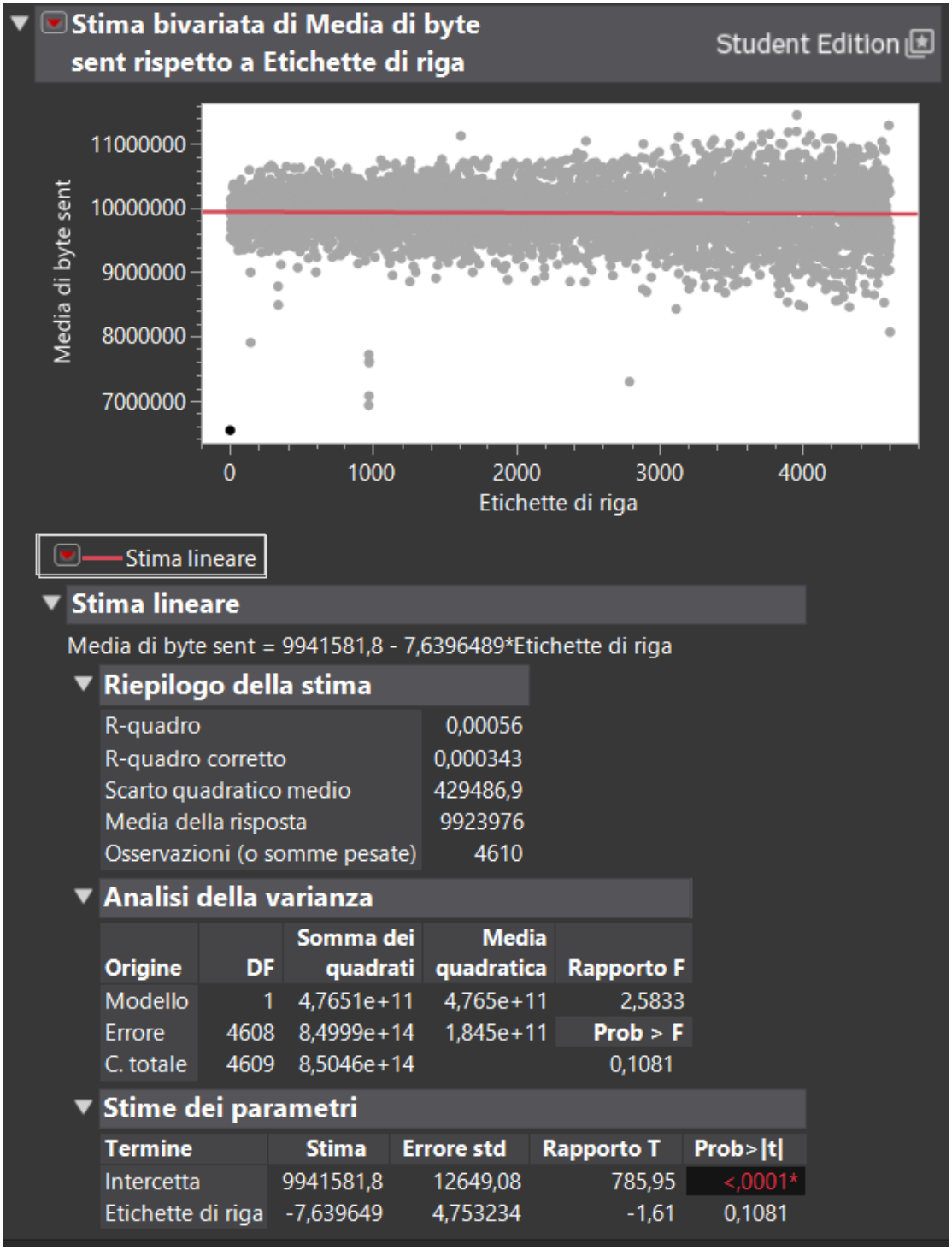


Figura 26: Stima lineare ByteSent

Con degli

$$R^2$$

cosi bassi possiamo vedere che l'adattamento del modello regressivo lineare non è un buon adattamento e quindi i dati non sono lineari.

4.1.1 Test di normalità

Di seguito è stato mostrato un test visivo di normalità con relativo test di ipotesi sui residui della nostra regressione per poter scegliere il giusto test che evidenzia i trend.

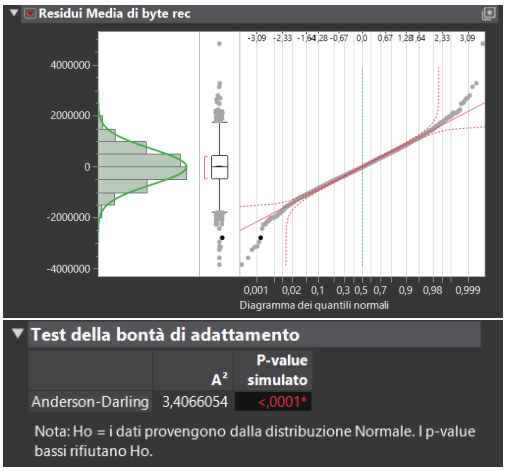


Figura 27: Normalità QQplot e test ipotesi byte rec

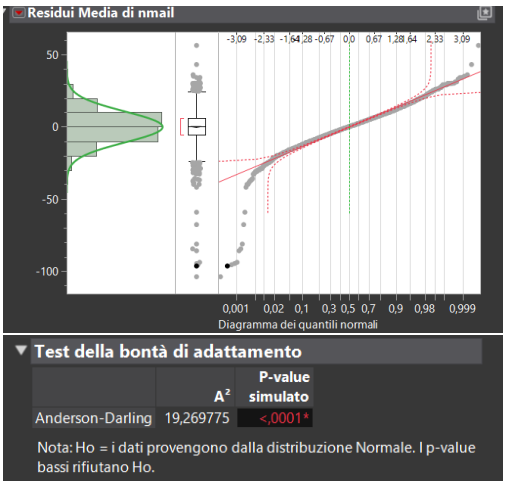


Figura 28: Normalità QQplot e test ipotesi nmail

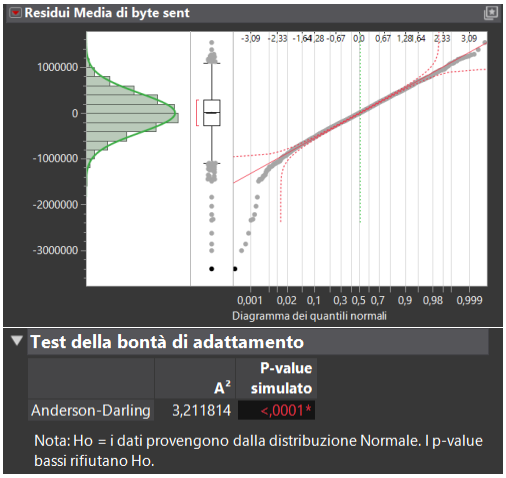


Figura 29: Normalità QQplot e test ipotesi byte sent

I test di normalità indicano che per tutte e tre le variabili i residui non sono normali, si è scelto di effettuare quindi dei test non parametrici, in particolare quello di Man-Kendall

4.1.2 Test di Mann-Kendal

I seguenti test servono a provare la presenza di trend nella distribuzione delle tre variabili.

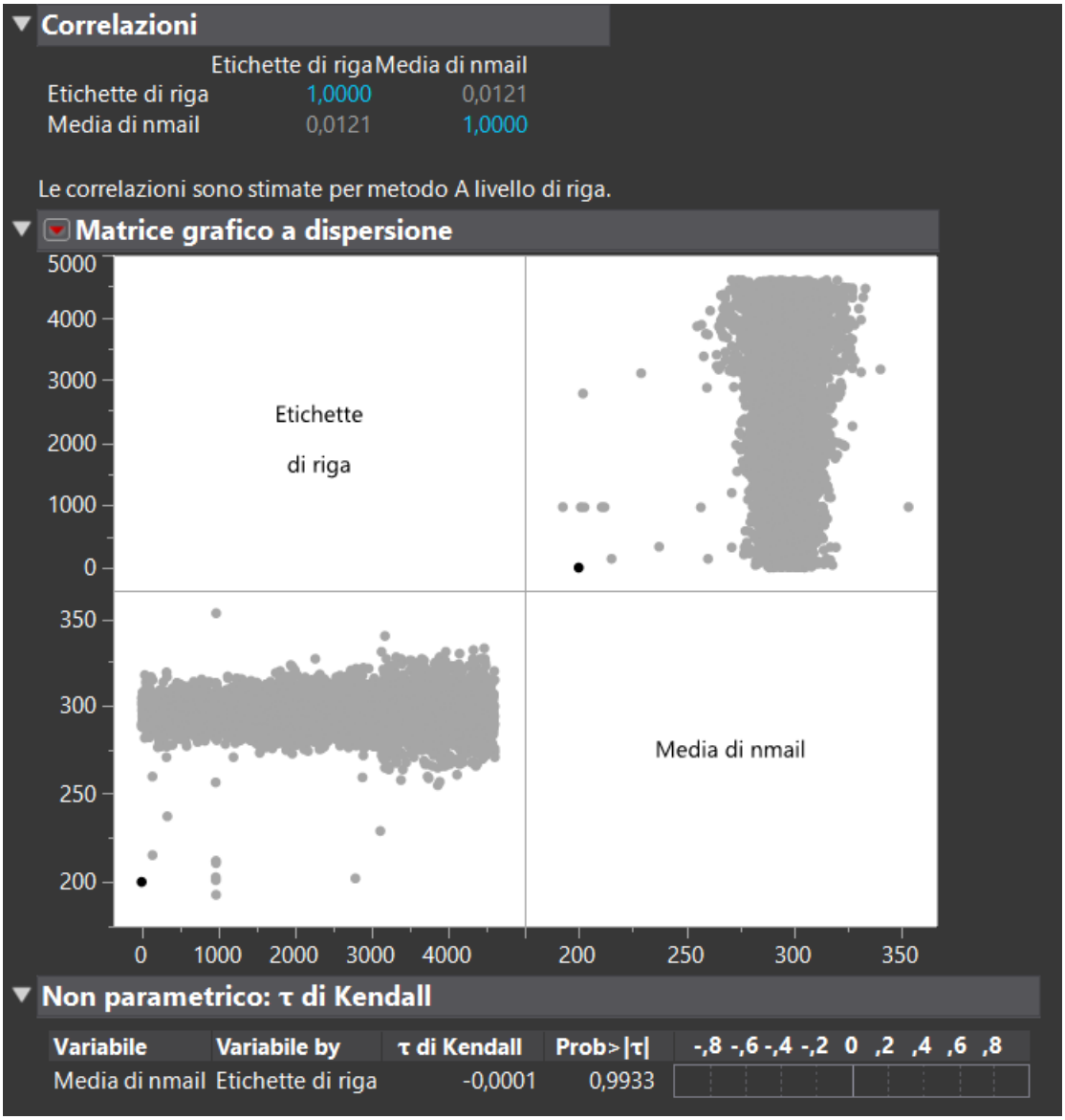


Figura 30: Analisi nmail



Figura 31: Analisi Byte rec

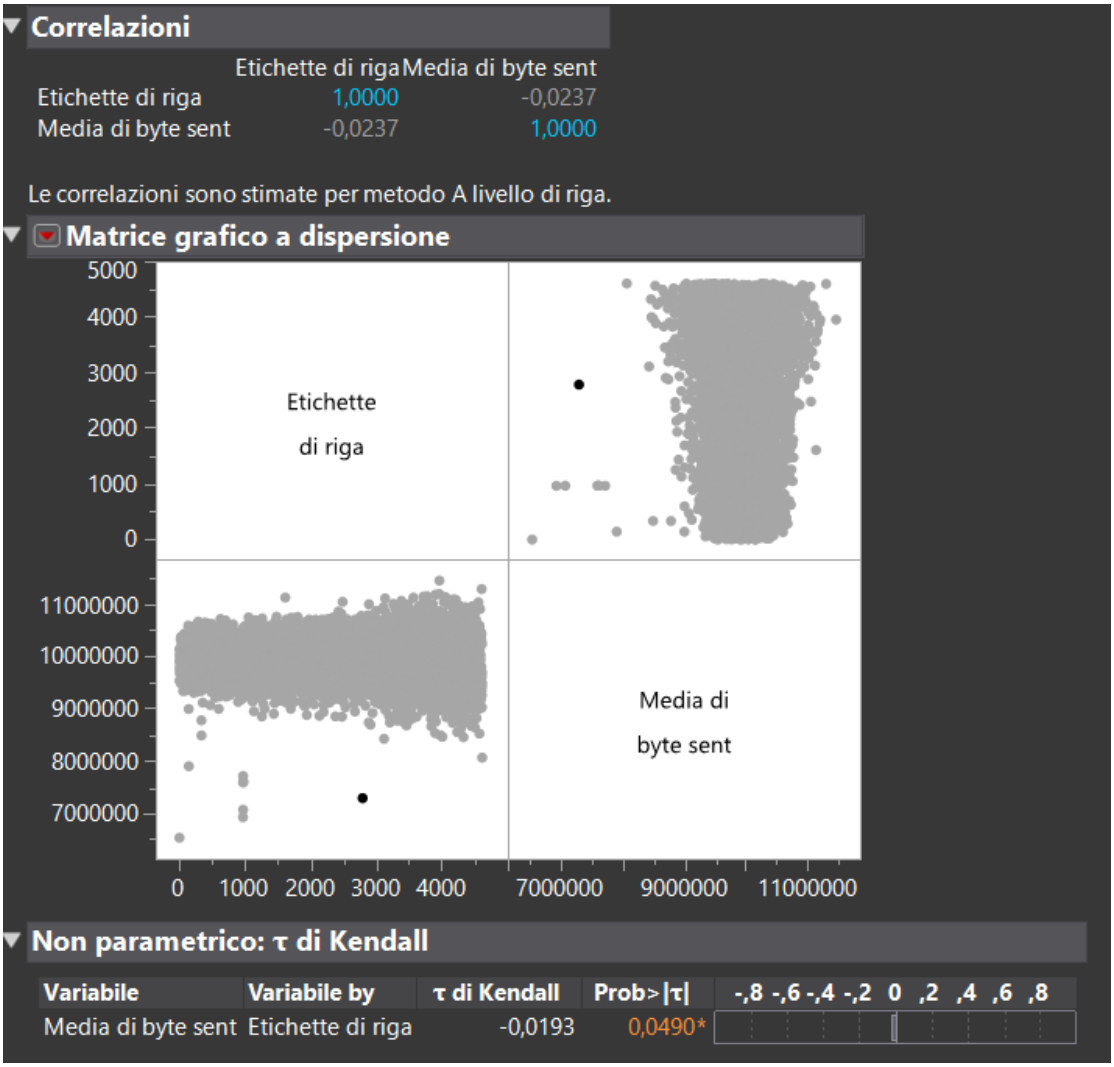


Figura 32: Analisi ByteSent

È possibile notare come la pendenza nulla della retta sia verificata solo nel caso di nmail, mentre negli altri due casi sebbene per poco è statisticamente valido pensare ad un trend nella distribuzione.

4.1.3 Theil - Sen

Per calcolare l'effettivo valore delle pendenze è stata applicata la procedura di Thail-sen tramite uno script python, i risultati sono esposti qui di seguito.

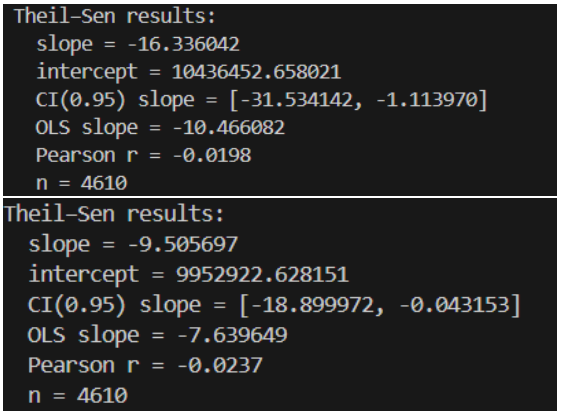


Figura 33: Theil - Sen per Byte Rec(Sopra) e Byte Sent (Sotto)

4.2 Exp2

Similmente a quanto fatto per il primo esperimento l'analisi si concentra sulle tre variabili: ByteRec, ByteSent e Nmail.

4.2.1 Analisi ByteRec

Stima lineare Dal grafico della stima lineare osserviamo un coefficiente R^2 estremamente basso, pari a circa 0.012. Deduciamo che il modello lineare non spiega adeguatamente la varianza dei dati e che la relazione tra le variabili è molto debole o disturbata da outliers.

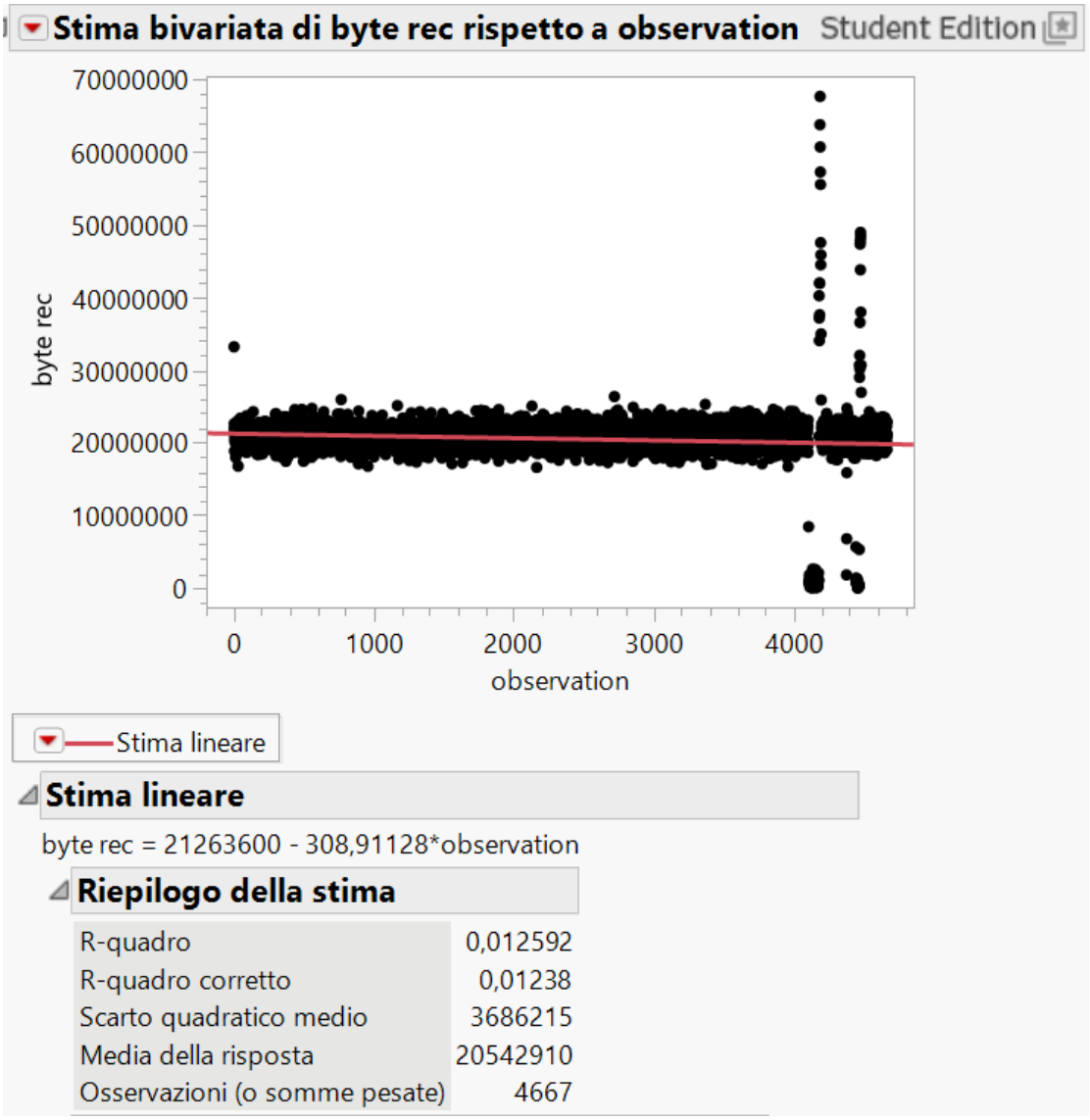


Figura 34: Stima lineare ByteRec (Exp2)

Test di normalità L'analisi dei residui conferma l'inadeguatezza del modello parametrico. Osservando il Q-Q plot, è evidente una forte deviazione dalla normalità, specialmente sulle code della distribuzione. Inoltre effettuando un plot dei residui rispetto alla risposta predetta notiamo chiaramente che i residui non sono omoschedastici

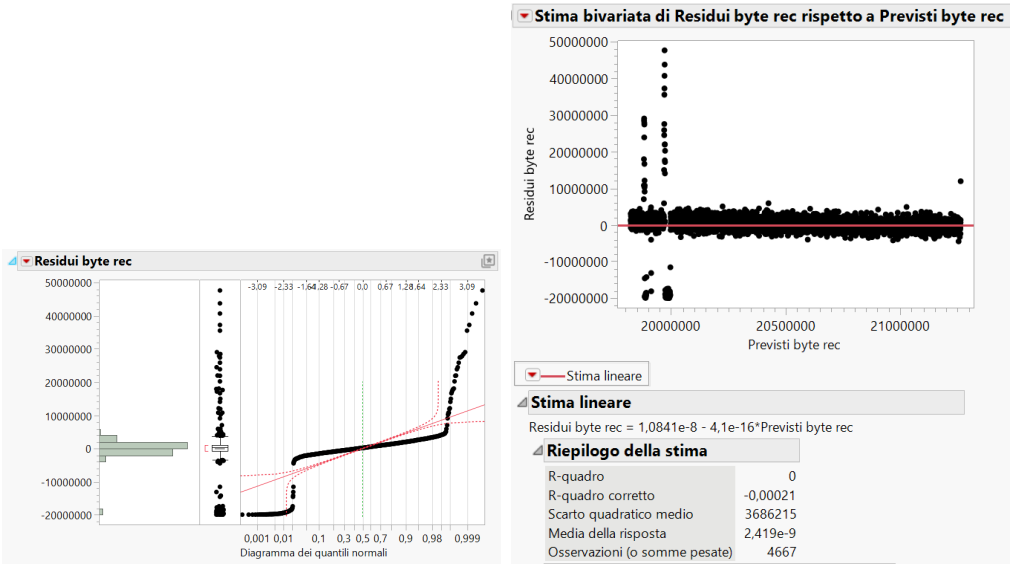


Figura 35: Analisi dei residui: Q-Q plot e residui vs risposta predetta

Tali conclusioni ci obbligano a rifiutare le conclusioni della regressione lineare standard e a procedere con metodi non parametrici.

Test di Mann-Kendall e Stima di Theil-Sen Applicando il test di Mann-Kendall otteniamo un τ di Kendall pari a -0.0316 e un p-value di 0.0012. Poiché il p-value è inferiore alla soglia di significatività $\alpha = 0.05$, **rigettiamo l'ipotesi nulla**: esiste un trend statisticamente significativo nella serie temporale, seppur di lieve entità.

Per quantificare questo trend, utilizziamo lo stimatore di Theil-Sen, che è meno sensibile agli outlier rispetto alla regressione dei minimi quadrati.

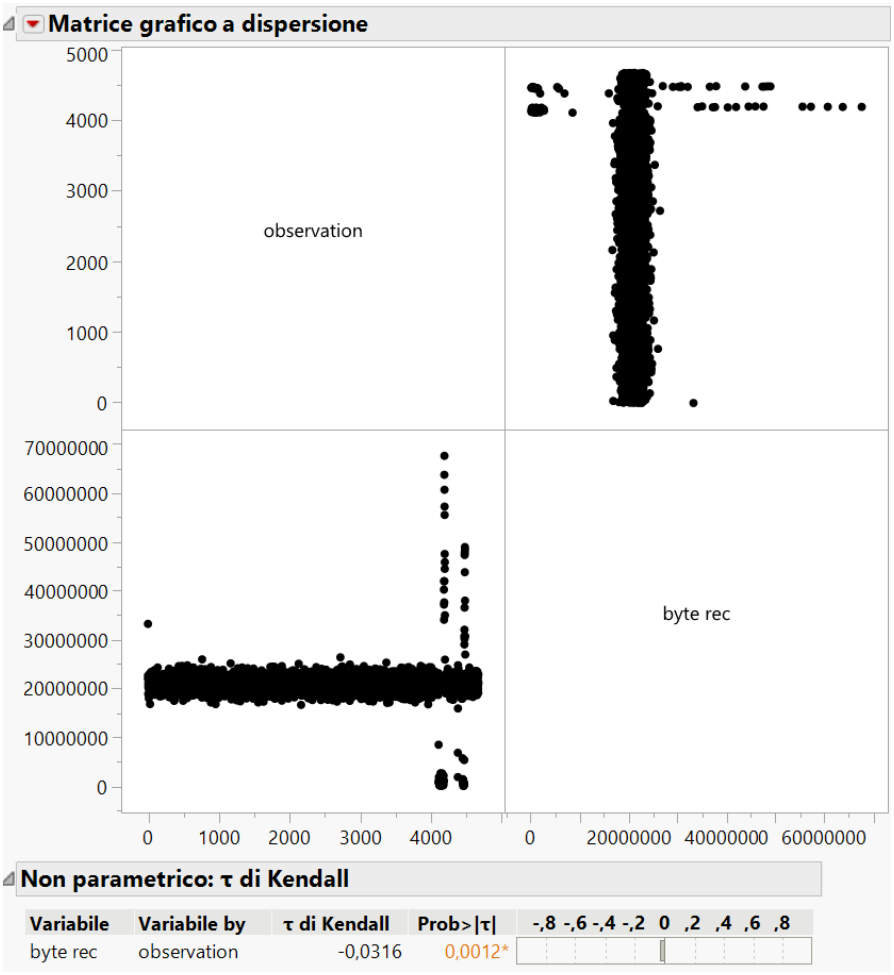


Figura 36: Matrice di dispersione e risultati Mann-Kendall

Risultati Regressione Theil-Sen

Parametro	Valore
Pendenza (slope)	-48.293610
Intercetta	20890131.9914
IC 95% - Limite Inferiore	-77.547733
IC 95% - Limite Superiore	-18.993147
Significatività statistica	Sì
Interpretazione	Trend significativo (Negativo)

Figura 37: Matrice di dispersione e risultati Mann-Kendall

In conclusione, i Byte Ricevuti mostrano una lievissima tendenza alla diminuzione nel tempo (trend negativo), confermata statisticamente dai test non parametrici.

4.2.2 Analisi ByteSent

Stima lineare Anche per i Byte Inviati, dal grafico della stima lineare osserviamo un coefficiente R^2 molto basso (0.034). Deduciamo che il modello lineare non spiega adeguatamente la varianza dei dati e che la relazione è influenzata da forti outliers.

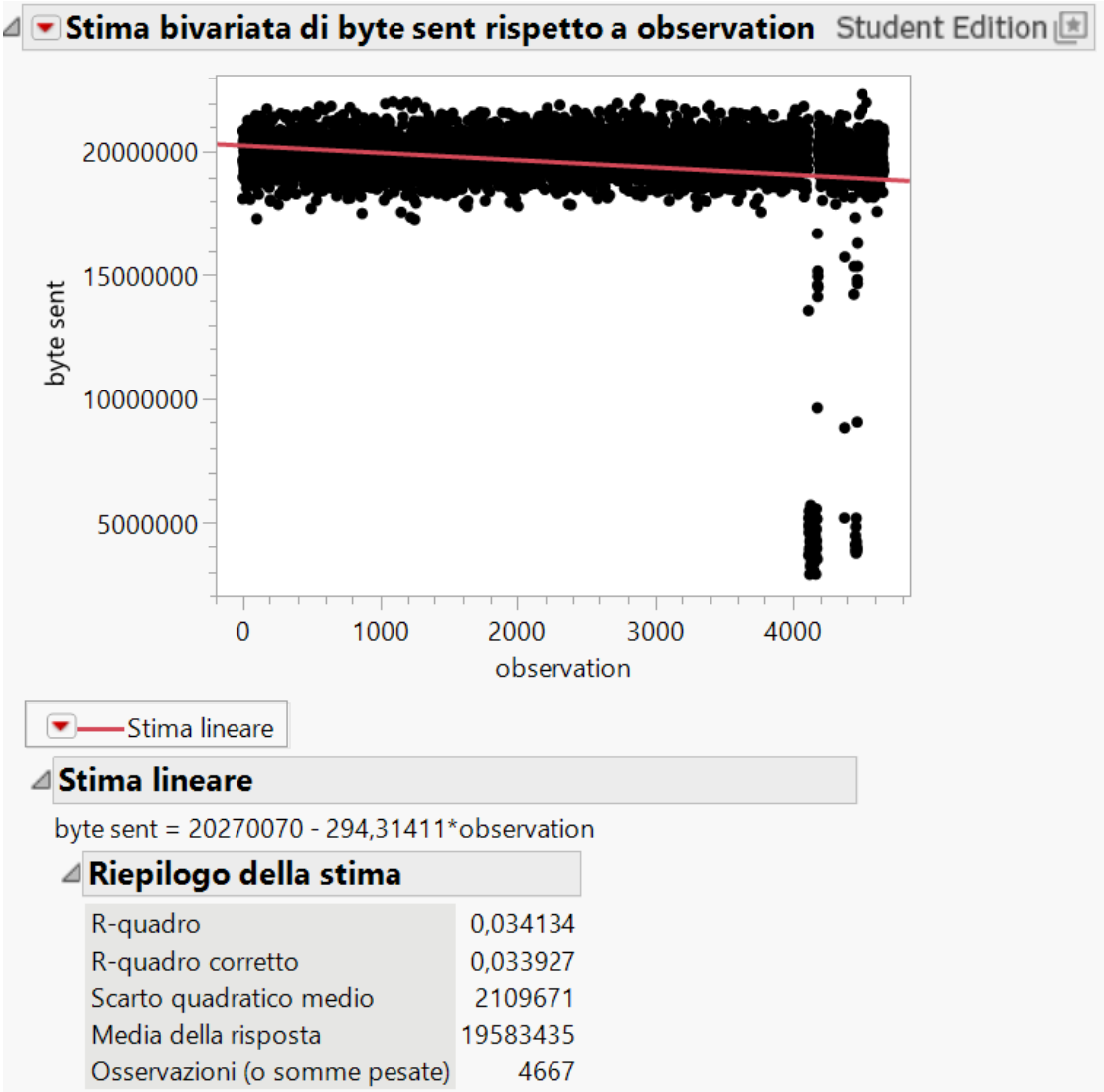


Figura 38: Stima lineare ByteSent (Exp2)

Test di normalità L'analisi dei residui conferma l'inadeguatezza del modello parametrico. Il Q-Q plot mostra una evidente forma a "S" (code pesanti) che indica non normalità. Inoltre, effettuando un plot dei residui rispetto alla risposta predetta, notiamo chiaramente che i residui non sono omoschedastici, evidenziando picchi di varianza anomala.

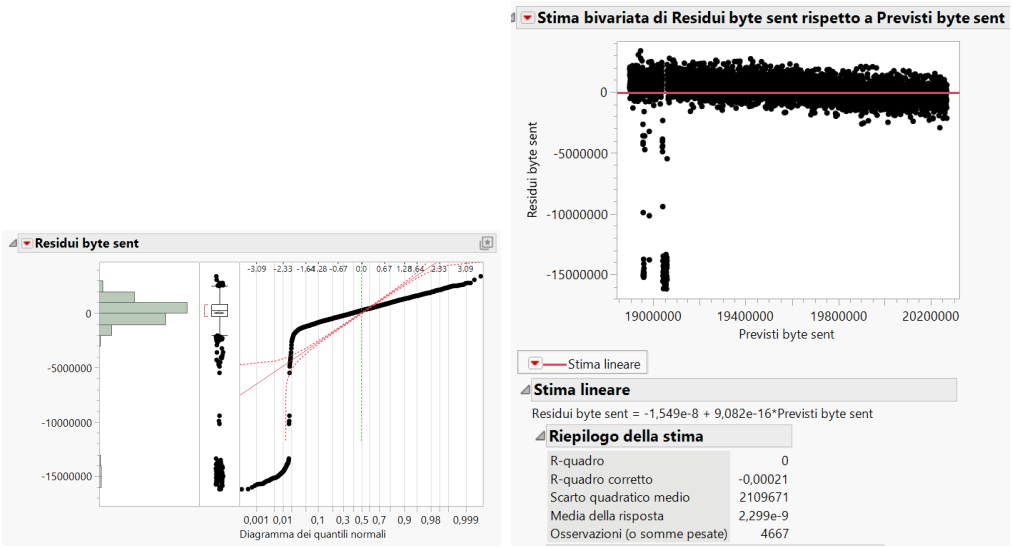


Figura 39: Analisi dei residui: Q-Q plot e residui vs risposta predetta

Tali conclusioni ci obbligano a rifiutare le conclusioni della regressione lineare standard e a procedere con metodi non parametrici.

Test di Mann-Kendall e Stima di Theil-Sen Applicando il test di Mann-Kendall otteniamo un τ di Kendall pari a -0.0392 e un p-value < 0.0001. Poiché il p-value è ampiamente inferiore alla soglia di significatività $\alpha = 0.05$, **rigettiamo l'ipotesi nulla**, confermando la presenza di un trend decrescente statisticamente significativo.

Per quantificare questo trend, utilizziamo lo stimatore di Theil-Sen.

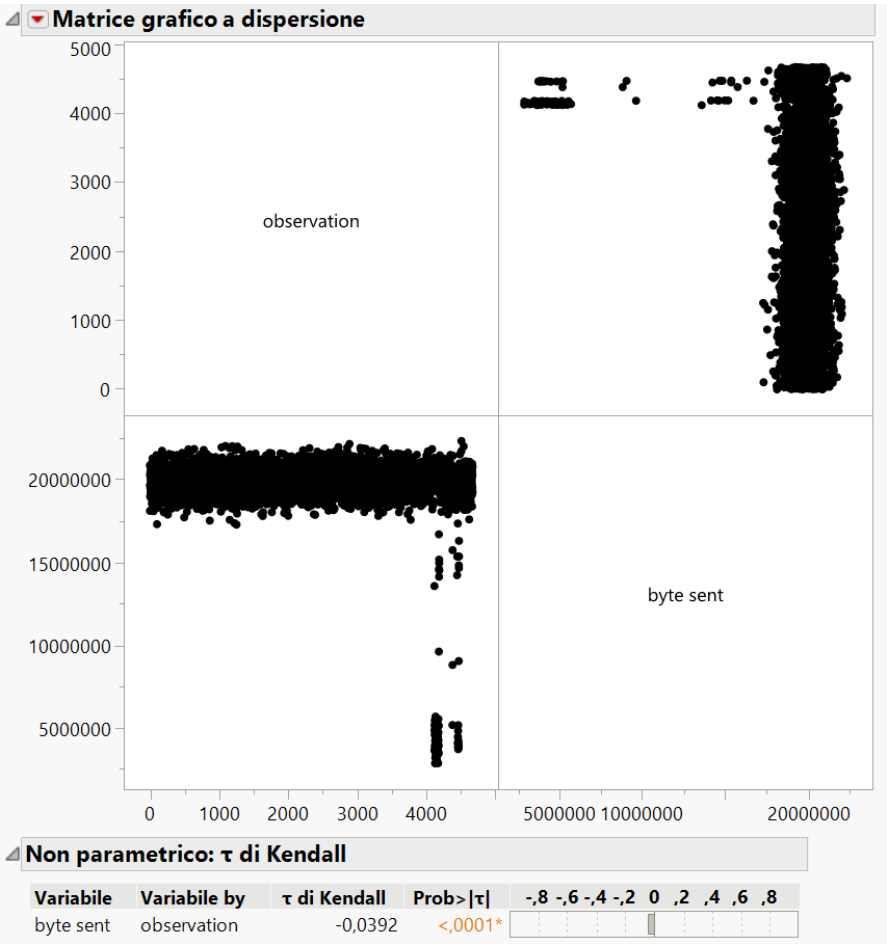


Figura 40: Matrice di dispersione e risultati Mann-Kendall ByteSent

Risultati Regressione Theil-Sen

Parametro	Valore
Pendenza (slope)	-34.063052
Intercetta	19903064.1005
IC 95% - Limite Inferiore	-50.672673
IC 95% - Limite Superiore	-17.374088
Significatività statistica	Sì
Interpretazione	Trend significativo (Negativo)

Figura 41: Risultati stima Theil-Sen ByteSent

In conclusione, anche per i Byte Inviati si rileva una tendenza alla diminuzione nel tempo, analoga a quella riscontrata per i Byte Ricevuti.

4.2.3 Analisi Nmail

Stima lineare Per la variabile Nmail, osserviamo un coefficiente R^2 estremamente basso (0.02). Deduciamo che il modello lineare è inadatto.

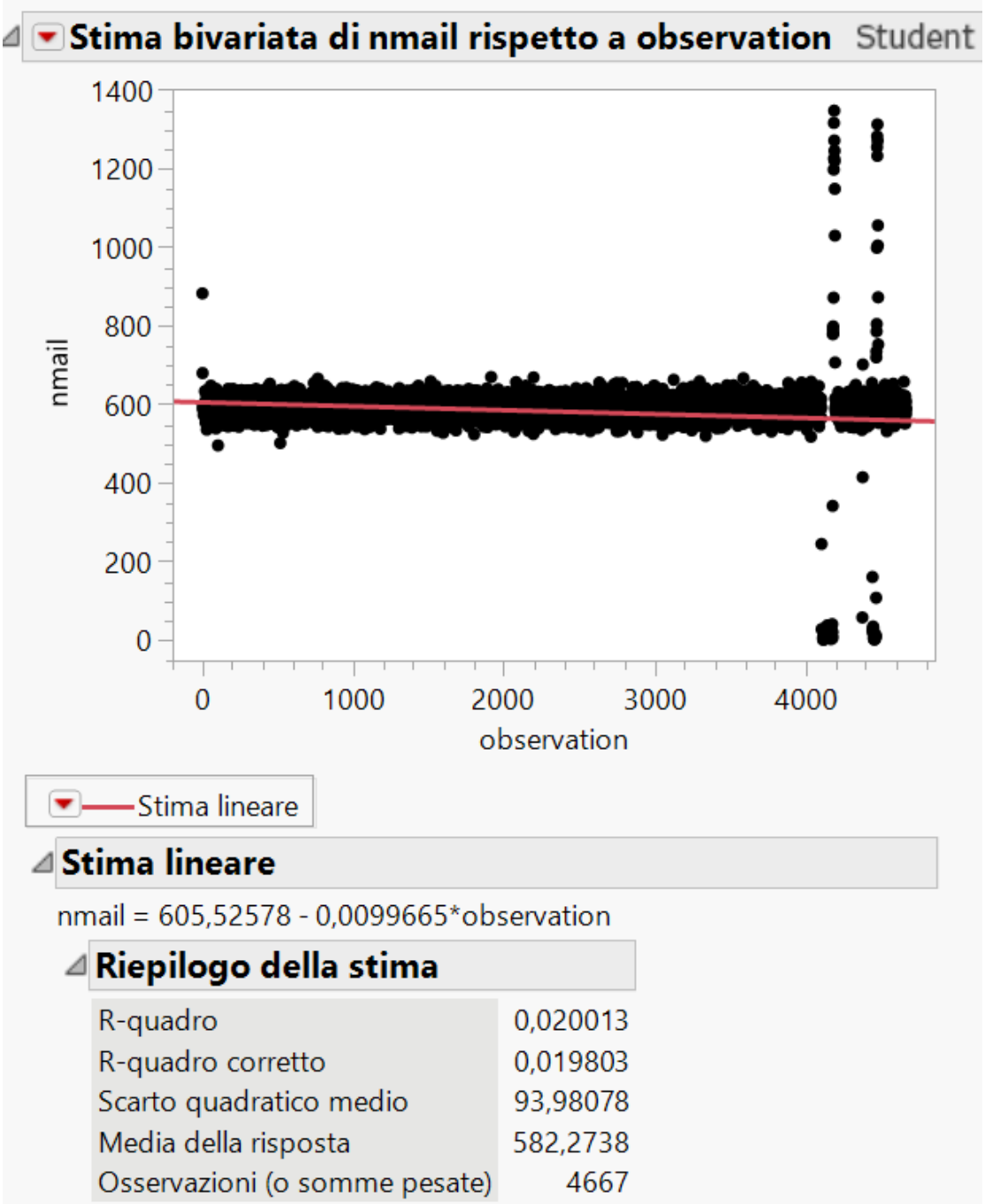


Figura 42: Stima lineare Nmail (Exp2)

Test di normalità L'analisi dei residui conferma l'inadeguatezza del modello parametrico. Osservando il Q-Q plot, è evidente una deviazione dalla normali-

tà. Inoltre effettuando un plot dei residui rispetto alla risposta predetta notiamo chiaramente eteroschedasticità causata dai picchi di attività anomala.

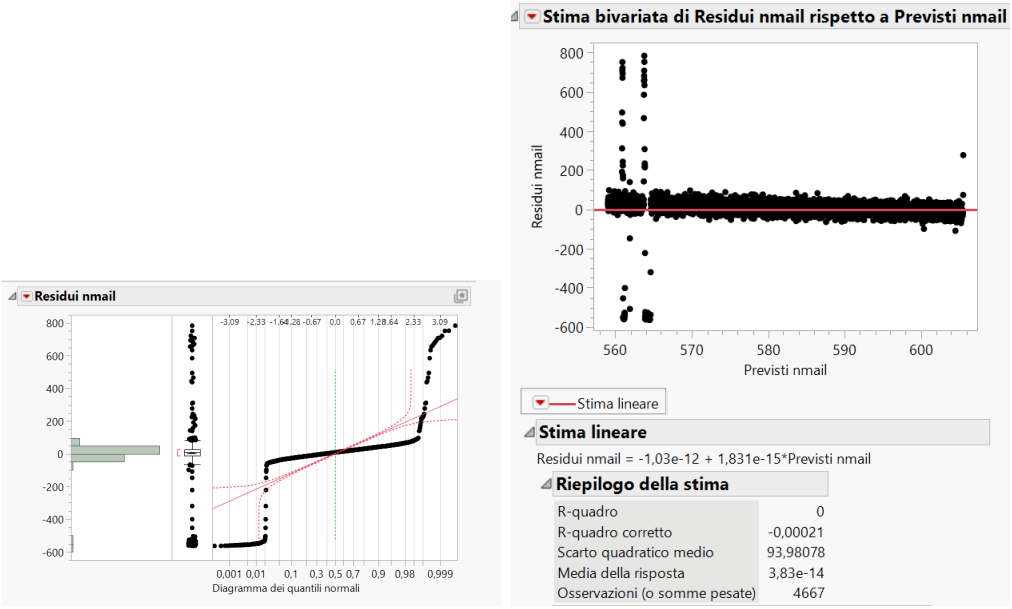


Figura 43: Analisi dei residui: Q-Q plot e residui vs risposta predetta

Tali conclusioni ci obbligano a rifiutare le conclusioni della regressione lineare standard e a procedere con metodi non parametrici.

Test di Mann-Kendall e Stima di Theil-Sen Applicando il test di Mann-Kendall otteniamo un τ di Kendall pari a -0.0242 e un p-value di 0.0138. Anche in questo caso il p-value è inferiore alla soglia $\alpha = 0.05$.

Utilizzando lo stimatore di Theil-Sen, notiamo che la pendenza stimata è praticamente nulla e l'intervallo di confidenza tocca lo zero, portando a non considerare il trend rilevante dal punto di vista pratico.

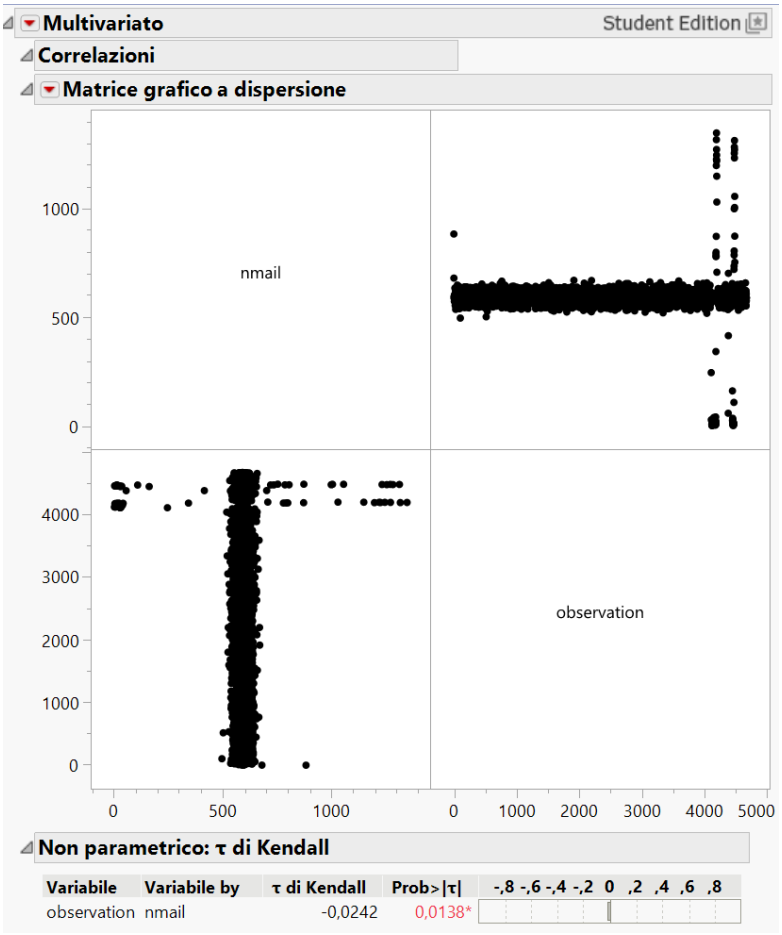


Figura 44: Matrice di dispersione e risultati Mann-Kendall Nmail

Risultati Regressione Theil-Sen

Parametro	Valore
Pendenza (slope)	-0.000597
Intercetta	592.3937
IC 95% - Limite Inferiore	-0.001125
IC 95% - Limite Superiore	0.000000
Significatività statistica	No
Interpretazione	Nessun trend significativo

Figura 45: Risultati stima Theil-Sen Nmail

In conclusione, a differenza delle altre variabili, per Nmail **non si rileva un trend pratico rilevante**, e il sistema può considerarsi stabile nel tempo.

4.3 VMres1

4.3.1 Stima lineare

Calcolando la stima lineare di questa distribuzione di dati è comparso un trend con pendenza positiva.

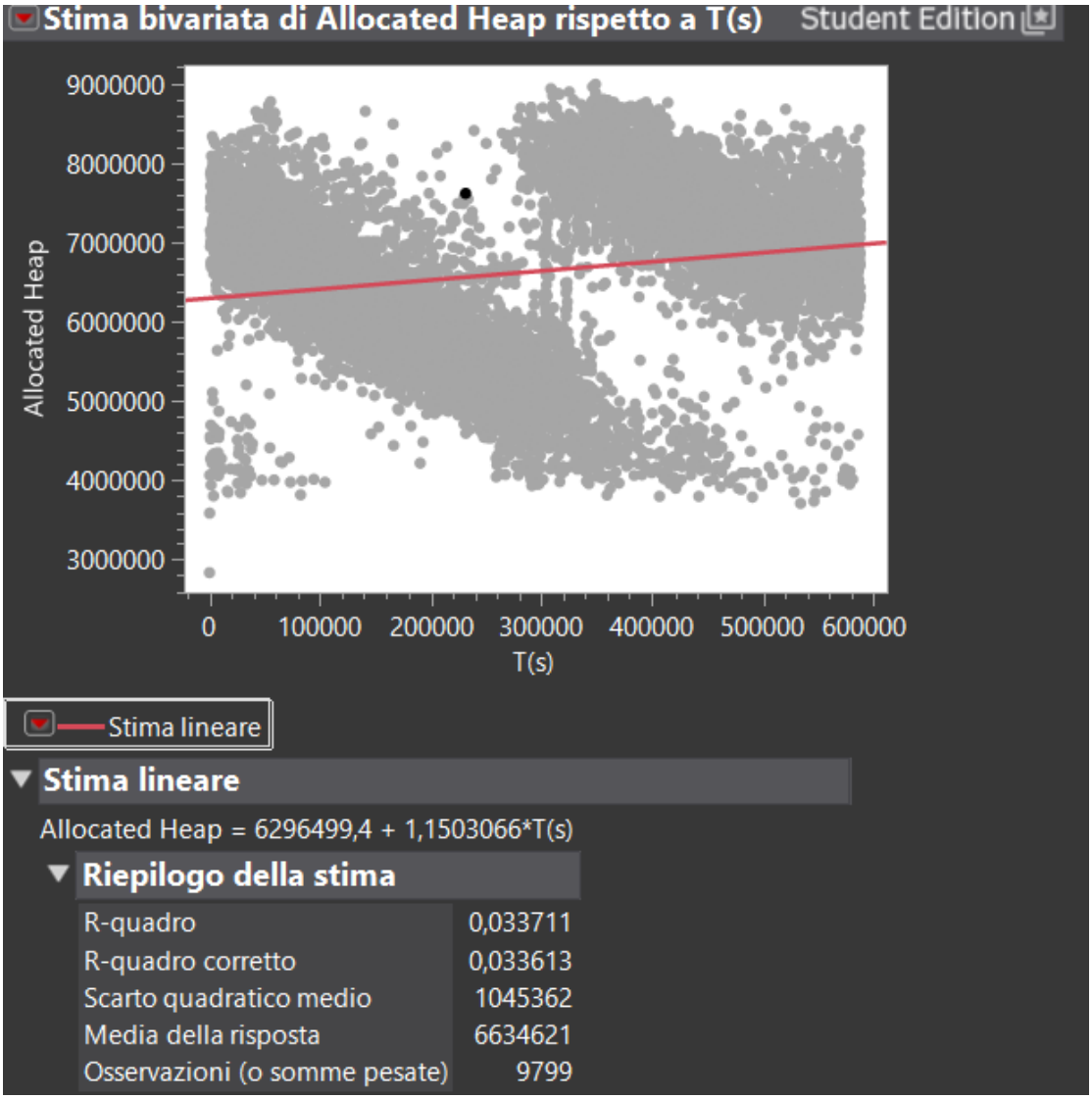


Figura 46: Stima lineare

4.3.2 Controllo della normalità

Successivamente abbiamo recuperato i residui della stima regressiva e ne abbiamo controllato la normalità.

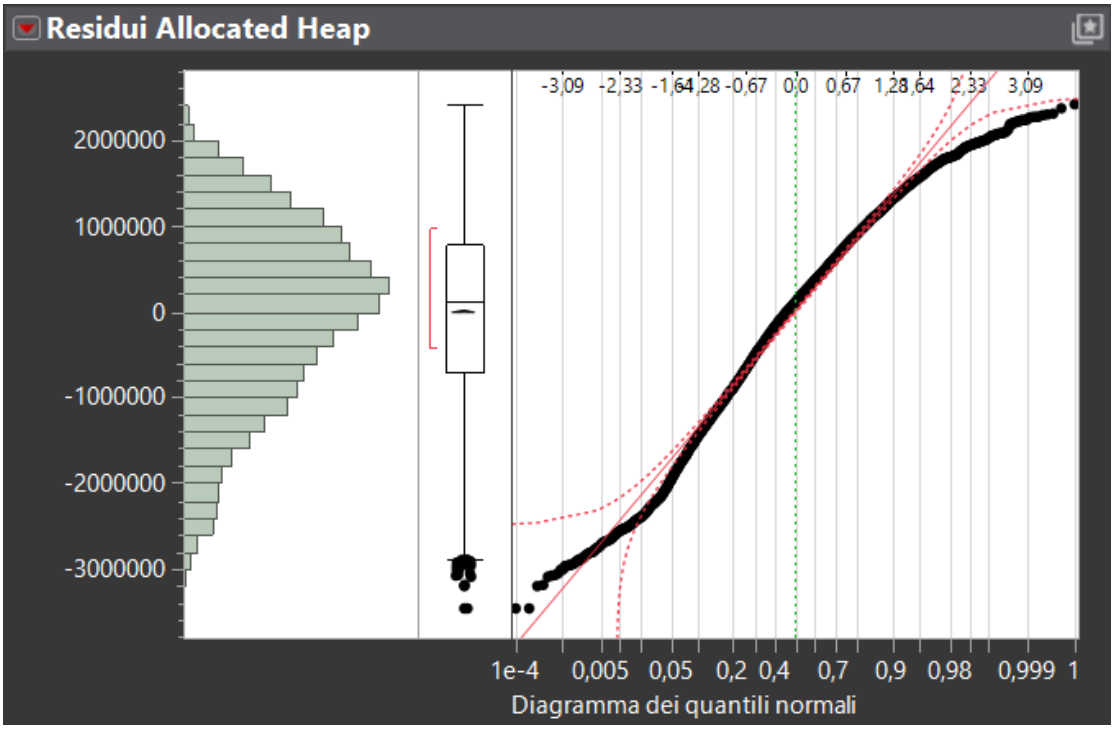


Figura 47: Test visivo Normalità

Come si evince dal test visivo la normalità non è riscontrata.

4.3.3 Test di Mann-Kendall

Data la non normaltà effettuiamo il test di Mann-Kendall per assicurarci che sia presente effettivamente una retta monotona crescente con pendenza significativa

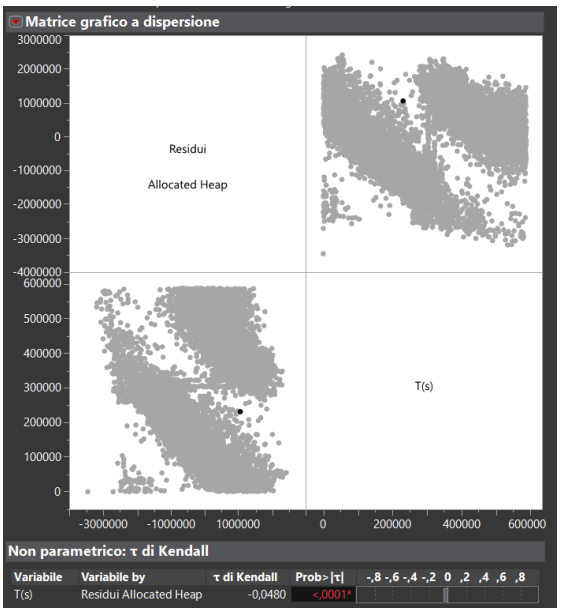


Figura 48: Analisi Multivariata e Mann-Kendall

Il p-value del test è minore di 0.05 quindi il test ha rigettato l'ipotesi nulla (Retta con pendenza costante)

4.3.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G

Una volta che abbiamo confermato la presenza di un trend possiamo calcolarci la pendenza della retta in modo più preciso usando la Stima di Theuk-Sen tramite uno script python.

```
Theil-Sen results:
  slope = 0.691122
  intercept = 6562275.461670
  CI(0.95) slope = [0.563777, 0.817824]
  OLS slope = 1.150307
  Pearson r = 0.1836
  n = 9799
```

Figura 49: Valori retta di regressione

La retta di regressione ha quindi come equazione:
$HeapAllocata = 6562275.462 + (0.691 \pm 0.13) * T(s)$
La formula per calcolare il tempo di saturazione della heap segue facilmente:

$$T(s) = \frac{10^9 - 6562275.462}{0.691 \pm 0.13} = (45.6 \pm 7.1) anni$$

4.4 VMres2

4.4.1 Stima lineare

Calcolando la stima lineare di questa distribuzione di dati è comparso un trend con pendenza positiva.

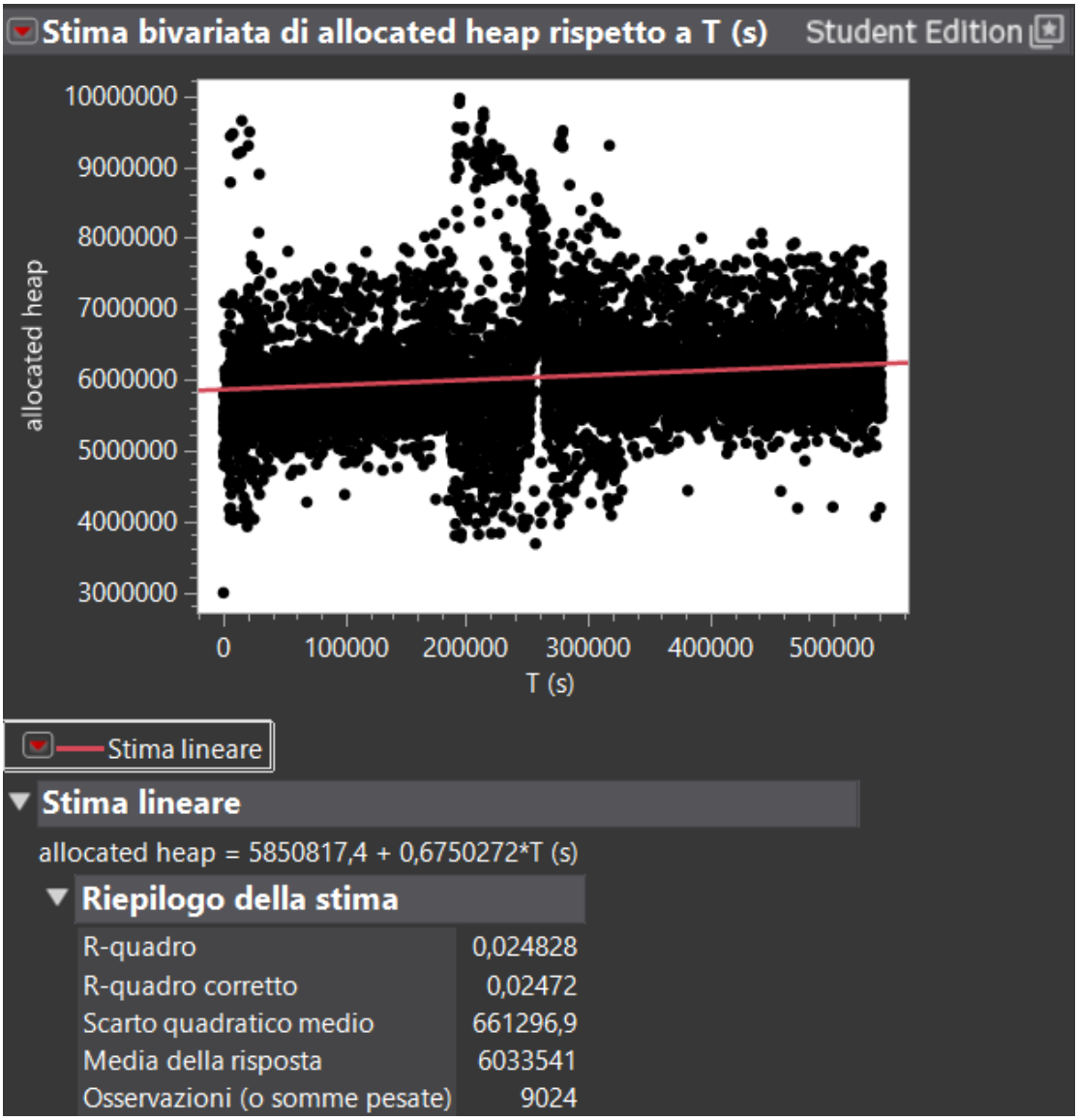


Figura 50: Stima lineare

4.4.2 Controllo della normalità

Successivamente abbiamo recuperato i residui della stima regressiva e ne abbiamo controllato la normalità.

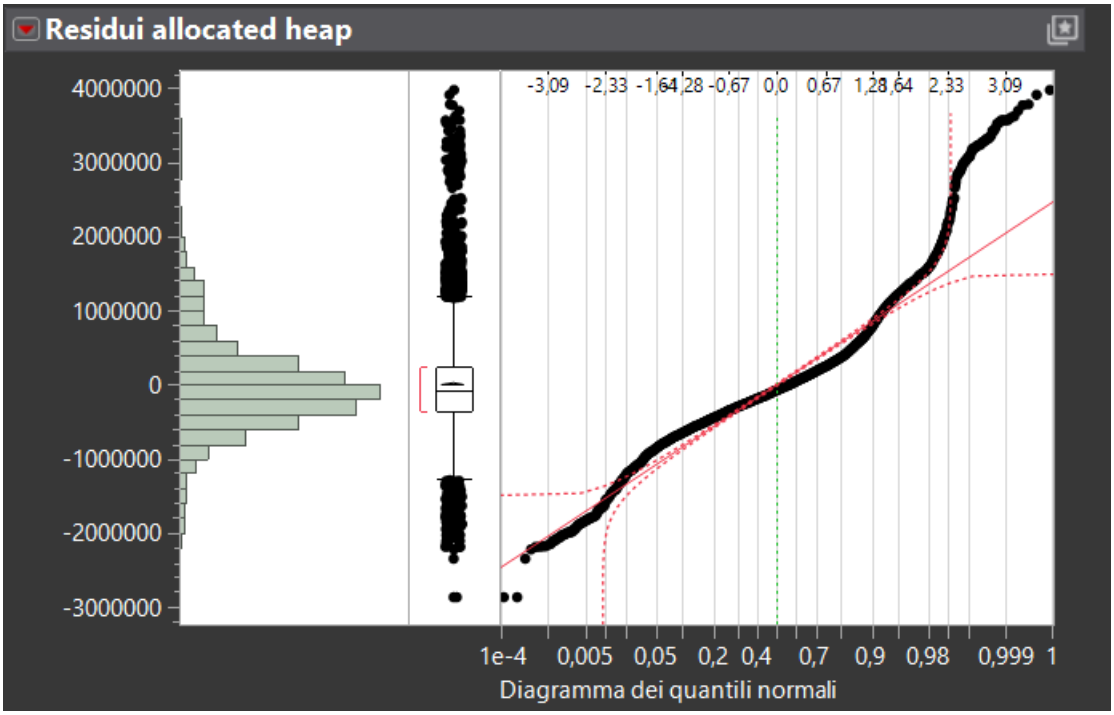


Figura 51: Test visivo Normalità

Come si evince dal test visivo la normalità non è riscontrata.

4.4.3 Test di Mann-Kendall

Data la non normaltà effettuiamo il test di Mann-Kendall per assicurarci che sia presente effettivamente una retta monotona crescente con pendenza significativa

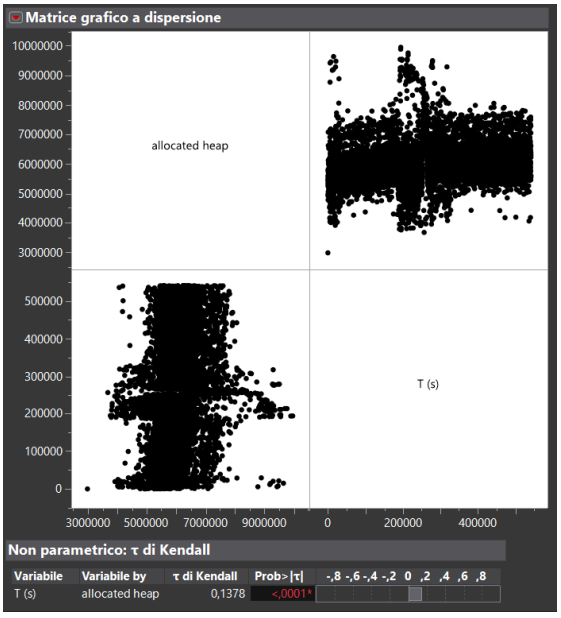


Figura 52: Analisi Multivariata e Mann-Kendall

Il p-value del test è minore di 0.05 quindi il test ha rigettato l'ipotesi nulla (Retta con pendenza costante)

4.4.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G

Una volta che abbiamo confermato la presenza di un trend possiamo calcolarci la pendenza della retta in modo più preciso usando la Stima di Theuk-Sen tramite uno script python.

```
Theil-Sen results:
  slope = 0.670950
  intercept = 5780180.563325
  CI(0.95) slope = [0.604864, 0.736654]
  OLS slope = 0.675027
  Pearson r = 0.1576
  n = 9024
```

Figura 53: Valori retta di regressione

La retta di regressione ha quindi come equazione:
$HeapAllocata = 5780180.563 + (0.671 \pm 0.07) * T(s)$
La formula per calcolare il tempo di saturazione della heap segue facilmente:

$$T(s) = \frac{10^9 - 5780180.563}{0.671 \pm 0.07} = (45.6 \pm 1.1) anni$$

4.5 VMres3

4.5.1 Stima lineare

Calcolando la stima lineare di questa distribuzione di dati è comparso un trend con pendenza positiva.

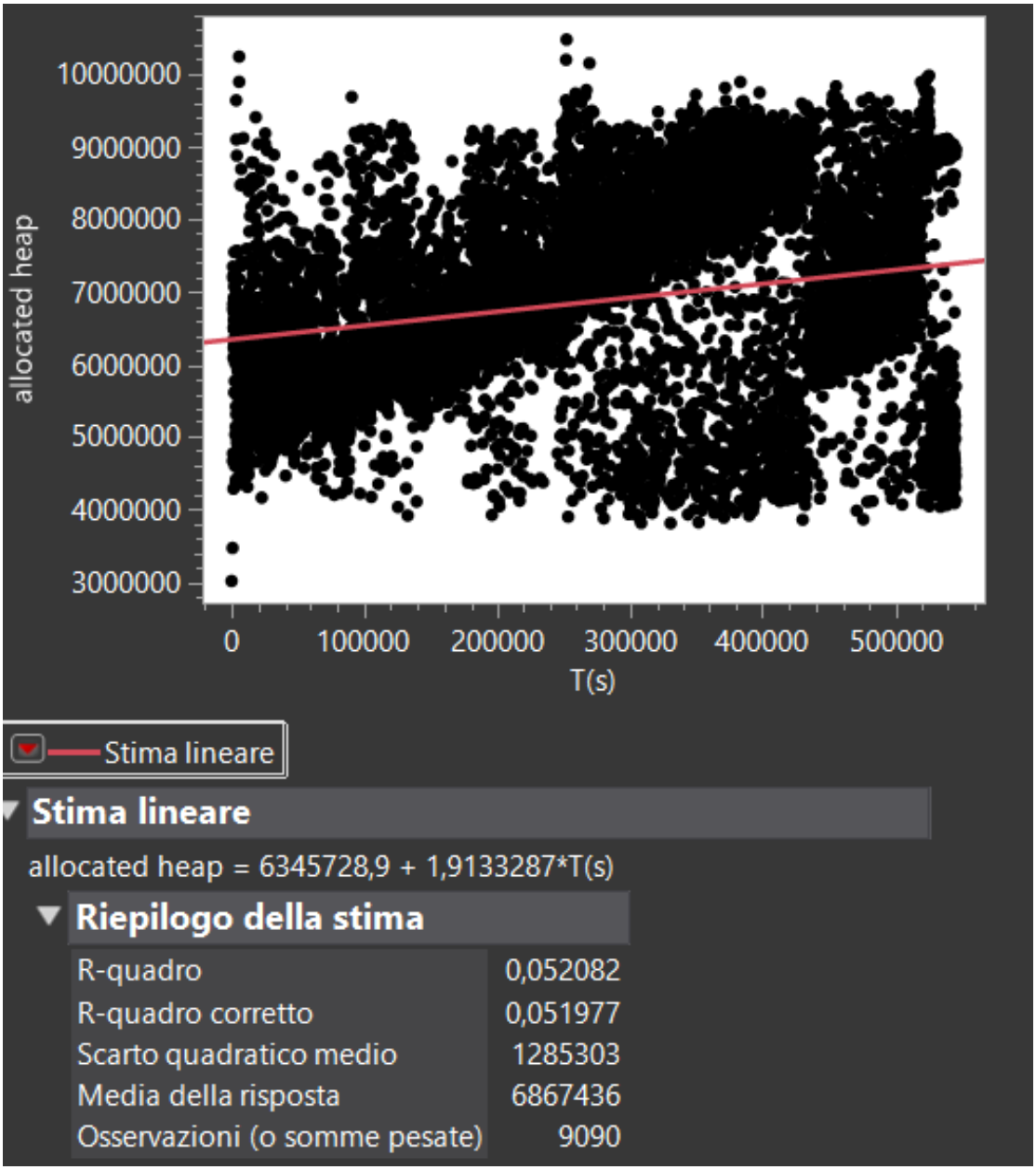


Figura 54: Stima lineare

4.5.2 Controllo della normalità

Successivamente abbiamo recuperato i residui della stima regressiva e ne abbiamo controllato la normalità.

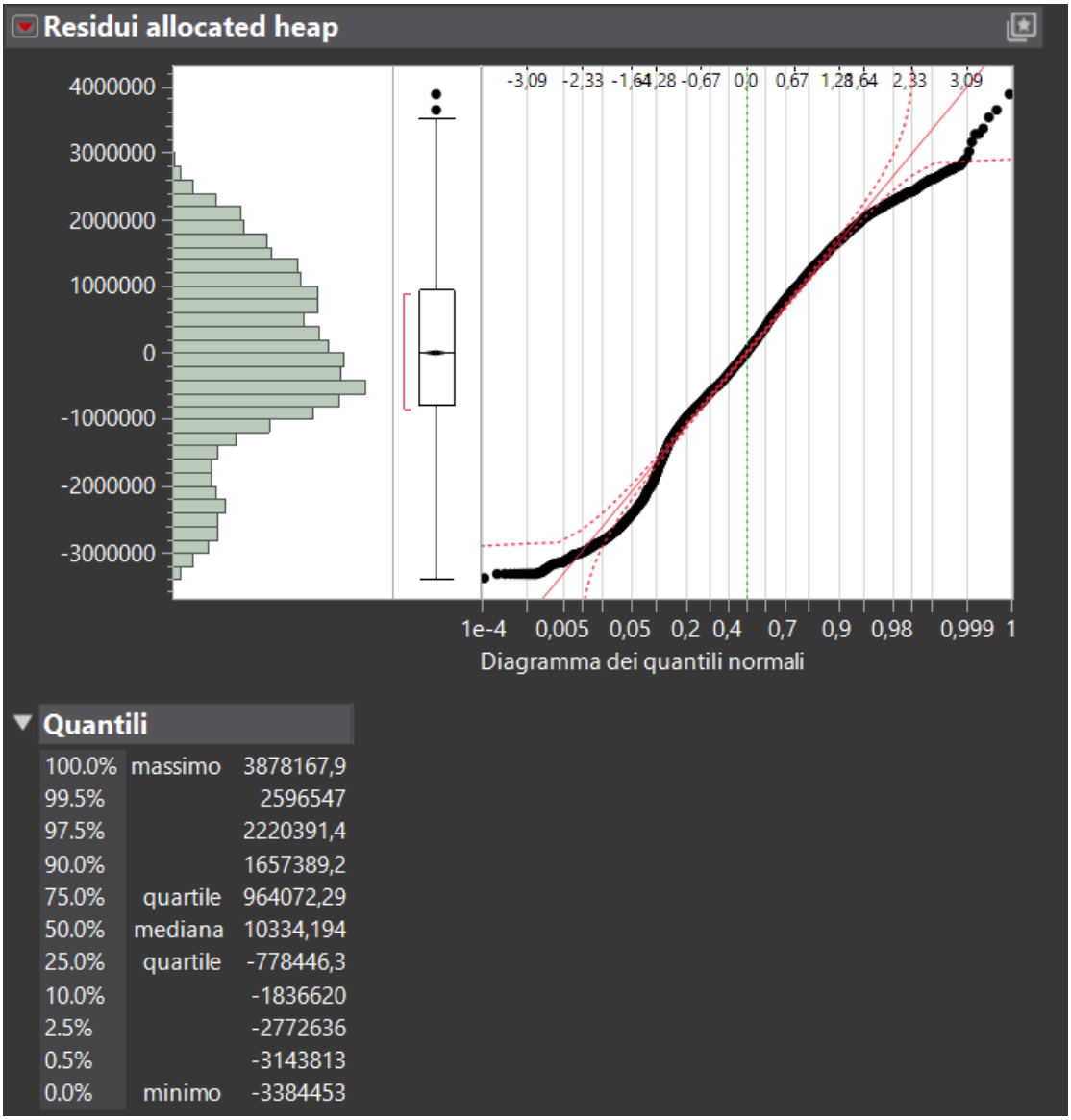


Figura 55: Test visivo Normalità

Come si evince dal test visivo la normalità non è riscontrata.

4.5.3 Test di Mann-Kendall

Data la non normaltà effettuiamo il test di Mann-Kendall per assicurarci che sia presente effettivamente una retta monotona crescente con pendenza significativa

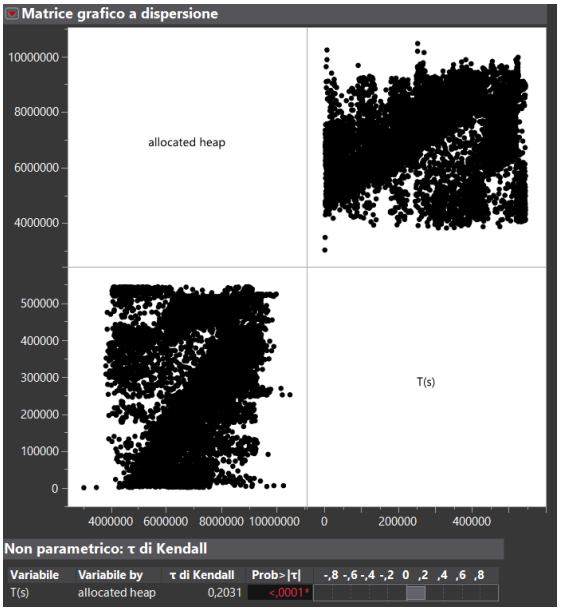


Figura 56: Analisi Multivariata e Mann-Kendall

Il p-value del test è minore di 0.05 quindi il test ha rigettato l'ipotesi nulla (Retta con pendenza costante)

4.5.4 Stima di Theil-Sen e tempo di saturazione della memoria heap grande 1G

Una volta che abbiamo confermato la presenza di un trend possiamo calcolarci la pendenza della retta in modo più preciso usando la Stima di Theuk-Sen tramite uno script python.

```
Theil-Sen results:
  slope = 2.903093
  intercept = 6041585.556431
  CI(0.95) slope = [2.714760, 3.092353]
  OLS slope = 1.913329
  Pearson r = 0.2282
  n = 9090
```

Figura 57: Valori retta di regressione

La retta di regressione ha quindi come equazione:
$HeapAllocata = 6041585.556 + (2.9 \pm 0.19) * T(s)$
La formula per calcolare il tempo di saturazione della heap segue facilmente:

$$T(s) = \frac{10^9 - 6041585.556}{2.9 \pm 0.19} = (10.9 \pm 0.7) anni$$

4.6 OS

In questa sezione analizzeremo tre dataset, chiamati *os1*, *os2* e *os3*. In questi dataset andremo a rilevare e stimare eventuali trend su 5 variabili:

- VMSize
- VMData
- RSS
- Byte letti
- Byte scritti

4.6.1 os1

Realizziamo il modello regressivo e calcoliamo la distribuzione dei residui:

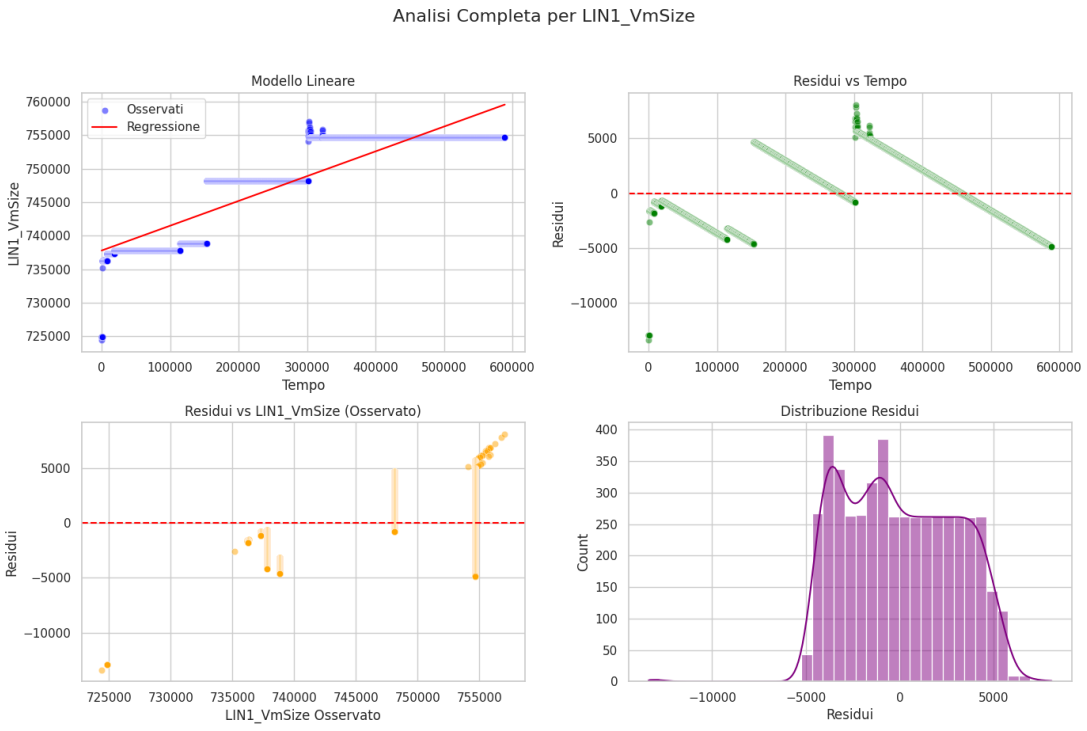


Figura 58: Analisi LIN1_VmSize

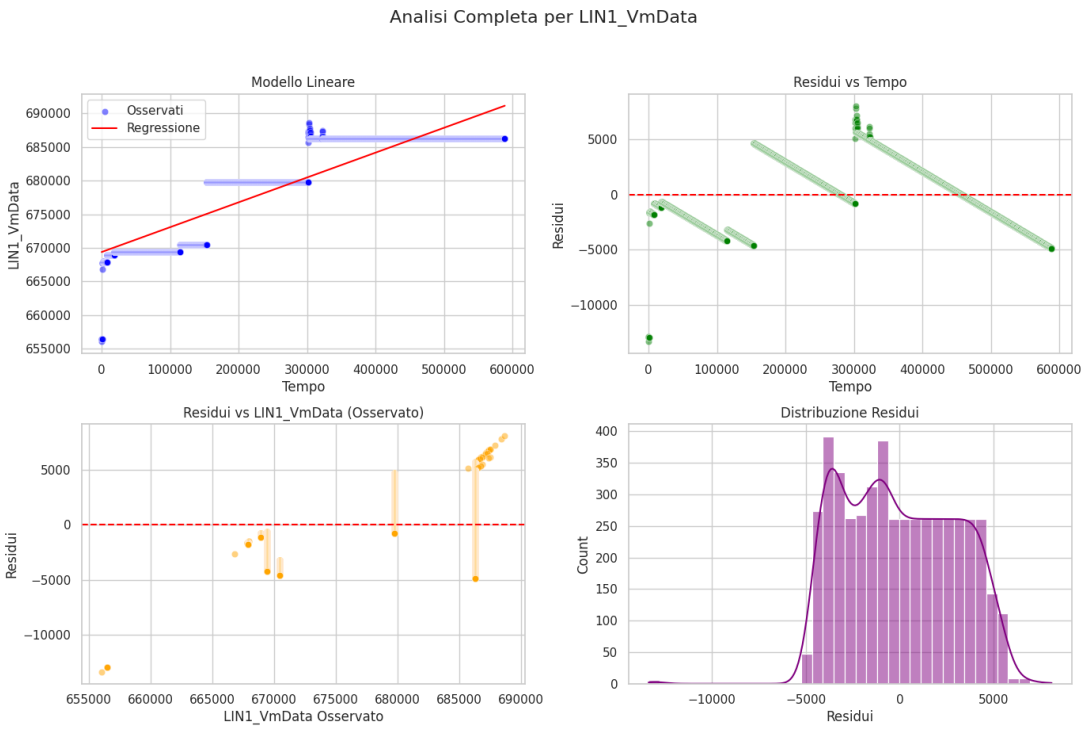


Figura 59: Analisi LIN1_VmData

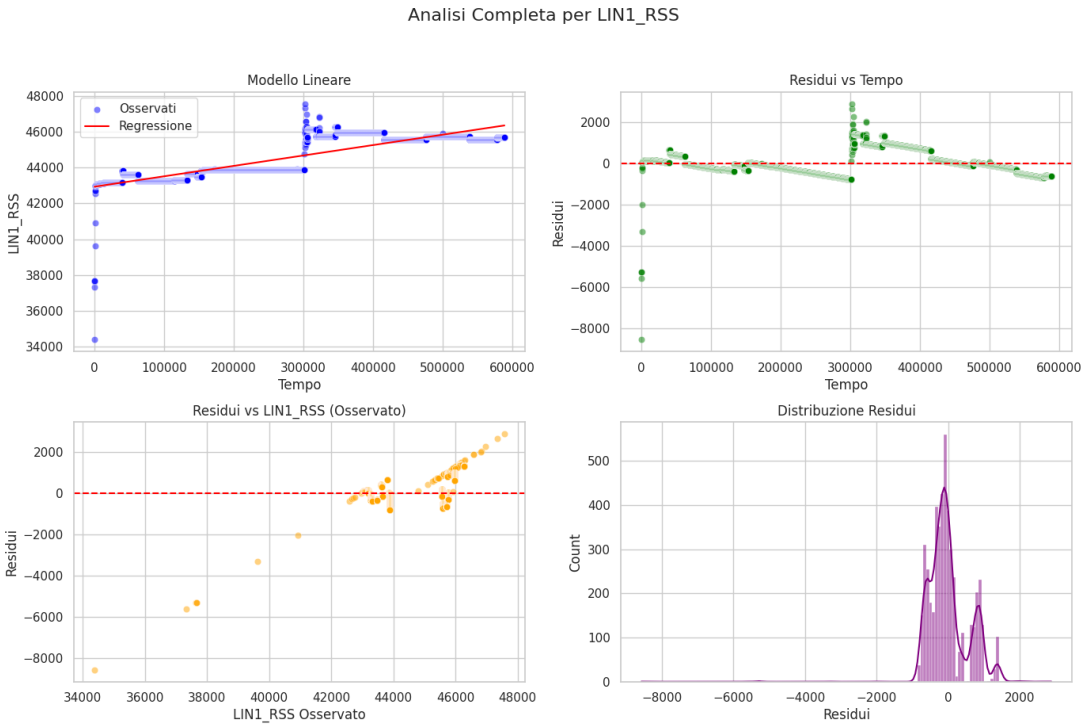


Figura 60: Analisi LIN1_RSS

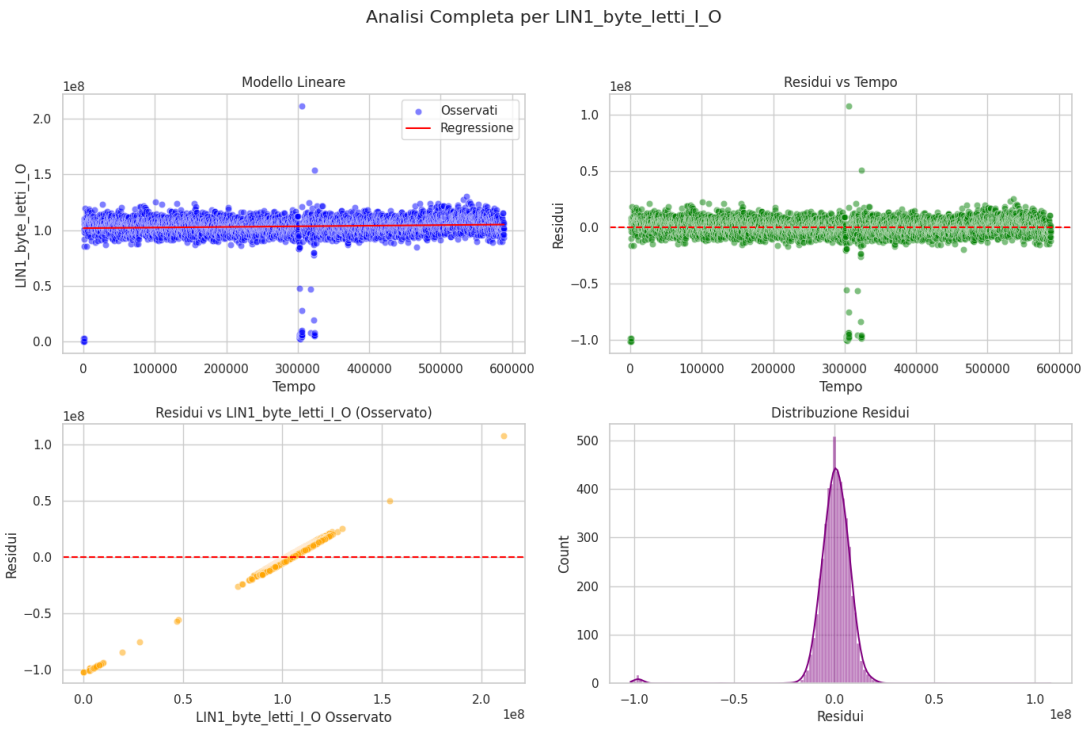


Figura 61: Analisi LIN1_byte_letti_I_O

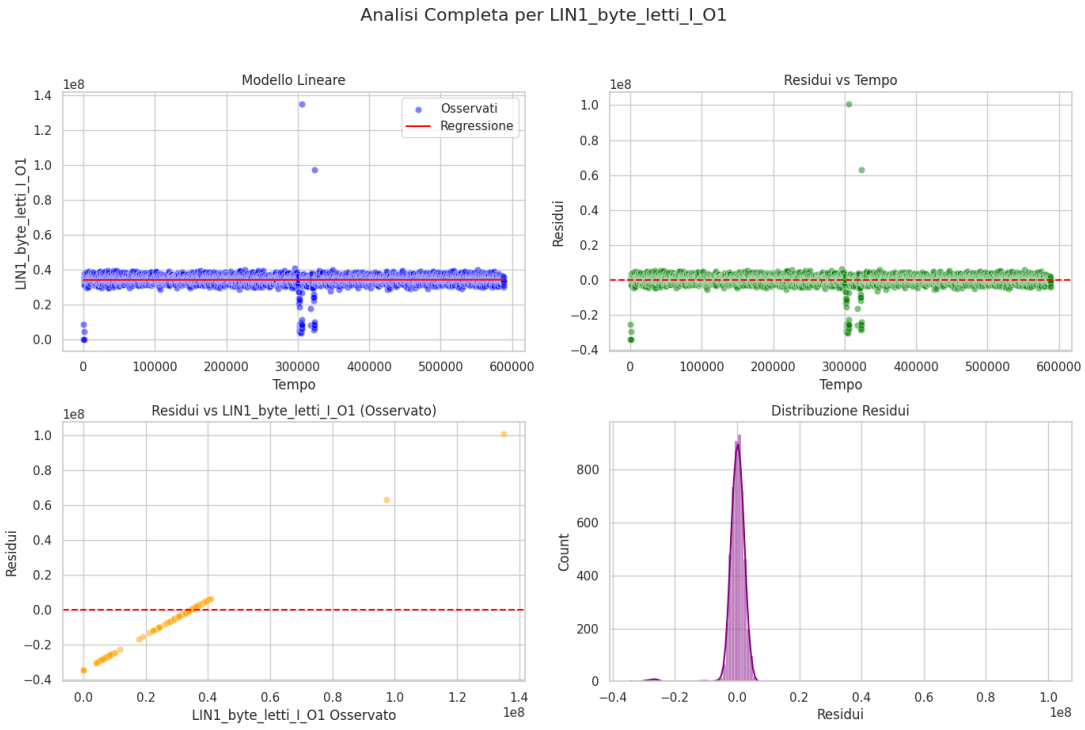


Figura 62: Analisi LIN1_byte_letti_I_O1

```
Risultati Test di Shapiro-Wilk:

Variabile: LIN1_VmSize
  Statistica W: 0.9610
  P-value: 2.0407e-34
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN1_VmData
  Statistica W: 0.9610
  P-value: 2.0439e-34
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN1_RSS
  Statistica W: 0.8687
  P-value: 1.5236e-53
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN1_byte_letti_I_O
  Statistica W: 0.5658
  P-value: 1.1006e-76
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN1_byte_letti_I_O1
  Statistica W: 0.5225
  P-value: 1.0448e-78
  Esito: Distribuzione NON normale (p < 0.05)
```

Figura 64: Test di Shapiro-Wilk

Effettuiamo il test di Shapiro-Wilk e andiamo a visualizzare il Q-Q plot:

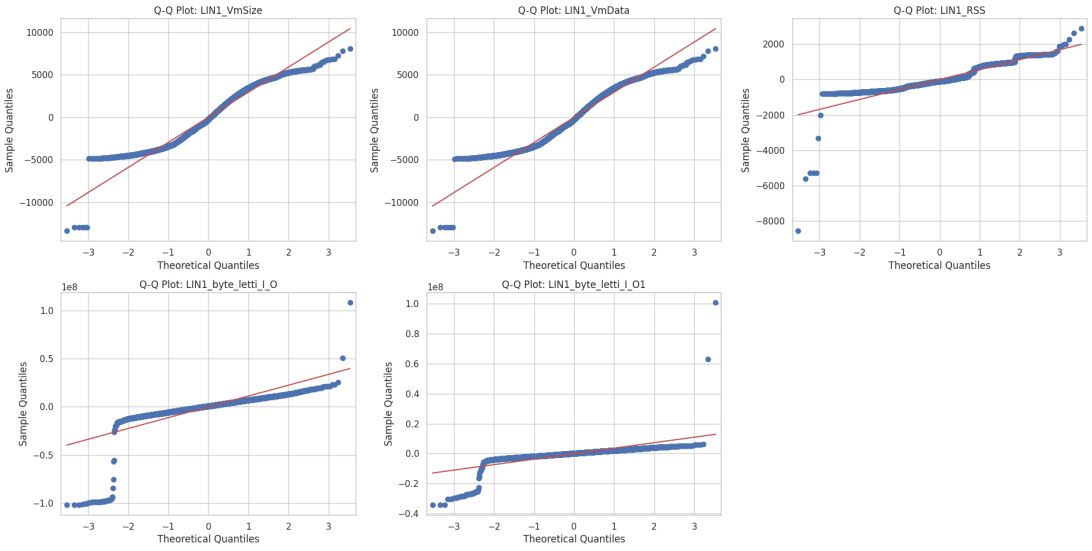


Figura 63: Q-Q plot

Si evince quindi che la distribuzione dei dati è non normale, per cui non ne testeremo l'omoschedasticità.

Effettuiamo il test di Mann-Kendall:

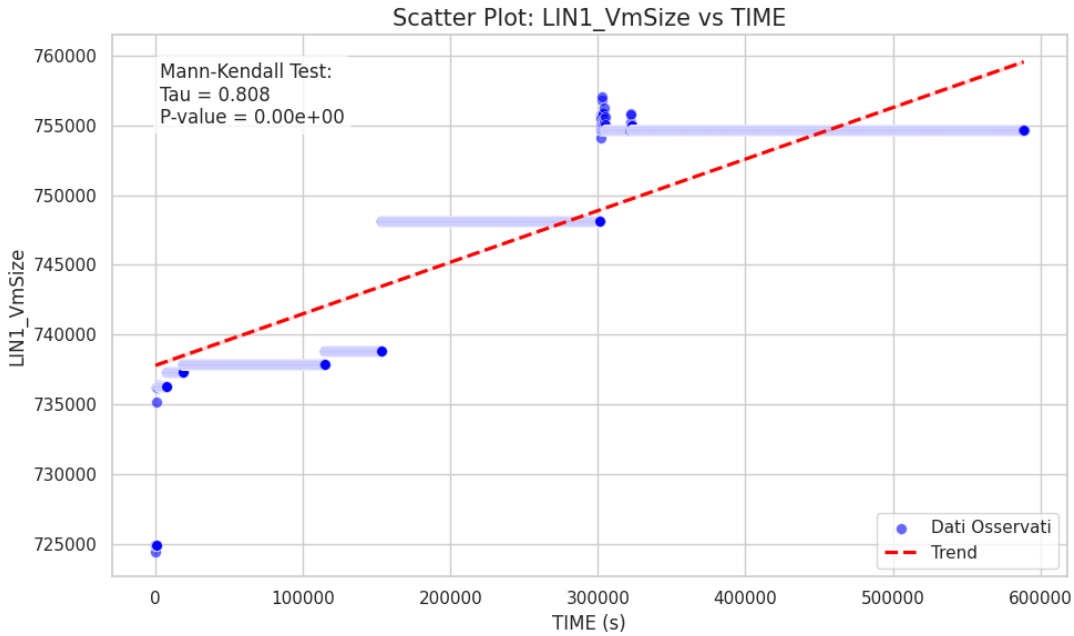


Figura 65: Test di Mann-Kendall VmSize

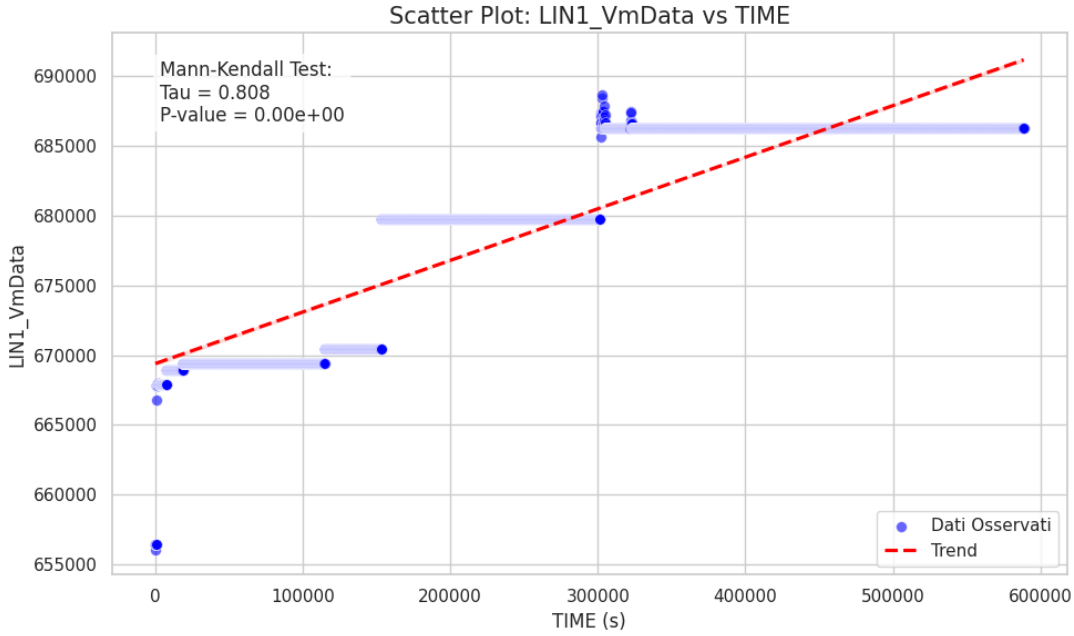


Figura 66: Test di Mann-Kendall VmData

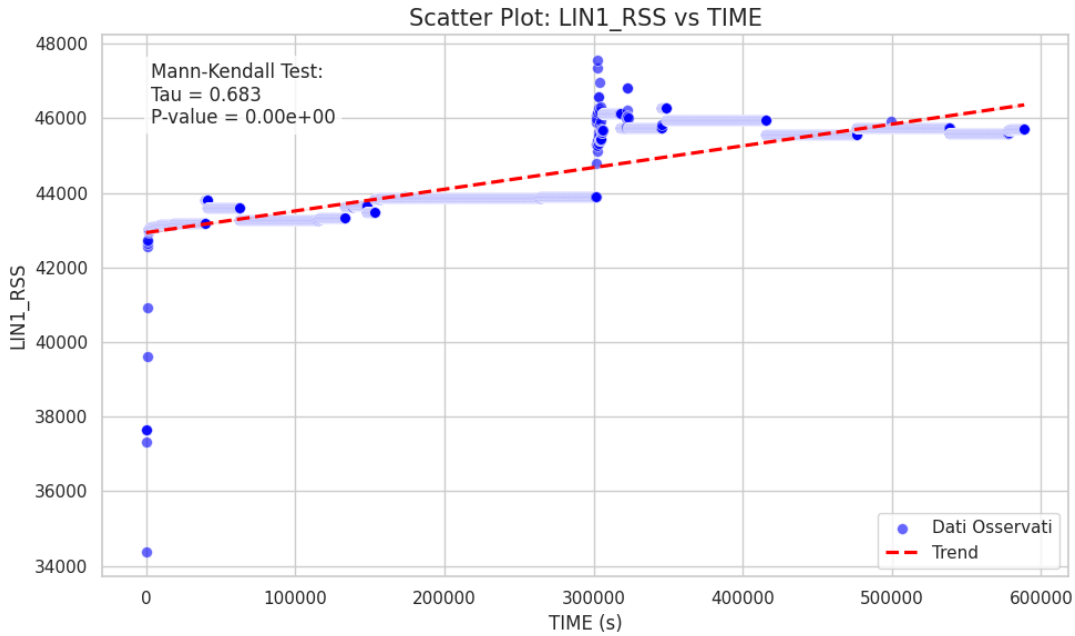


Figura 67: Test di Mann-Kendall RSS

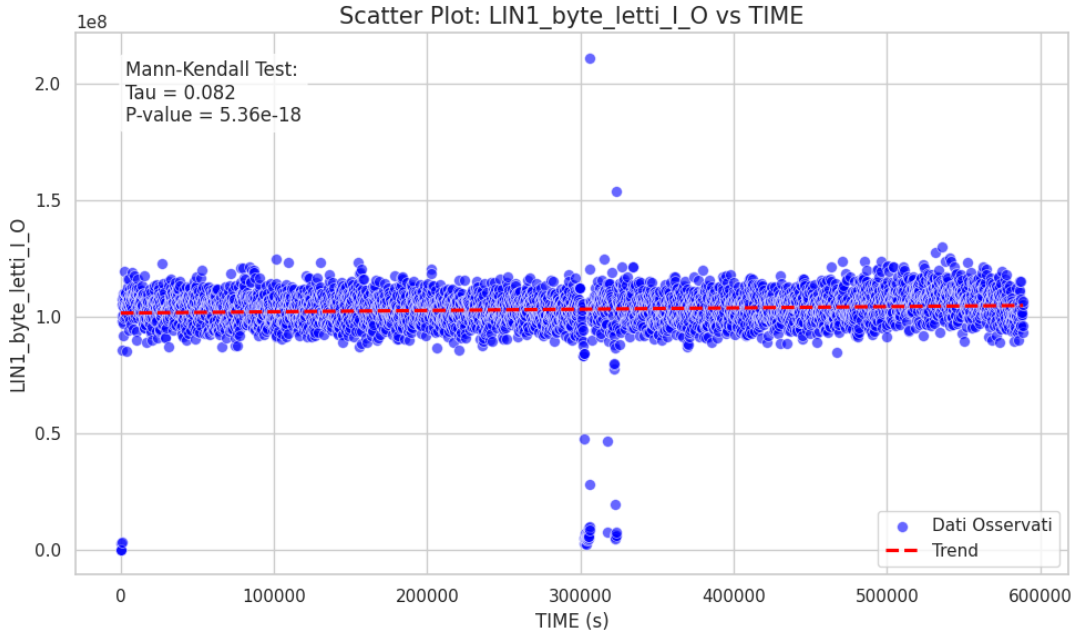


Figura 68: Test di Mann-Kendall byte_letti_I_O

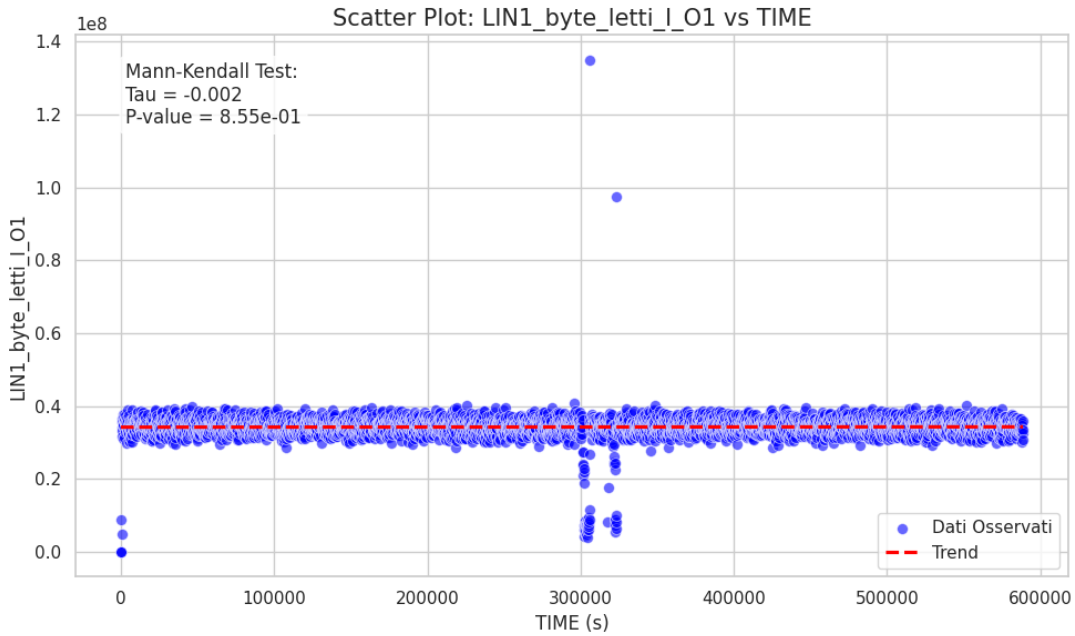


Figura 69: Test di Mann-Kendall byte_letti_I_O1

Dal τ di Kendall ci fornisce informazioni sulla correlazione tra le variabili:

- VmSize: correlazione positiva molto forte
- VmData: correlazione positiva molto forte
- RSS: correlazione positiva forte
- byte_letti_I_O: correlazione molto debole
- byte_letti_I_O1: correlazione molto debole

Dal p-value si può vedere che solamente per LIN1_byte_letti_i_O1 il trend non è statisticamente significativo, mentre per tutte le altre variabili lo è.

Per stimare la pendenza della retta di regressione utilizziamo uno stimatore di Theil-Sen:

Metric	Slope	Lower	Upper	Intercept
VmSize	0.03157	0.03071	0.03232	738854.3947841726
VmData	0.03157	0.03071	0.03232	670450.3947841726
RSS	0.00497	0.00487	0.00505	42428.80701754386
byte_letti_I_O	4.73757	3.67391	5.80459	102763229.503125
byte_letti_I_O1	-0.03081	-0.36088	0.29842	/

Tabella 3: Valori retta di regressione

4.6.2 os2

Realizziamo il modello regressivo e calcoliamo la distribuzione dei residui:

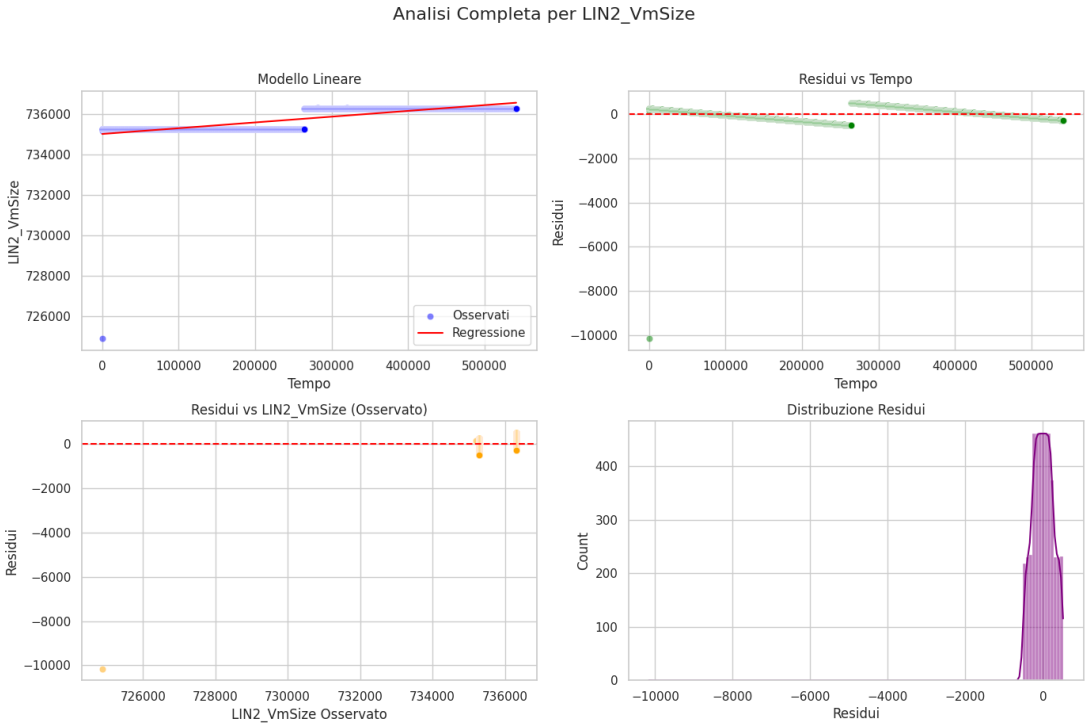


Figura 70: Analisi LIN2_VmSize

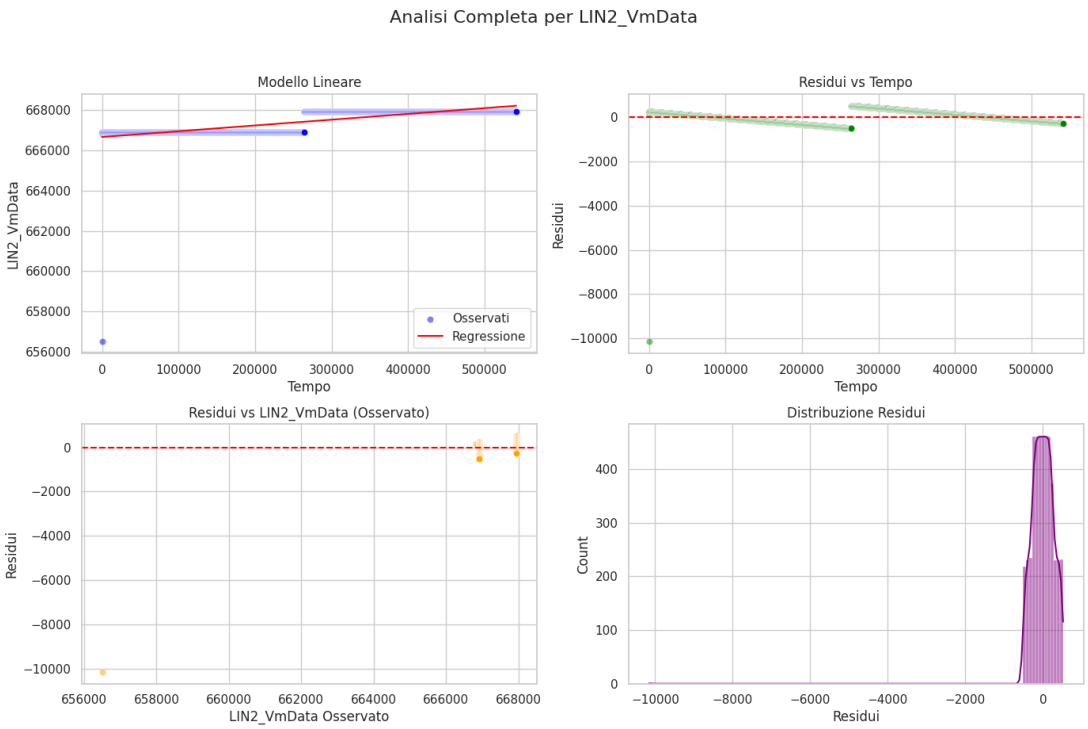


Figura 71: Analisi LIN2_VmData

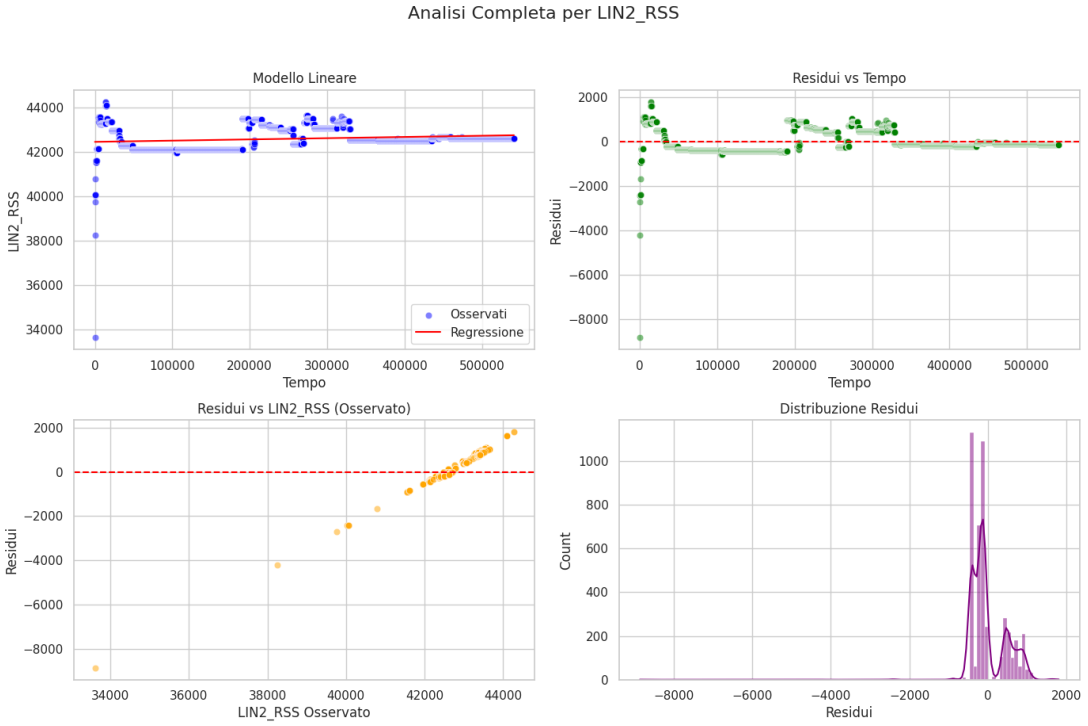


Figura 72: Analisi LIN2_RSS

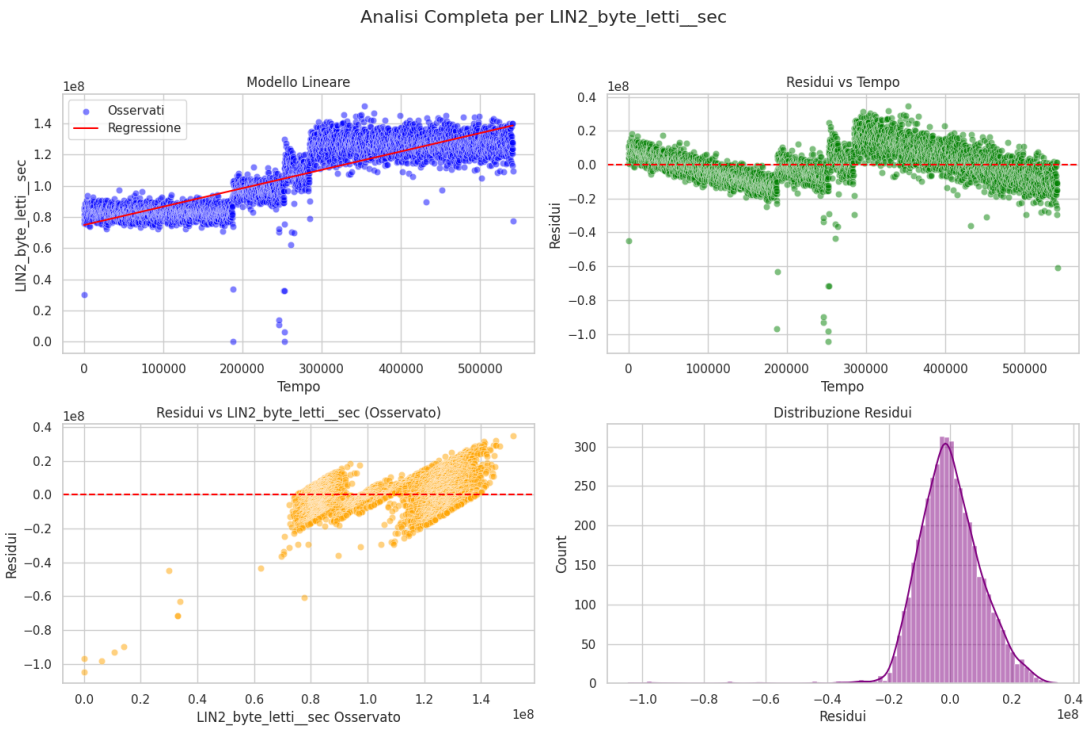


Figura 73: Analisi LIN2_byte_letti__sec

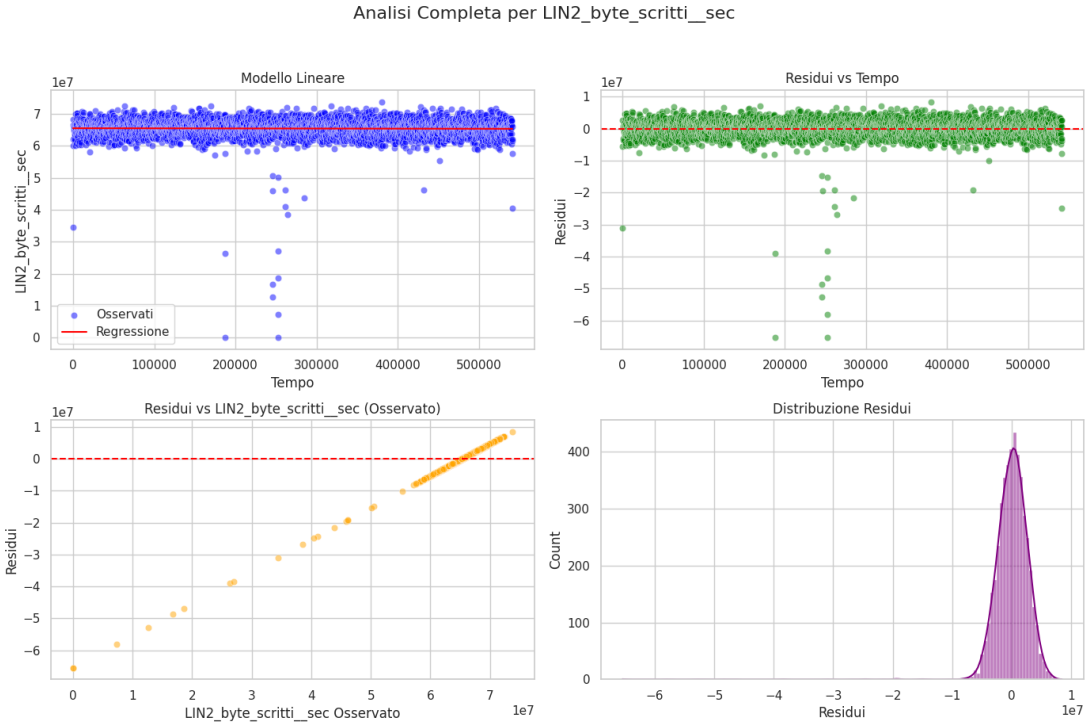


Figura 74: Analisi LIN2_byte_scriti__sec

```
Risultati Test di Shapiro-Wilk:

Variabile: LIN2_VmSize
  Statistica W: 0.7770
  P-value: 1.5824e-61
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN2_VmData
  Statistica W: 0.7770
  P-value: 1.5824e-61
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN2_RSS
  Statistica W: 0.8035
  P-value: 3.6333e-59
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN2_byte_letti__sec
  Statistica W: 0.9380
  P-value: 7.4790e-40
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN2_byte_scritti__sec
  Statistica W: 0.6417
  P-value: 8.7844e-71
  Esito: Distribuzione NON normale (p < 0.05)
```

Figura 76: Test di Shapiro-Wilk

Effettuiamo il test di Shapiro-Wilk e andiamo a visualizzare il Q-Q plot:

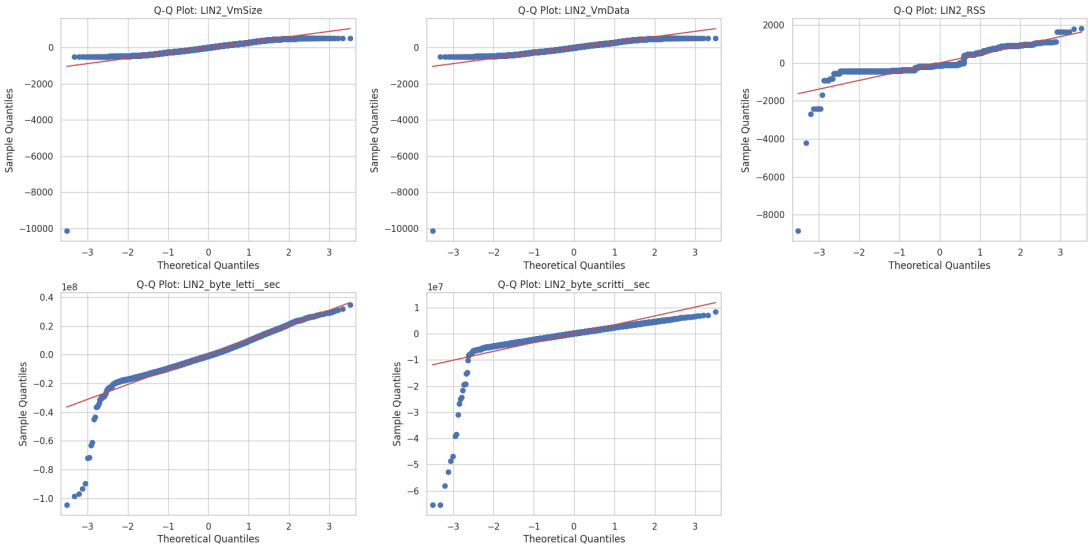


Figura 75: Q-Q plot

Si evince quindi che la distribuzione dei dati è non normale, per cui non ne testeremo l'omoschedasticità.

Effettuiamo il test di Mann-Kendall:

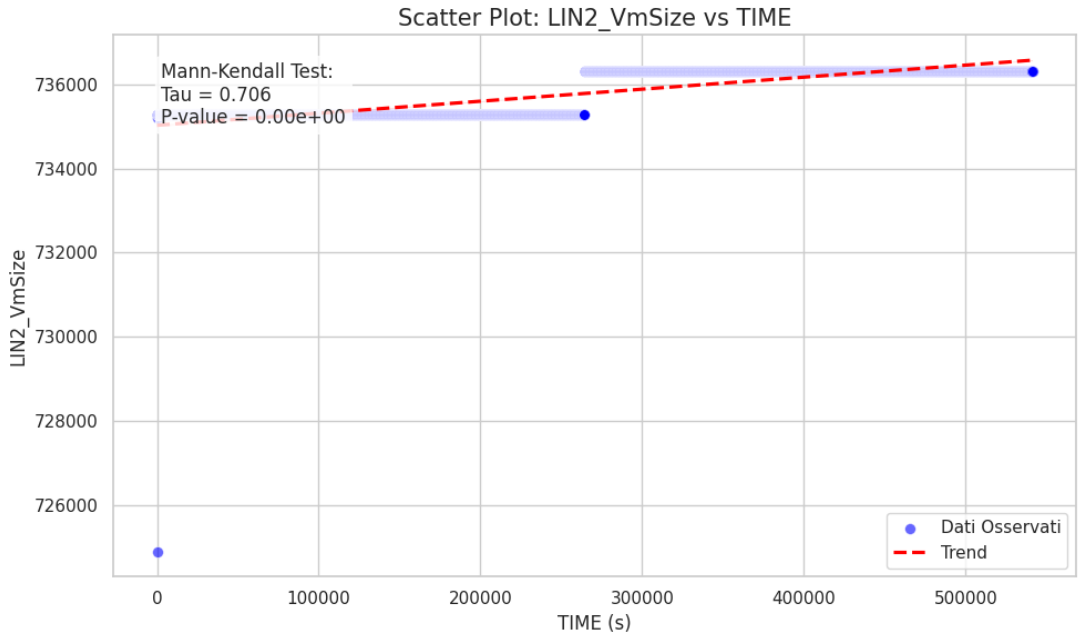


Figura 77: Test di Mann-Kendall VmSize

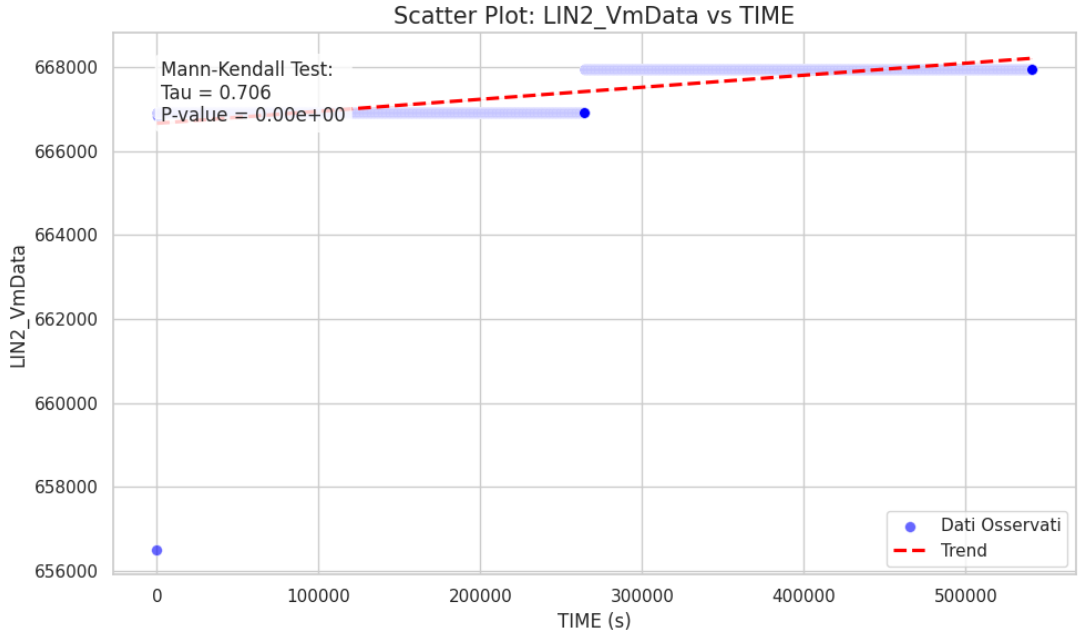


Figura 78: Test di Mann-Kendall VmData

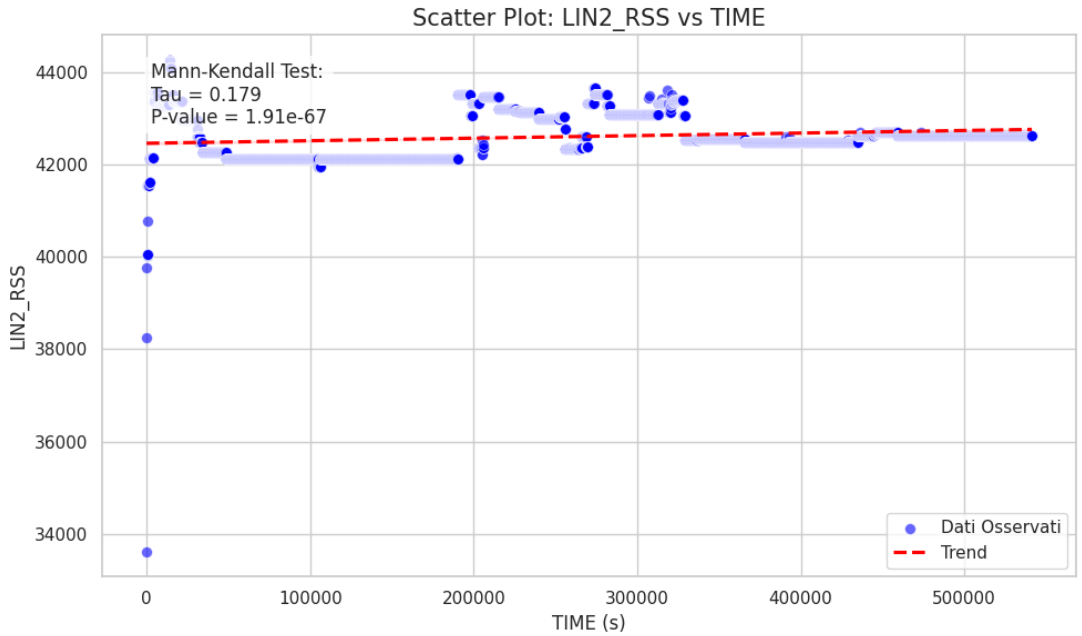


Figura 79: Test di Mann-Kendall RSS

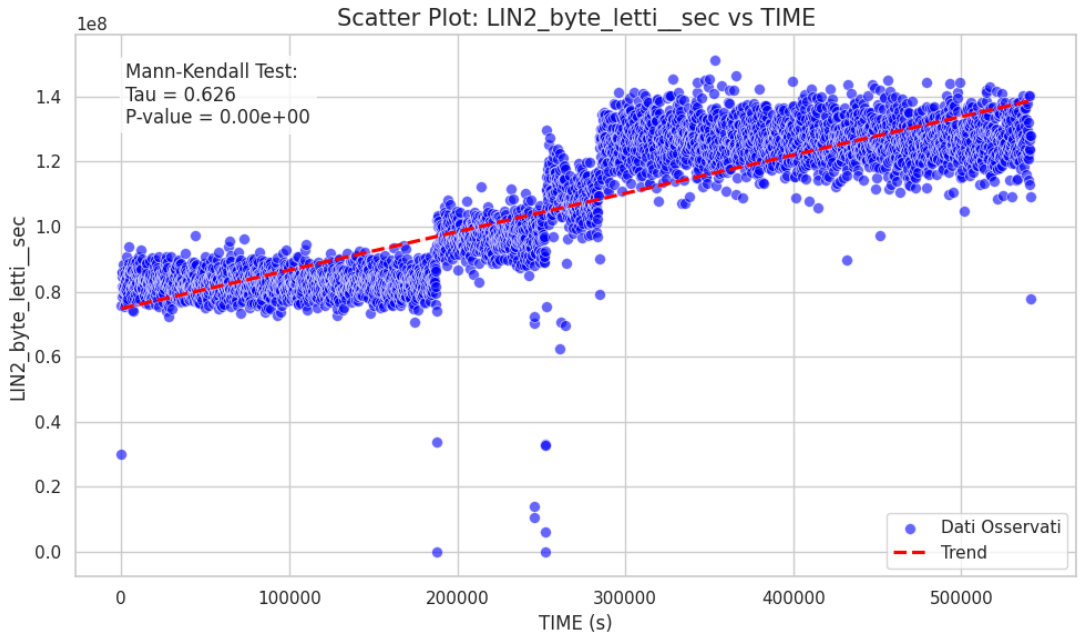


Figura 80: Test di Mann-Kendall byte_letti__sec

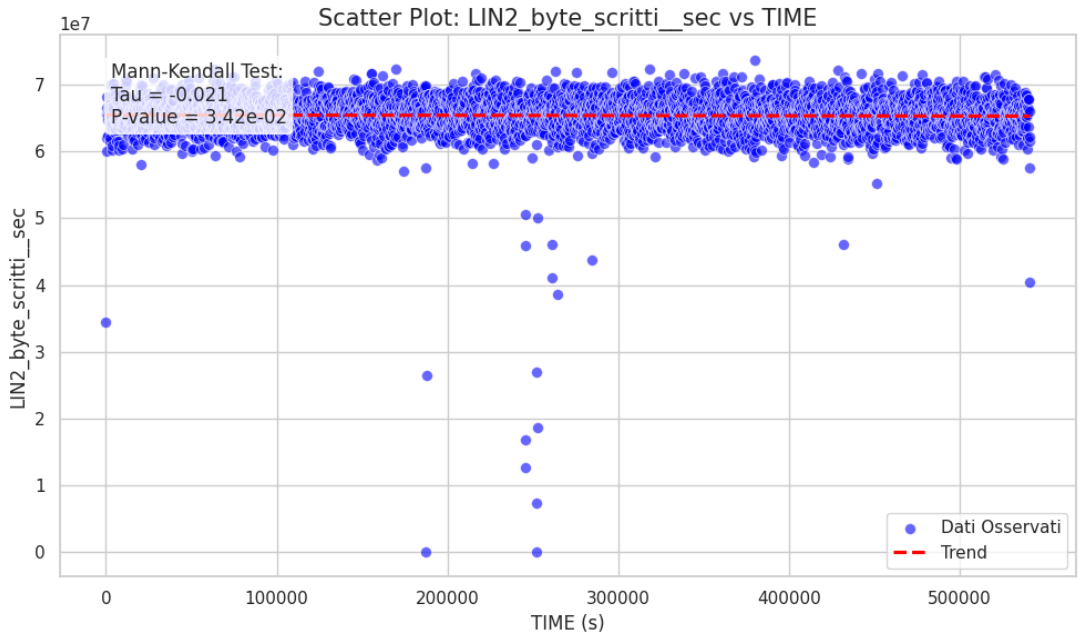


Figura 81: Test di Mann-Kendall byte_scritti__sec

Dal τ di Kendall ci fornisce informazioni sulla correlazione tra le variabili:

- VmSize: correlazione positiva forte
- VmData: correlazione positiva forte
- RSS: correlazione positiva debole
- byte_letti__sec: correlazione positiva forte
- byte_scritti__sec: correlazione negativa molto debole

In questo caso, siccome tutti i p-value sono minori di 0.05, i trend sono tutti significativamente statistici.

Per stimare la pendenza della retta di regressione utilizziamo uno stimatore di Theil-Sen:

Metric	Slope	Lower	Upper	Intercept
VmSize	0.00191	0.0	0.00208	735790.3761488455
VmData	0.00191	0.0	0.00208	667414.3761488455
RSS	0.00063	0.00055	0.00074	42384.85453267744
byte_letti__sec	114.55350	112.73359	116.37659	77733384.23784722
byte_scritti__sec	-0.48861	-0.93900	-0.03631	65693615.57629428

Tabella 4: Valori retta di regressione

4.6.3 os3

Realizziamo il modello regressivo e calcoliamo la distribuzione dei residui:

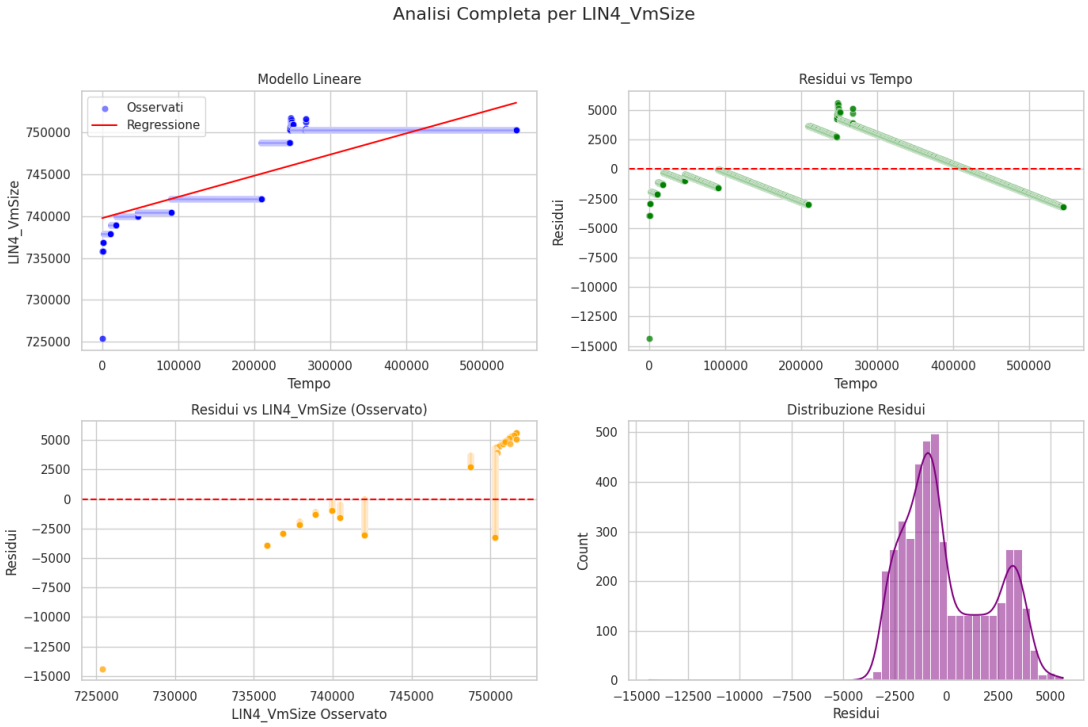


Figura 82: Analisi LIN4_VmSize

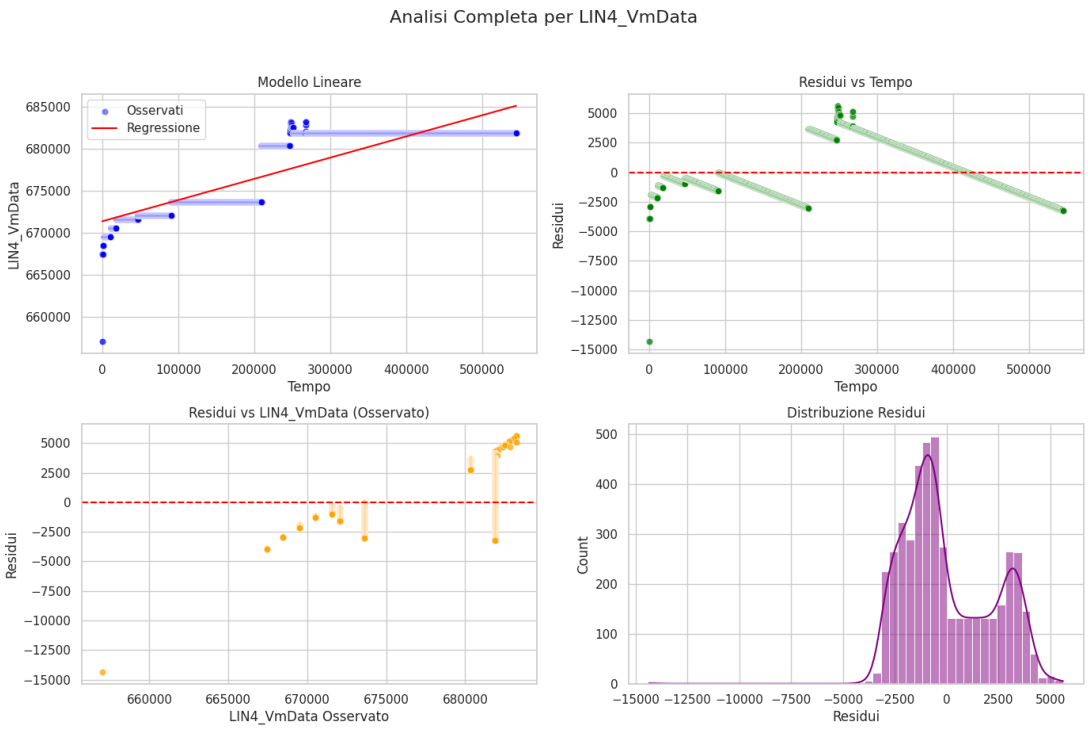


Figura 83: Analisi LIN4_VmData

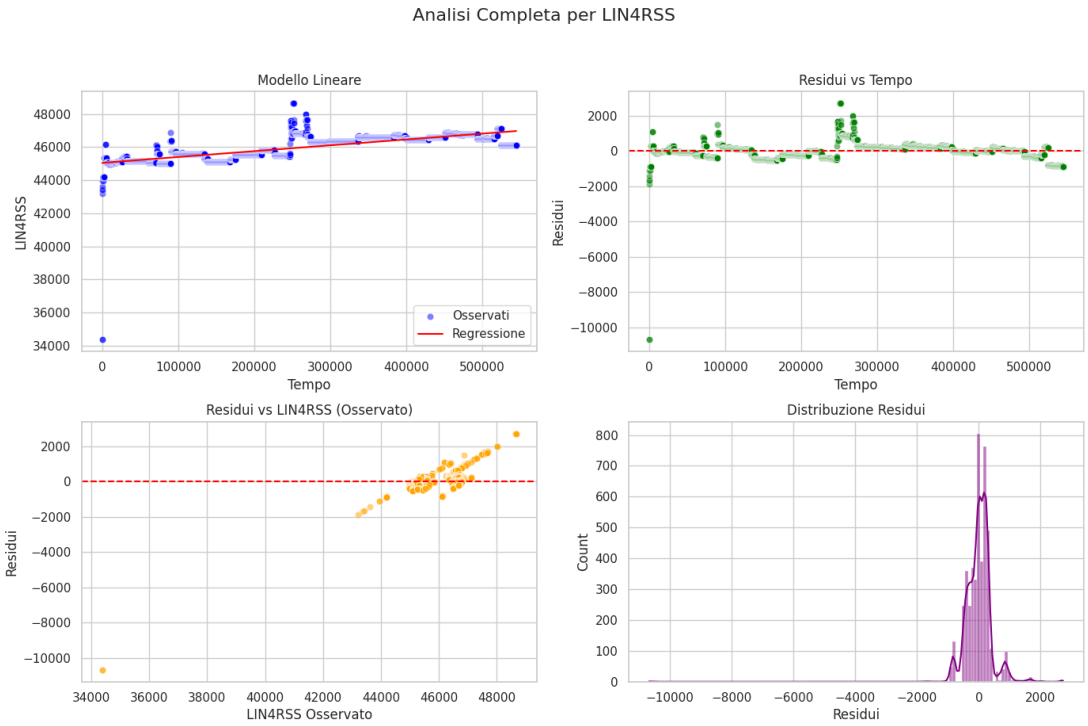


Figura 84: Analisi LIN4_RSS

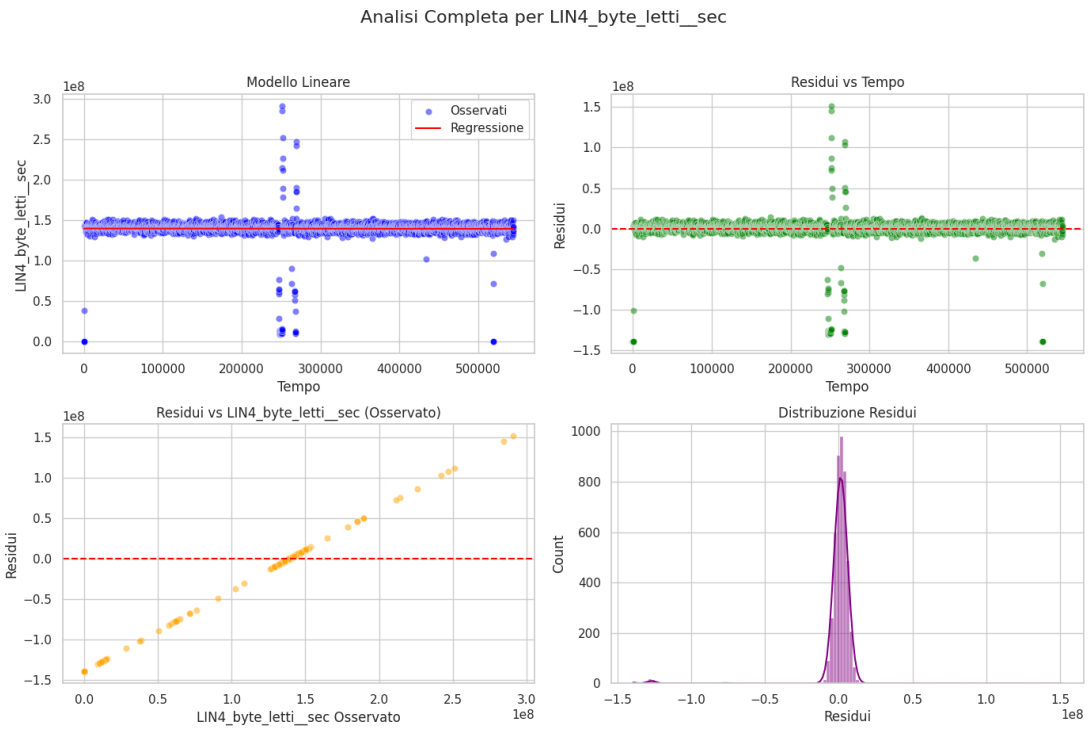


Figura 85: Analisi LIN4_byte_letti__sec

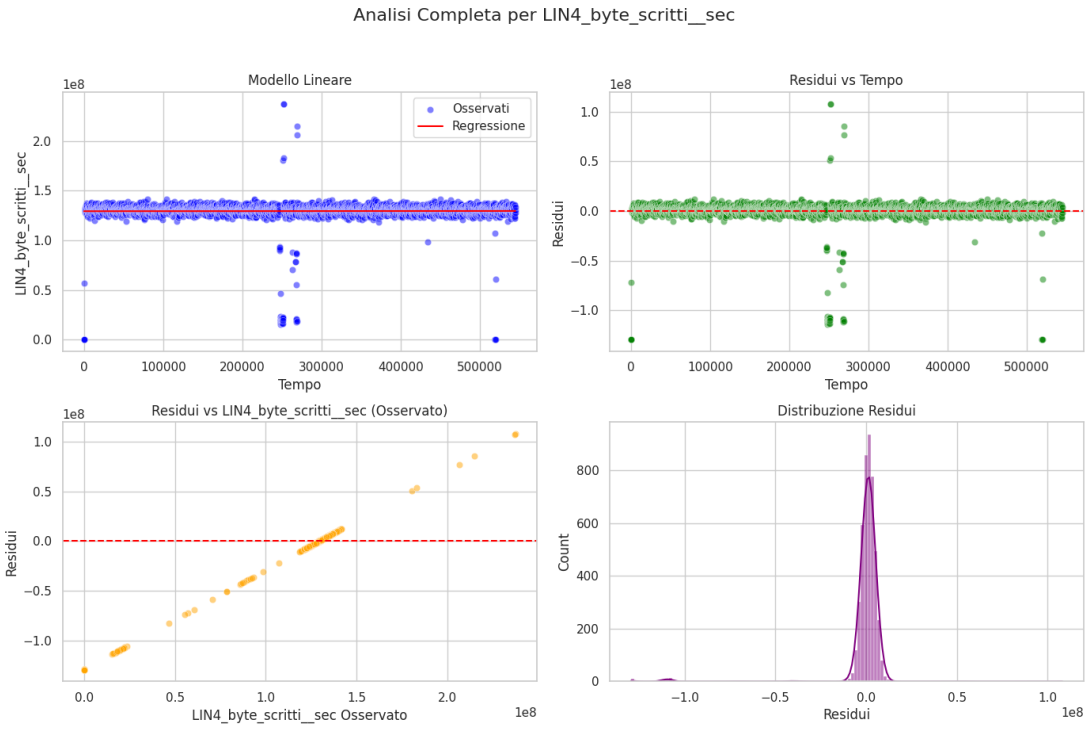


Figura 86: Analisi LIN4_byte_scriti__sec

```
Risultati Test di Shapiro-Wilk:

Variabile: LIN4_VmSize
  Statistica W: 0.9279
  P-value: 2.9469e-42
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN4_VmData
  Statistica W: 0.9280
  P-value: 2.9920e-42
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN4RSS
  Statistica W: 0.7418
  P-value: 1.9165e-64
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN4_byte_letti__sec
  Statistica W: 0.3043
  P-value: 5.2016e-85
  Esito: Distribuzione NON normale (p < 0.05)

Variabile: LIN4_byte_scritti__sec
  Statistica W: 0.2988
  P-value: 3.4679e-85
  Esito: Distribuzione NON normale (p < 0.05)
```

Figura 88: Test di Shapiro-Wilk

Effettuiamo il test di Shapiro-Wilk e andiamo a visualizzare il Q-Q plot:

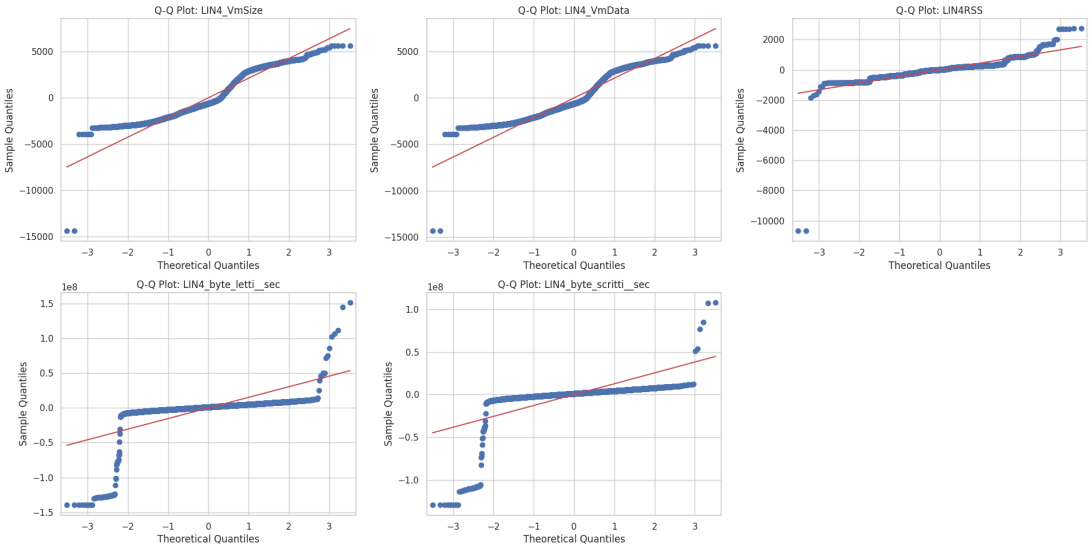


Figura 87: Q-Q plot

Si evince quindi che la distribuzione dei dati è non normale, per cui non ne testeremo l'omoschedasticità.

Effettuiamo il test di Mann-Kendall:

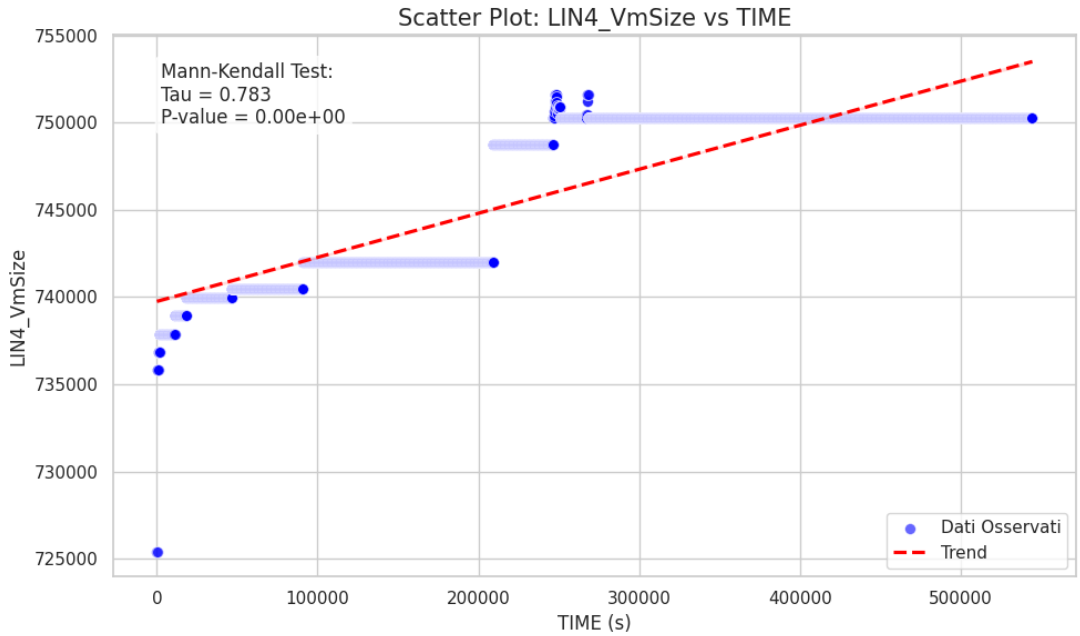


Figura 89: Test di Mann-Kendall VmSize

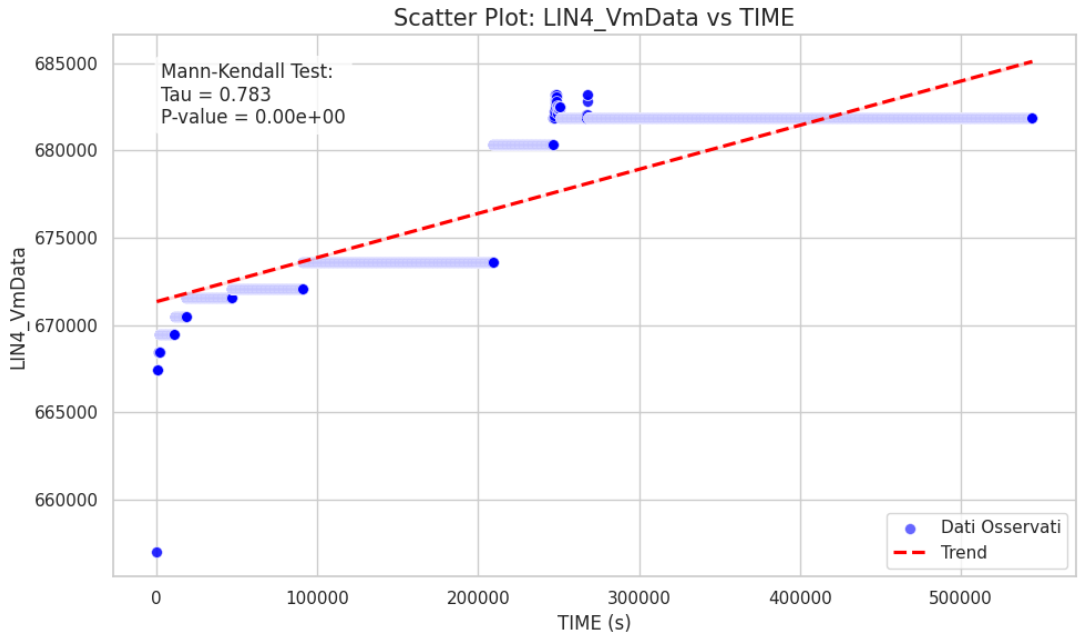


Figura 90: Test di Mann-Kendall VmData

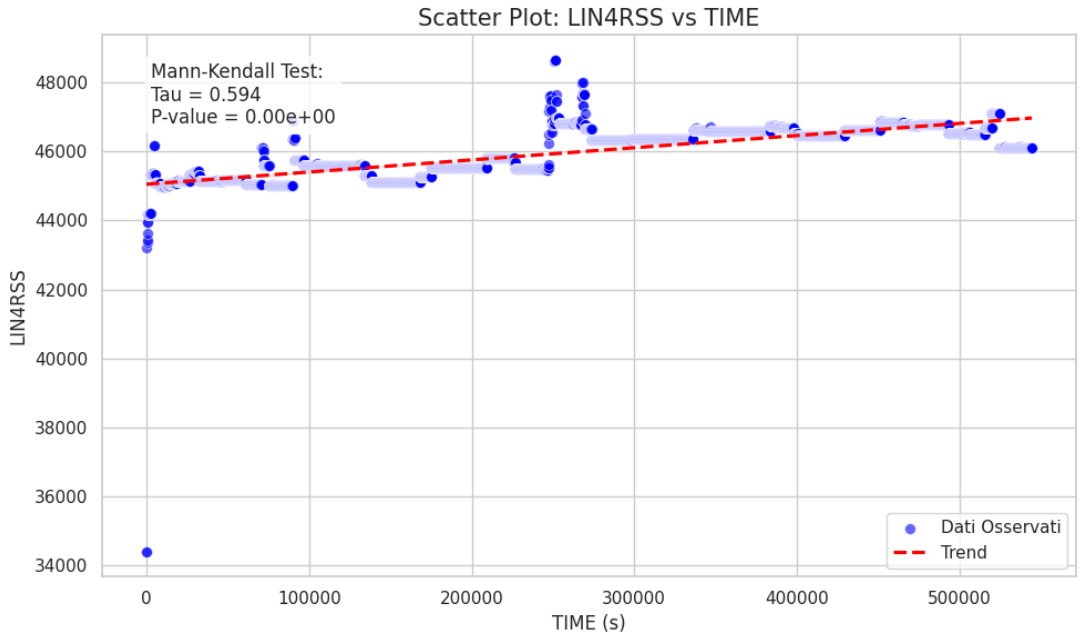


Figura 91: Test di Mann-Kendall RSS

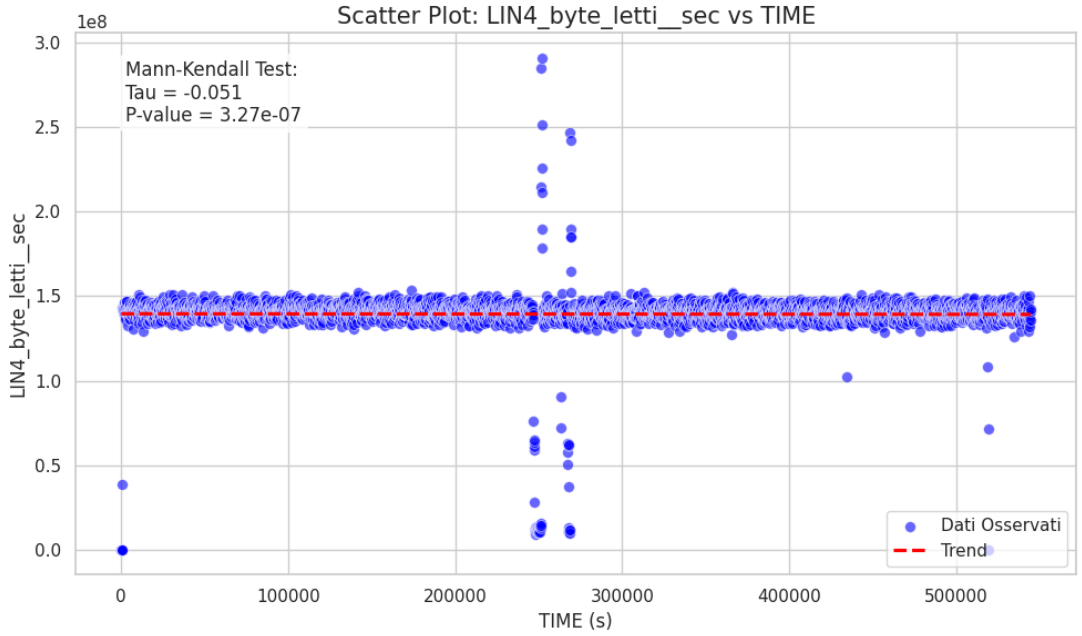


Figura 92: Test di Mann-Kendall byte_letti__sec

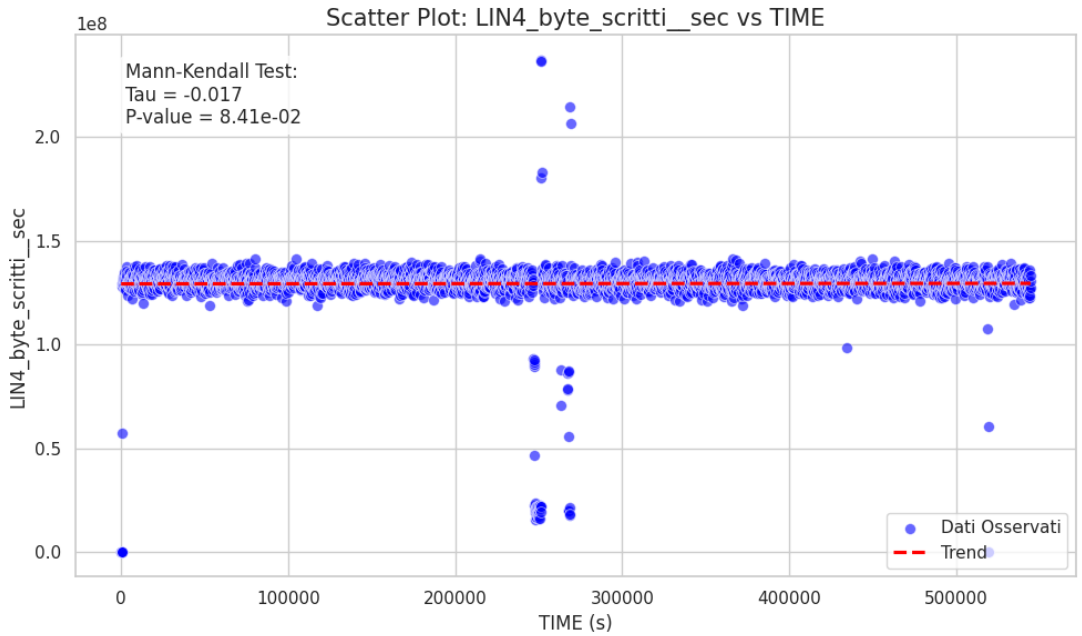


Figura 93: Test di Mann-Kendall byte_scritti__sec

Dal τ di Kendall ci fornisce informazioni sulla correlazione tra le variabili:

- VmSize: correlazione positiva forte
- VmData: correlazione positiva forte
- RSS: correlazione positiva forte
- byte_letti__sec: correlazione negativa molto debole
- byte_scritti__sec: correlazione negativa molto debole

Dal p-value si può vedere che solamente per LIN1_byte_scritti__sec il trend non è statisticamente significativo, mentre per tutte le altre variabili lo è.

Per stimare la pendenza della retta di regressione utilizziamo uno stimatore di Theil-Sen:

Metric	Slope	Lower	Upper	Intercept
VmSize	0.02165	0.02127	0.02202	744374.599
VmData	0.02165	0.02127	0.02202	675970.599
RSS	0.00345	0.00338	0.00352	45386.471
byte_letti__sec	-1.89104	-2.61830	-1.16700	141291080.771
byte_scritti__sec	-0.56246	-1.19808	0.07666	/

Tabella 5: Valori retta di regressione

5 Design Of Experiments

In questo capitolo verrà presentato il design dell'esperimento fatto con gli strumenti utilizzati durante il capacity test. Le valutazioni sono state prese cercando dei fattori che intuitivamente spiegassero meglio le variabili di risposta, ovvero il Throughput, il Response Time e la Latenza. Abbiamo deciso di lavorare con 2 fattori a 3 livelli ed effettuando 4 catture (ripetizioni) da circa 2 minuti ciascuno per un totale quindi di 36 osservazioni

Livello	**CTT (req/min)**	**Dimensione File**
Livello 1	25% (2500)	500 KB
Livello 2	50% (5000)	2 MB
Livello 3	75% (7500)	8 MB

Tabella 6: Definizione dei livelli per i fattori CTT e Dimensione File.

I CTT sono stati presi a partire dal valore di CTT in cui il server raggiungeva la sua usable capacity(10000) e assegnando ad ogni livello una percentuale di quest'ultimo. Una volta scelti i fattori con i loro livelli è stato fatto un analisi dei fattori per le tre response variable scelte.

5.1 Response Time

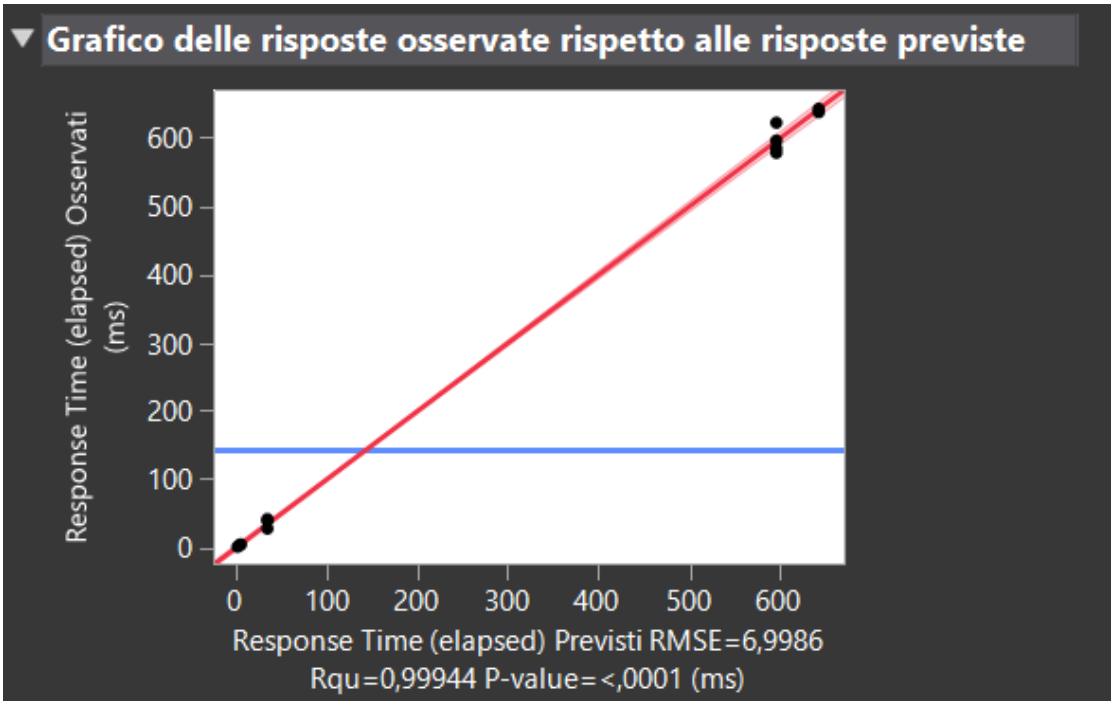
Per il response time i risultati ottenuti sono i seguenti.

DimFile	**CTT 25% (2500)**	**CTT 50% (5000)**	**CTT 75% (7500)**
500 KB	1.43122, 1.45836, 1.32436, 1.33443	1.48589, 1.10983, 1.04436, 1.07850	1.09846, 1.03019, 1.01087, 1.00959
2 MB	3.65022, 3.78398, 3.64381, 3.75228	3.42860, 3.52354, 3.42731, 3.50701	4.93845, 4.96505, 5.05369, 4.62665
8 MB	39.2229, 28.0734, 41.5805, 27.6377	585.8695, 579.1072, 623.3217, 597.0841	645.1628, 643.7646, 638.7637, 644.9041

Tabella 7: Osservazioni raccolte per il Response Time (ms) in base alla combinazione dei fattori.

5.1.1 Stima del modello e analisi della varianza

Come prima cosa è stata effettuata un analisi dell'importanza dei fattori andando a fare una stima del modello con lo strumento offerto da JMP.



Riepilogo effetti

Analisi della varianza

Origine	DF	Somma dei quadrati	Media quadratica	Rapporto F
Modello	8	2343149,9	292894	5979,842
Errore	27	1322,5	49	Prob > F
C. totale	35	2344472,4		<,0001*

Test degli effetti

Origine	Nparm	DF	Somma dei quadrati	Rapporto F	Prob > F
CTT	2	2	306590,9	3129,744	<,0001*
Dimensione file	2	2	1424232,9	14538,87	<,0001*
CTT*Dimensione file	4	4	612326,2	3125,377	<,0001*

Figura 94: Tabella Response Time

Dalla figura possiamo individuare i valori fondamentali che ci servono per calcolare l'importanza di tutti i nostri fattori

$$SSA = 306590,9 \qquad SSB = 1424232,9$$

$$SSAB = 612326,2 \qquad SST = 2344472,4$$

$$\text{Varianza non spiegata} = \frac{SSE}{SST} = \frac{1322,5}{2344473,4} = 5,64 * 10^{-4} = \mathbf{0{,}0005\%}$$

$$\text{Importanza A} = \frac{SSA}{SST} = \frac{306590,9}{2344473,4} = 0,13 = \mathbf{13\%}$$

$$\text{Importanza B} = \frac{SSB}{SST} = \frac{1424232,9}{2344473,4} = 0,60 = \mathbf{60\%}$$

$$\text{Importanza AB} = \frac{SSAB}{SST} = \frac{612326,2}{2344473,4} = 0,27 = \mathbf{27\%}$$

Dai calcoli è possibile notare che l'imprtanza del fattore B (Dimensione File) è molto alta, poichè spiega una buona parte della varianza totale, inoltre si nota come sia più importante l'interazione tra i fattori A e B che il fattore A preso singolarmente.

5.1.2 Normalità e omoschedasticità dei residui

Dopo aver stimato il modello andiamo ad esaminare la normalità e la omoschedasticità dei residui, questo ci permetterà di scegliere il test opportuno per la significatività dei fattori.

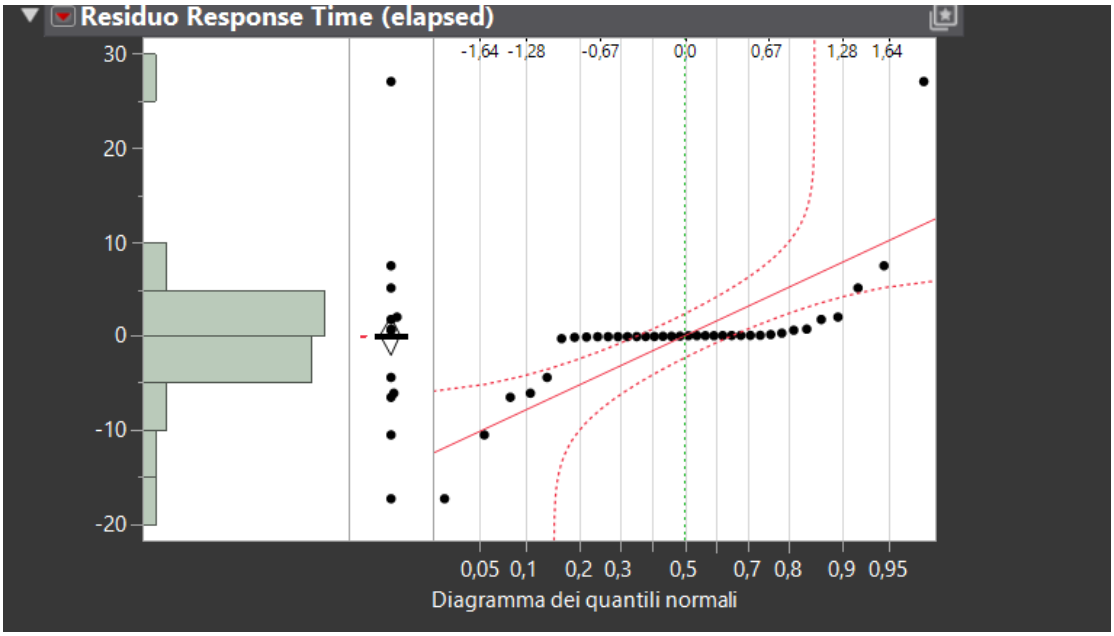


Figura 95: QQ plot Residui

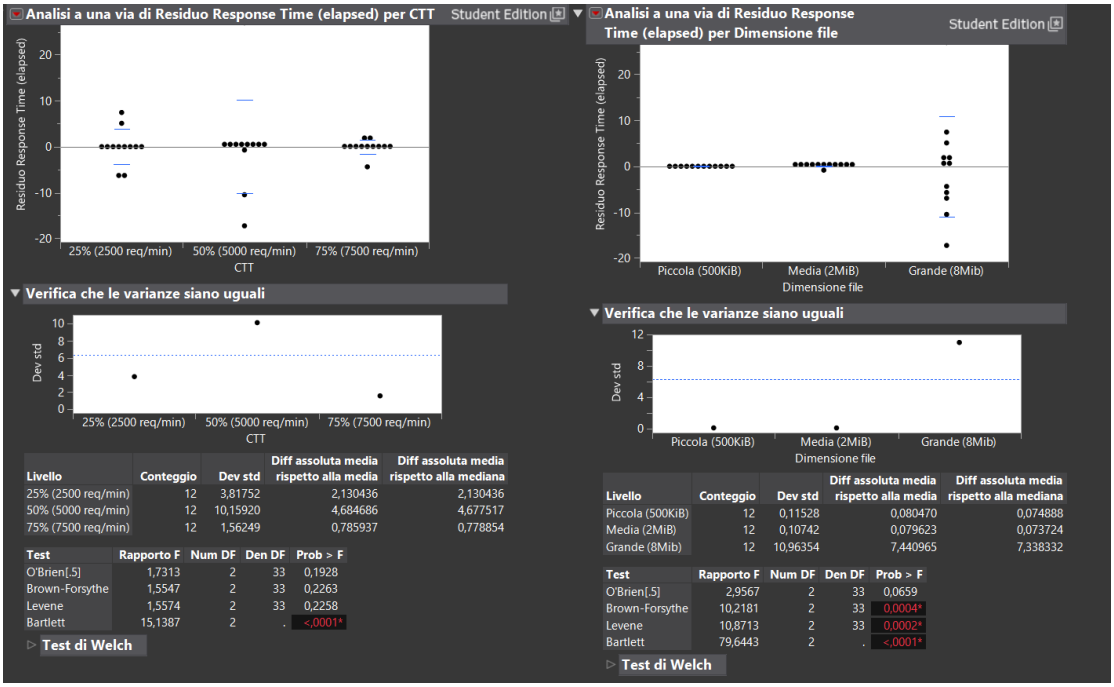


Figura 96: Analisi Varianza

Con un diagramma QQ è facile notare come i residui non si distribuiscono secondo una normale, mentre per quanto riguarda la omoschedasticità la evidenza è meno marcata ma grazie ai test fatti (sebbene non all'unanimità) possiamo concludere che almeno rispetto alle dimensioni del file i residui non sono neanche omoschedastici.

5.1.3 Test di significatività

Avendo appurato la non normalità dei residui si è optato per un test non parametrico, in particolare quello di Kruskal-Wallis.

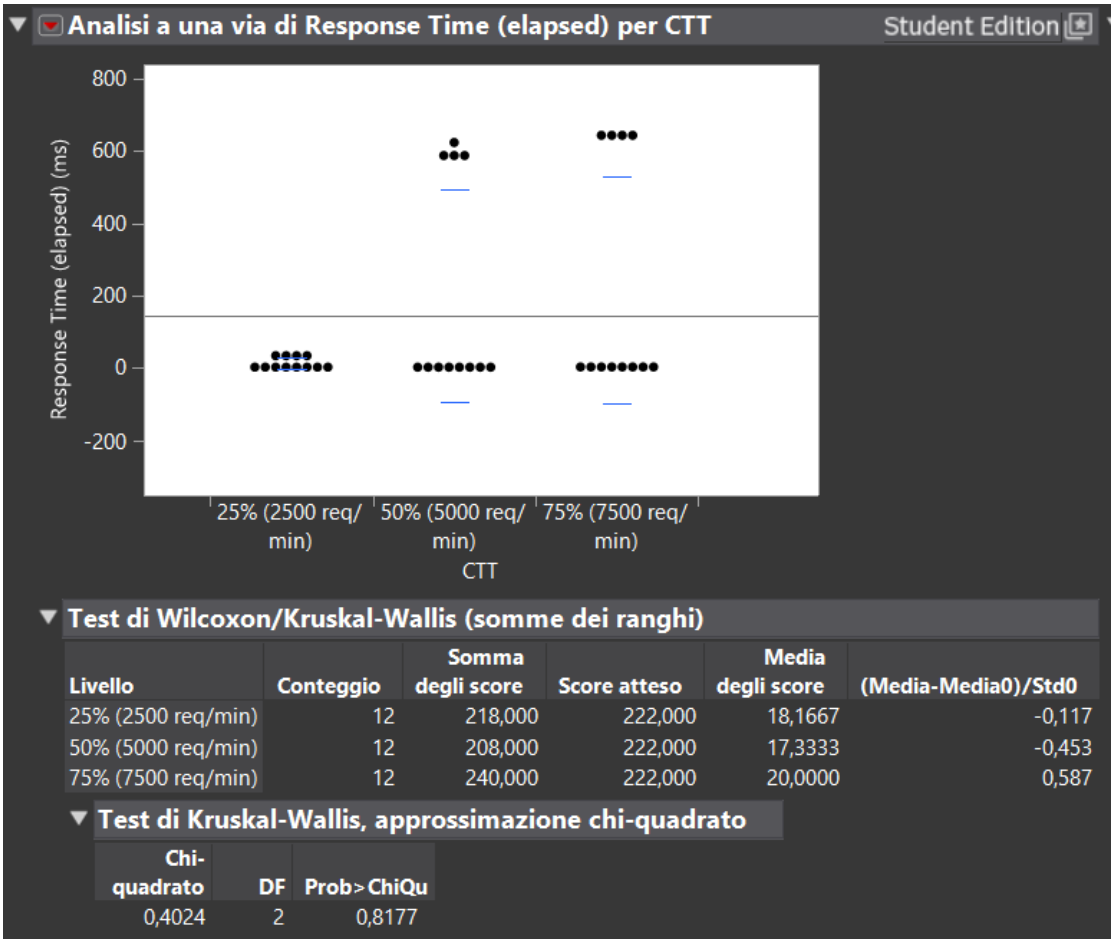


Figura 97: KW per CTT

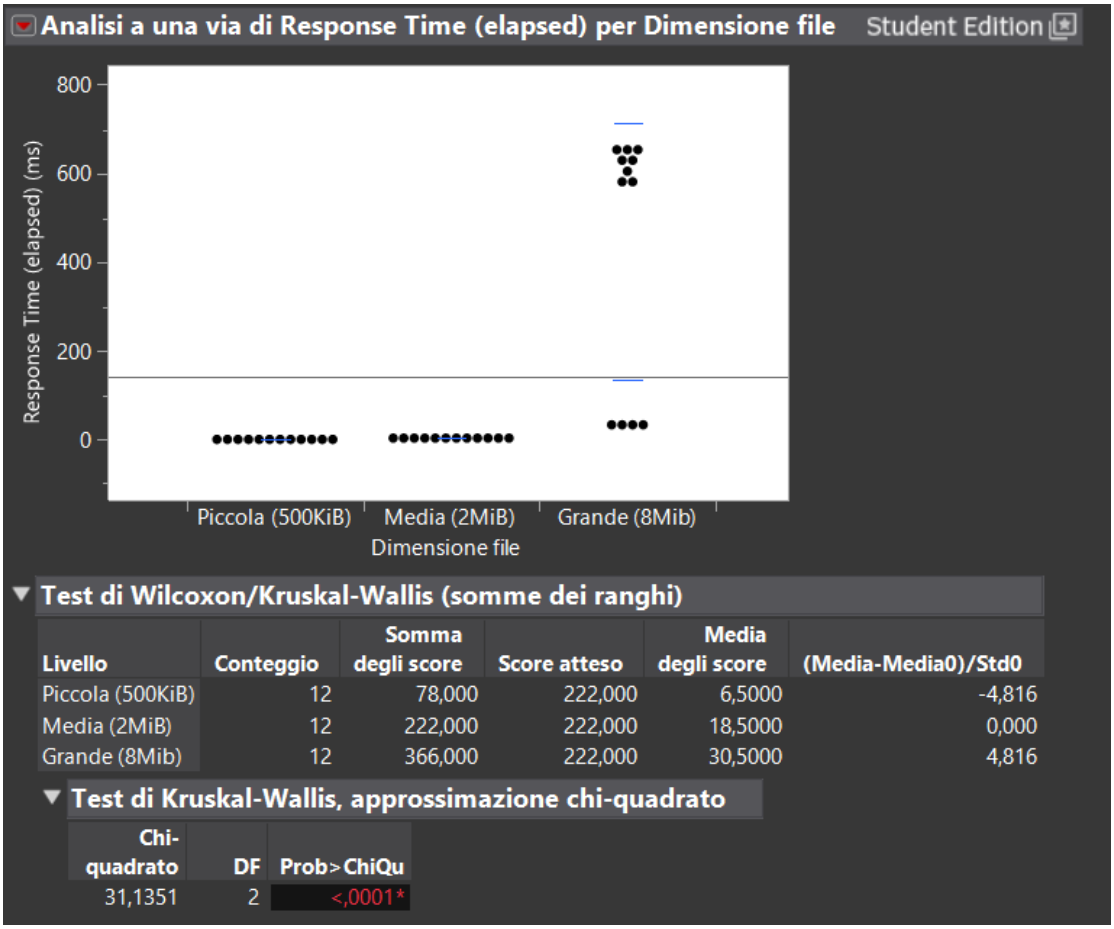


Figura 98: KW per Dimensione File

Da questo ultimo test possiamo notare come in realtà il CTT non è un fattore significativo nello spiegare la varianza, mentre possiamo dire che la dimensione dei file lo è.

5.2 Throughput

In questa sezione viene analizzato l'impatto dei fattori sulla variabile di risposta **Throughput**. L'esperimento segue un disegno fattoriale 3^2 con 4 repliche per ogni combinazione, per un totale di 36 osservazioni, come mostrato nella Figura 99.

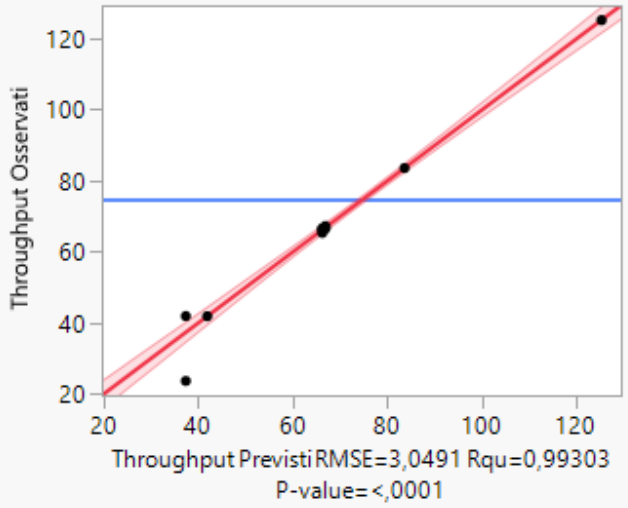
CTT	Dimensione file	Throughput (req/s)
25% (2500 req/min) 50% (5000 req/min) 75% (7500 req/min)	Piccola (500KiB) Media (2MiB) Grande (8Mib)	125 23,7
25% (2500 req/min)	Grande (8Mib)	41,90831
25% (2500 req/min)	Grande (8Mib)	41,908511
25% (2500 req/min)	Grande (8Mib)	41,902284
25% (2500 req/min)	Grande (8Mib)	41,898789
25% (2500 req/min)	Media (2MiB)	41,929351
25% (2500 req/min)	Media (2MiB)	41,921227
25% (2500 req/min)	Media (2MiB)	41,927054
25% (2500 req/min)	Media (2MiB)	41,92519
25% (2500 req/min)	Piccola (500KiB)	23,686803
25% (2500 req/min)	Piccola (500KiB)	41,931449
25% (2500 req/min)	Piccola (500KiB)	41,927754
25% (2500 req/min)	Piccola (500KiB)	41,927287
50% (5000 req/min)	Grande (8Mib)	66,629971
50% (5000 req/min)	Grande (8Mib)	66,612167
50% (5000 req/min)	Grande (8Mib)	65,314088
50% (5000 req/min)	Grande (8Mib)	66,197285
50% (5000 req/min)	Media (2MiB)	83,566853
50% (5000 req/min)	Media (2MiB)	83,562177
50% (5000 req/min)	Media (2MiB)	83,567767
50% (5000 req/min)	Media (2MiB)	83,568696
50% (5000 req/min)	Piccola (500KiB)	83,565986
50% (5000 req/min)	Piccola (500KiB)	83,575185
50% (5000 req/min)	Piccola (500KiB)	83,576547
50% (5000 req/min)	Piccola (500KiB)	83,576147
75% (7500 req/min)	Grande (8Mib)	66,766379
75% (7500 req/min)	Grande (8Mib)	66,497956
75% (7500 req/min)	Grande (8Mib)	67,25647
75% (7500 req/min)	Grande (8Mib)	66,868354
75% (7500 req/min)	Media (2MiB)	125,20361
75% (7500 req/min)	Media (2MiB)	125,19806
75% (7500 req/min)	Media (2MiB)	125,18276
75% (7500 req/min)	Media (2MiB)	125,2057
75% (7500 req/min)	Piccola (500KiB)	125,22793
75% (7500 req/min)	Piccola (500KiB)	125,22101
75% (7500 req/min)	Piccola (500KiB)	125,22725
75% (7500 req/min)	Piccola (500KiB)	125,21963

Figura 99: Osservazioni per il Throughput (req/s).

5.2.1 Stima del modello e analisi della varianza

Per valutare l'importanza relativa dei fattori e delle loro interazioni, è stata effettuata la scomposizione della Sum of Squares Total(SST). I valori sono stati estratti dall'analisi della varianza e dal test degli effetti del modello riportati nella Figura 100.

Grafico delle risposte osservate rispetto alle risposte previste



Riepilogo della stima

R-quadro	0,993028
R-quadro corretto	0,990962
Scarto quadratico medio	3,049073
Media della risposta	74,64394
Osservazioni (o somme pesate)	36

Analisi della varianza

Origine	DF	Somma dei quadrati	Media quadratica	Rapporto F
Modello	8	35749,846	4468,73	480,6718
Errore	27	251,015	9,30	Prob > F
C. totale	35	36000,861		<,0001*

Test degli effetti

Origine	Nparm	DF	Somma dei quadrati	Rapporto F	Prob > F
CTT	2	2	25805,620	1387,870	<,0001*
Dimensione file	2	2	4814,015	258,9059	<,0001*
CTT*Dimensione file	4	4	5130,211	137,9557	<,0001*

Figura 100: Riepilogo della stima e analisi della varianza per il Throughput.

L'allocazione della variazione è la seguente:

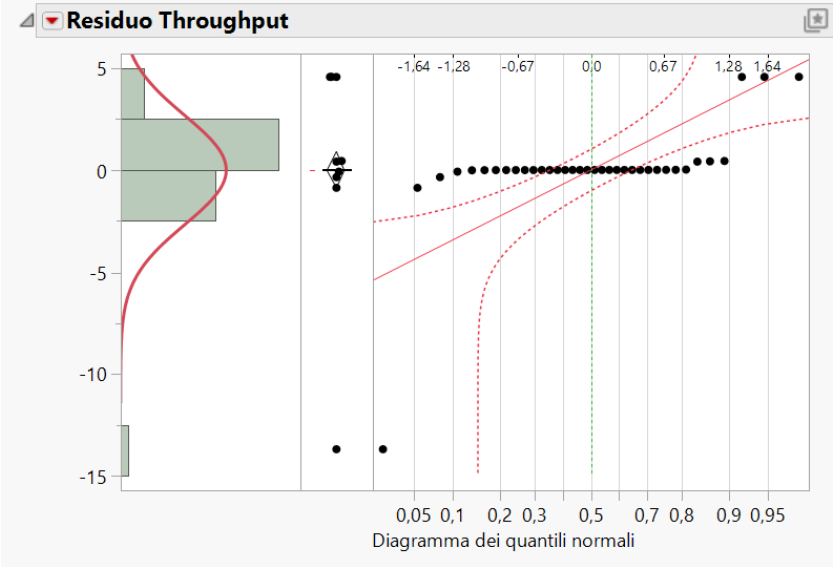
- **SST (Sum of square totale):** $36000,861$
- **Importanza CTT:** $\frac{25805,620}{36000,861} \approx 71,68\%$

- **Importanza Dimensione File:** $\frac{4814{,}015}{36000{,}861} \approx 13,37\%$
- **Importanza Interazione (CTT*Dim. File):** $\frac{5130{,}211}{36000{,}861} \approx 14,25\%$
- **Importanza Errore:** $\frac{251{,}015}{36000{,}861} \approx 0,70\%$

Al contrario del response time, il fattore con maggiore importanza per il Throughput è il **CTT**, che spiega oltre il 71% della variazione totale. L'interazione tra i due fattori ($14,25\%$) risulta leggermente superiore della sola dimensione del file ($13,37\%$), mentre l'errore sperimentale è estremamente contenuto ($0,70\%$).

5.2.2 Normalità e omoschedasticità

Prima di procedere con i test di significatività, sono state verificate le assunzioni di normalità e omoschedasticità dei residui.



(a) Diagramma dei quantili normali.

Test della bontà di adattamento

Test W di Shapiro-Wilk

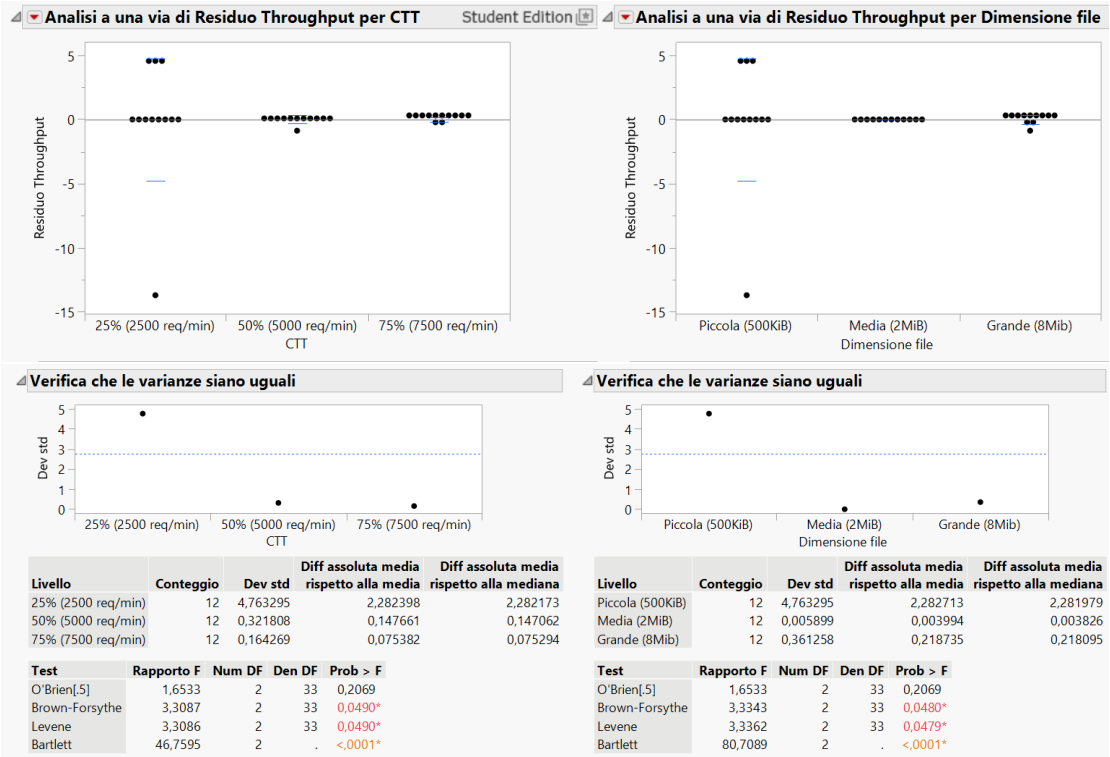
W	Prob<W
0,414867	<,0001*

Nota: Ho = i dati provengono dalla distribuzione Normale. I p-value bassi rifiutano Ho.

(b) Test di Shapiro-Wilk.

Figura 101: Verifica della normalità dei residui del Throughput.

Come si evince dalla Figura 101b, il test di **Shapiro-Wilk** restituisce un p-value $< 0,0001$. Essendo inferiore a $\alpha = 0,05$, l'ipotesi nulla di normalità viene rifiutata. La Figura 101a conferma visivamente tale risultato.



Livello	Conteggio	Dev std	Diff assoluta media rispetto alla media	Diff assoluta media rispetto alla mediana
25% (2500 req/min)	12	4,763295	2,282398	2,282173
50% (5000 req/min)	12	0,321808	0,147661	0,147062
75% (7500 req/min)	12	0,164269	0,075382	0,075294

Test	Rapporto F	Num DF	Den DF	Prob > F
O'Brien[.5]	1,6533	2	33	0,2069
Brown-Forsythe	3,3087	2	33	0,0490*
Levene	3,3086	2	33	0,0490*
Bartlett	46,7595	2	.	<,0001*

Livello	Conteggio	Dev std	Diff assoluta media rispetto alla media	Diff assoluta media rispetto alla mediana
Piccola (500KiB)	12	4,763295	2,282713	2,281979
Media (2MiB)	12	0,005899	0,003994	0,003826
Grande (8Mib)	12	0,361258	0,218735	0,218095

Test	Rapporto F	Num DF	Den DF	Prob > F
O'Brien[.5]	1,6533	2	33	0,2069
Brown-Forsythe	3,3343	2	33	0,0480*
Levene	3,3362	2	33	0,0479*
Bartlett	80,7089	2	.	<,0001*

Figura 102: Analisi dei residui e test di omoschedasticità.

Per quanto riguarda l'omoschedasticità i test di Brown-Forsythe, Levene e Bartlett presentano p-value inferiori a 0,05 per entrambi i fattori. Ciò indica che la varianza non è uniforme tra i livelli (eteroschedasticità).

5.2.3 Test di significatività

Data la violazione delle assunzioni di normalità e omoschedasticità, la significatività dei fattori è stata valutata tramite il test non parametrico di **Kruskal-Wallis** (Figura 103).

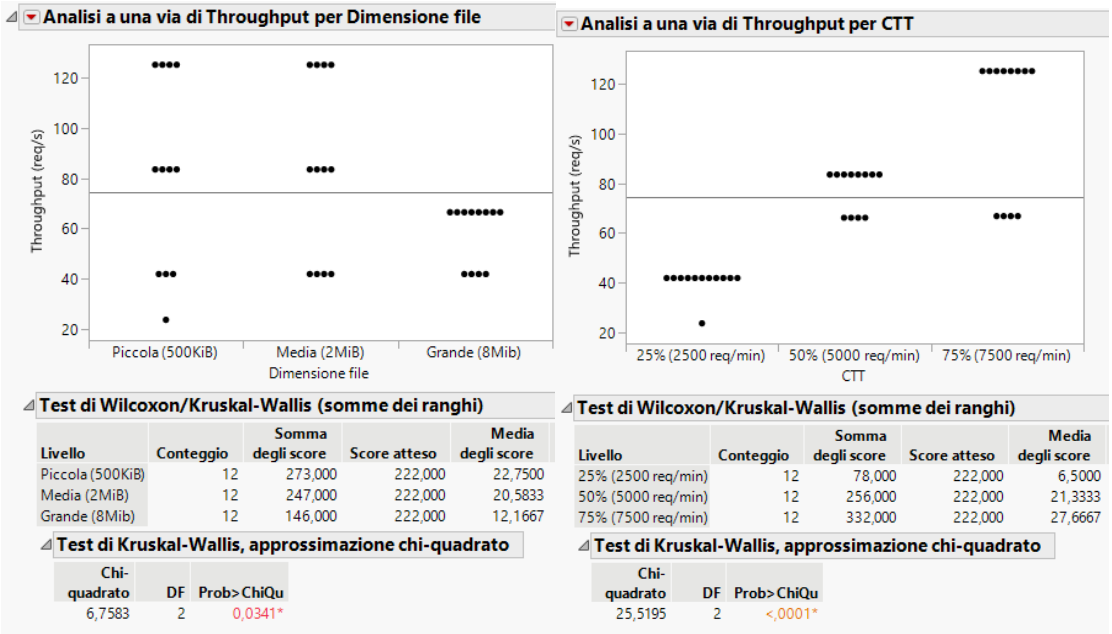


Figura 103: Test di Kruskal-Wallis per i fattori CTT e Dimensione File.

Dall'analisi dei risultati in Figura 103, entrambi i fattori risultano statisticamente **significativi** per il Throughput, presentando p-value inferiori a 0,05 ($< 0,0001$ per CTT e $0,0341$ per Dimensione file).

In conclusione, il fattore CTT risulta essere sia il più importante (spiegando la quota maggiore di variazione) sia il più significativo dal punto di vista statistico (rapportando questa stessa varianza all'errore).

5.3 Latency

Per analizzare la latency abbiamo utilizzato uno script Python ottenendo i seguenti risultati:

	Capacity	File_Size	Latency	Repetition
0	25	large	3.218124	1
1	25	large	2.077076	2
2	25	large	3.284899	3
3	25	large	2.111332	4
4	50	large	35.526535	1
5	50	large	34.139673	2
6	50	large	37.939048	3
7	50	large	35.289507	4
8	75	large	39.261659	1
9	75	large	40.562228	2
10	75	large	44.206614	3
11	75	large	40.239937	4
12	25	medium	0.959692	1
13	25	medium	1.026256	2
14	25	medium	0.968174	3
15	25	medium	0.975335	4
16	50	medium	0.804923	1
17	50	medium	0.776011	2
18	50	medium	0.793893	3
19	50	medium	0.788504	4
20	75	medium	1.113983	1
21	75	medium	1.078689	2
22	75	medium	1.118527	3
23	75	medium	1.011056	4
24	25	small	0.686895	1
25	25	small	0.652082	2
26	25	small	0.599125	3
27	25	small	0.639968	4
28	50	small	29.888168	1
29	50	small	0.570117	2
30	50	small	0.523946	3
31	50	small	0.544977	4
32	75	small	0.642058	1
33	75	small	0.561762	2
34	75	small	0.560336	3
35	75	small	0.559211	4

Figura 104: Risultati latency

5.3.1 Stima del modello e analisi della varianza

Una volta ottenuto i dati, passiamo all'analisi dell'importanza dei fattori:

```
--- A. Generazione Modello e ANOVA ---
                            sum_sq    df          F        PR(>F)
C(Capacity)               1372.144588   2.0  27.698575  2.874724e-07
C(File_Size)              4825.940018   2.0  97.418060  4.487199e-13
C(Capacity):C(File_Size)  2229.040227   4.0  22.498081  2.881357e-08
Residual                   668.769120  27.0        NaN           NaN
```

Figura 105: Parametri Latency

Dai quali effettuiamo i calcoli sulle Sum of Squares:

$$SSA = 1372,133588 \qquad SSB = 4825,940018$$

$$SSAB = 2229,040227 \qquad SST = 9095,893953$$

$$\text{Varianza non spiegata} = \frac{SSE}{SST} = \frac{668,769120}{9095,893953} = 0,075 = \mathbf{7,5\%}$$

$$\text{Importanza A} = \frac{SSA}{SST} = \frac{1372,133588}{9095,893953} = 0,15 = \mathbf{15\%}$$

$$\text{Importanza B} = \frac{SSB}{SST} = \frac{4825,940018}{9095,893953} = 0,53 = \mathbf{53\%}$$

$$\text{Importanza AB} = \frac{SSAB}{SST} = \frac{2229,040227}{9095,893953} = 0,245 = \mathbf{24,5\%}$$

Quello che ricaviamo dai questi dati sono le stesse osservazioni effettuate sul Response Time, in particolare sull'importanza del fattore B e sull'interazione tra fattore A e fattore B.

5.3.2 Normalità e omoschedasticità

Per l'analisi di normalità e omoschedasticità andiamo a visualizzare il plot Q-Q:

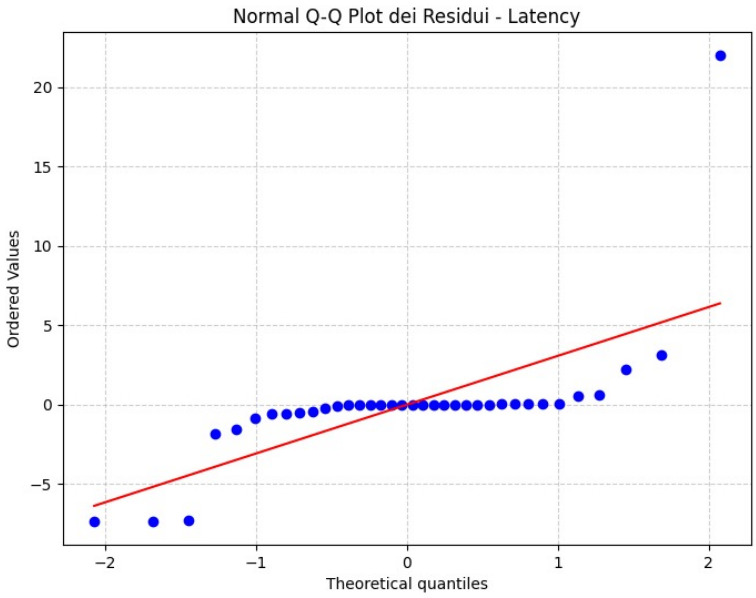


Figura 106: Q-Q plot

Anche in questo caso i residui non sono distribuiti secondo una normale e l'omoschedasticità non è marcata.

Effettuiamo dei test numerici per confermare la validità delle nostre assunzioni:

- **Normalita**: per la normalità dei residui, avendo pochi campioni, andiamo a effettuare il test di Shapiro-Wilk:

```
--- Test di Normalità (Shapiro-Wilk sui Residui) ---
   Oggetto del test: Residui del modello (Y_real - Y_pred)
   Risultato: W=0.4995, P-value=6.2932e-10
   >> ESITO: VIOLATO. I residui NON sono normali.
```

Figura 107: Test di Shapiro-Wilk

- **Omoschedasticità**: per verificare l'omoschedasticità effettuiamo il test di Levene, più robusto quando abbiamo pochi campioni:

```
--- Test di Omocedasticità (Levene) ---
   A. Fattore Capacity:  P-value=6.2527e-02 -> Omocedastico (Varianze Uguali)
   B. Fattore File Size: P-value=5.7770e-03 -> ETEROSCEDASTICO (Varianze Diverse)
```

Figura 108: Test di Levene

5.3.3 Test di significatività

Una volta cadute sia l'assunzione di normalità sia quella di omoschedasticità, possiamo passare al test non parametrico. In particolare, effettueremo il test di Kruskal-Wallis:

```
RISULTATI KRUSKAL-WALLIS:
- Impatto Capacity:  P=7.5917e-01 (Trascurabile)
- Impatto File Size: P=1.2891e-06 (SIGNIFICATIVO)
```

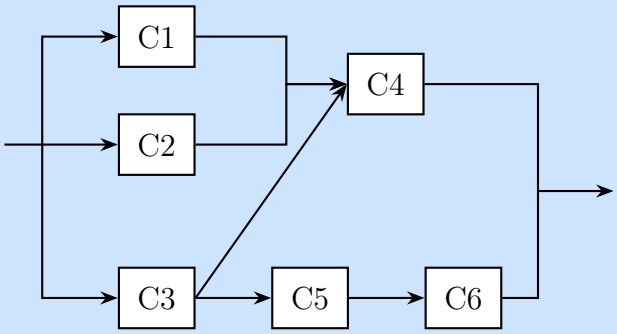
Figura 109: Test di Kruskal-Wallis

Anche in questo caso si vede che a essere significativa non è la capacity, bensì la file size.

6 Relay Diagram Block

6.1 Esercizio 1

Find the $R(t)$ and MTTF for the system whose reliability diagram is given below. In calculating MTTF, assume all components are identical and fail randomly with failure rate λ.

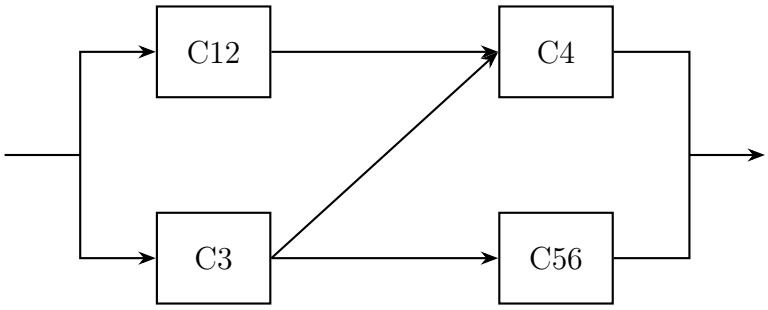


Innanzitutto definiamo la reliability dei vari componenti, in quanto uguali, come:

$$R_1 = R_2 = R_3 = R_4 = R_5 = R_6 = R = e^{-\lambda t}$$

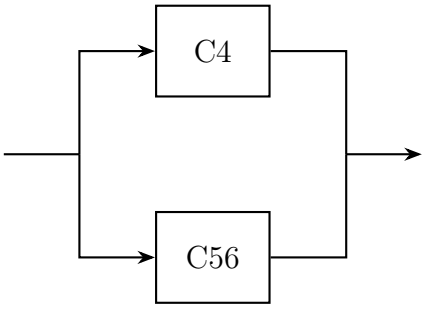
La prima cosa che notiamo è che C1 e C2 sono in parallelo e C5 e C6 sono in serie. Applichiamo le formule e ridisegniamo il grafico:

$$R_{C_1C_2} = 1 - (1 - R_1)(1 - R_2) = 1 - (1 - R)^2$$
$$R_{C_5C_6} = R_5 R_6 = R^2$$



Da questo punto in poi applichiamo il **Conditioning**, andando a considerare come componente condizionato C3. Prendiamo quindi in considerazione due casistiche diverse:

- C3 funziona, diventando quindi un **corto circuito**
- C3 non funziona, diventando quindi un **circuito aperto**

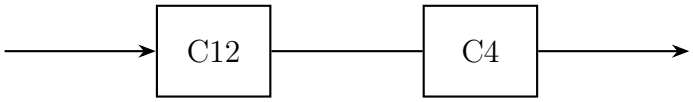


Corto circuito Il corto circuito porterà alla cortocircuitazione anche di C12, portando quindi al seguente grafico:

Il calcolo è, quindi, il seguente parallelo:

$$R_{UP} = 1 - (1 - R_4)\left(1 - R^2\right) = 1 - (1 - R)^2 (1 + R)$$

Circuito aperto In questo caso l'aperto appena creato cancella anche C56. Il grafico risultante è il seguente:



Il calcolo è il seguente:

$$R_{DOWN} = \left[1 - \left(1 - R^2\right)\right] R_4 = \left[1 - (1 - R)^2\right] R$$

Risultato finale Combiniamo i due risultati ottenuti utilizzando la legge della probabilità totale:

$$\begin{aligned}
R_{sys} &= R_3 R_{UP} + (1 - R_3) R_{DOWN} = \\
&= R\left[1 - (1 - R)^2 (1 + R)\right] + (1 - R)\left[1 - (1 - R)^2\right] R = \\
&= R\left[1 - \left(R^2 - 2R + 1\right)(1 + R)\right] + \left(R - R^2\right)\left(2R - R^2\right) = \\
&= R\left(-R^3 + 2R^2 - R - R^2 + 2R\right) + 2R^2 - R^3 - 2R^3 + R^4 = \\
&= -R^4 + 2R^3 - R^2 - R^3 + 2R^2 + 2R^2 - 3R^3 + R^4 = \\
&= -2R^3 + 3R^2
\end{aligned}$$

Applicando la definizione iniziale della reliability otteniamo che:

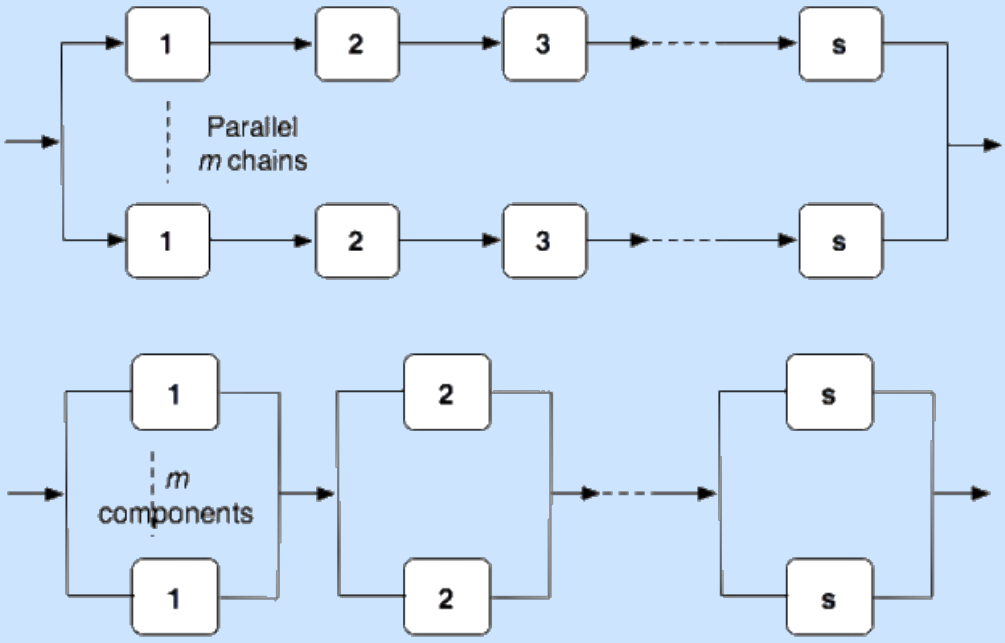
$$R_{sys} = -2e^{-3\lambda t} + 3e^{-2\lambda t}$$

Adesso andiamo a calcolare il **Mean Time To Failure** (MTTF):

$$MTTF = \int_0^{+\infty} R_{sys} dt = \int_0^{+\infty} -2e^{-3\lambda t} + 3e^{-2\lambda t} dt =$$
$$= -2\int_0^{+\infty} e^{-3\lambda t} dt + 3\int_0^{+\infty} e^{-2\lambda t} dt = \left[\frac{2}{3\lambda} e^{-3\lambda t} - \frac{3}{2\lambda} e^{-2\lambda t}\right]_0^{+\infty} =$$
$$= \frac{3}{2\lambda} - \frac{2}{3\lambda}$$
$$MTTF = \frac{5}{6\lambda}$$

6.2 Esercizio 2

We want to compare two different schemes of increasing reliability of a system using redundancy. Suppose that the system needs s identical components in series for proper operation. Further suppose that we are given $(m \times s)$ components. Given that the reliability of an individual component is R, derive the expressions for the reliabilities of two configurations. For $m = 2$ and $s = 4$, compare the two expressions as function of a mission time t.
Let MTTF of the component be 800 hours.
Out of the two schemes shown in the figure below, which one will provide a higher reliability? Modify the scheme that has lower reliability in order to reach the same reliability of the other given the above MTTF (800 h).



Calcoliamo la reliability di una singola catena lunga s:

$$R_{chain}(t) = \prod_{i=1}^{s} R_i = R^s$$

Dalla quale calcoliamo la reliability complessiva del sistema, data dal parallelo tra m catene:

$$R_{sys,1}(t) = 1 - \prod_{i=1}^{m} [1 - R_{chain,i}(t)] = 1 - (1 - R^s)^m$$

Per il secondo sistema calcoliamo innanzitutto la reliability di un singolo componente grande m:

$$R_{comp}(t) = 1 - \prod_{i=1}^{m} (1 - R) = 1 - (1 - R)^m$$

Per la reliability complessiva del secondo sistema calcoliamo la serie, lunga s, di componenti:

$$R_{sys,2}(t) = \prod_{i=1}^{s} R_{comp,i}(t) = (1 - (1 - R)^m)^s$$

Svolgendo i calcoli con $m = 2$ e $s = 4$ otteniamo che:

$$R_{sys,1}(t) = 1 - \left(1 - R^4\right)^2 = 1 - \left(R^8 - 2R^4 + 1\right) = -R^8 + 2R^4$$
$$R_{sys,2}(t) = \left(1 - (1 - R)^2\right)^4 = R^8 - 8R^7 + 24R^6 - 32R^5 + 16R^4$$

Andiamo a considerare una reliability esponenziale. Le reliability dei due sistemi saranno:

$$R_{sys,1}(t) = -e^{-8\lambda t} + 2e^{-4\lambda t}$$
$$R_{sys,2}(t) = e^{-8\lambda t} - 8e^{-7\lambda t} + 24e^{-6\lambda t} - 32e^{-5\lambda t} + 16e^{-4\lambda t}$$

Considerando come failure rate $\lambda = \frac{1}{MTTF} = \frac{1}{800\ h} = \frac{1}{2.880.000\ s}$, realizziamo il grafico delle due funzioni.
Otteniamo le seguenti curve:

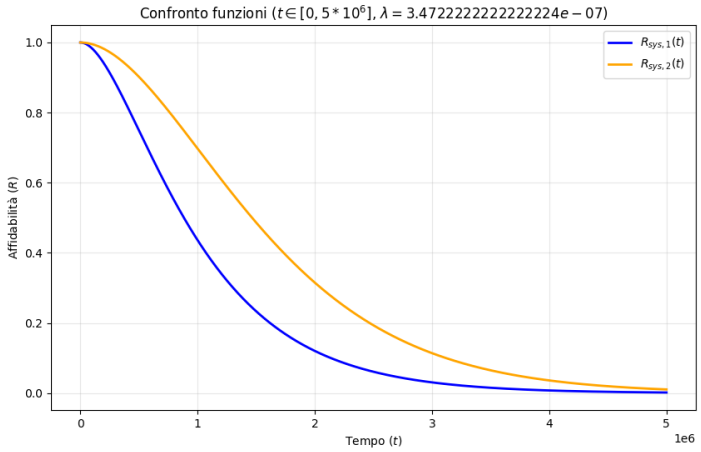


Figura 110: Confronto tra le reliability

Dal grafico di vede che il secondo sistema offre una maggiore reliability.

Adesso proviamo a eguagliare le reliability dei due sistemi modificando il numero m di catene in parallelo del primo sistema. Per fare ciò eguagliamo le due reliability all'istante di tempo $t = MTTF$:

$$
\begin{aligned}
R_{sys,1}\left(MTTF\right) &= R_{sys,2}\left(MTTF\right) \\
1 - \left(1 - e^{-4}\right)^m &= e^{-8} - 8e^{-7} + 24e^{-6} - 32e^{-5} + 16e^{-4} \\
1 - \left(1 - e^{-4}\right)^m &= 0.13 \\
\left(1 - e^{-4}\right)^m &= 0.87 \\
\left(0.982\right)^m &= 0.87 \\
e^{m\ ln(0.982)} &= e^{ln(0.87)} \\
m &= \frac{ln\left(0.87\right)}{ln\left(0.982\right)} \approx 7.50
\end{aligned}
$$

Andiamo quindi a scegliere $m = 8$. Andando a fare il grafico della nuova funzione notiamo che fino a $t = MTTF$ il sistema 1 ha una maggiore reliability:

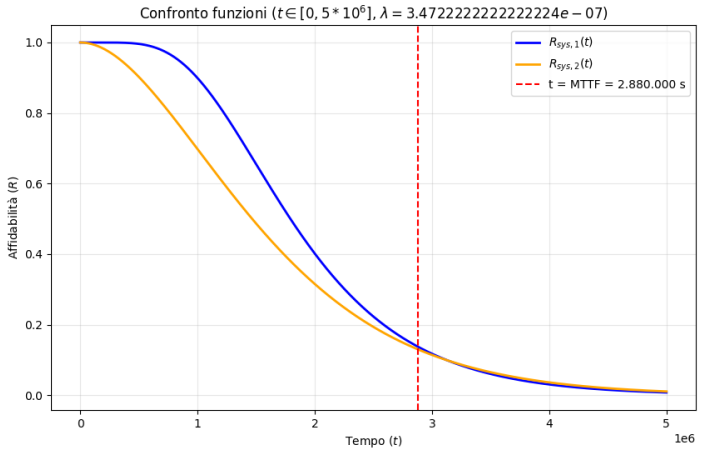
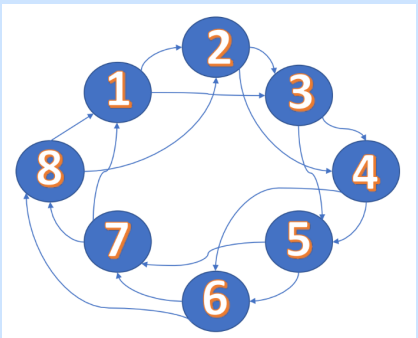


Figura 111: Grafico con $m = 8$

6.3 Esercizio 3

The architecture of a network of computers in a banking system is shown below. The architecture is called a skip-ring network and is designed to allow processors to communicate even after node failures have occurred. For example, if node 1 fails, node 8 can bypass the failed node by routing data over the alternative link connecting nodes 8 and 2. Assuming the links are perfect and the nodes each have a reliability of R_m, derive an expression for the reliability of the network. If R_m obeys the exponential failure law and the failure rate of each node is 0.005 failures per hour, determine the reliability of the system at the end of a 48-hour period.



Il sistema descritto è una rete a 8 nodi denominata "skip-ring": ogni nodo, come vediamo, è collegato con i suoi due predecessori e con i suoi due successori secondo un sistema ad anello. Dal testo si evince che il sistema fallisce se si interrompe la catena di comunicazione, ovvero se si verificano guasti in due nodi consecutivi.

Reliability di un singolo nodo Sappiamo che ogni nodo ha un tasso di guasto costante $\lambda = 0.005$ guasti/ora e poichè la R_m segue la legge esponenziale otteniamo

per un tempo $t = 48$:

$$R_m(t) = e^{-\lambda t} = e^{-0.005 \cdot 48} = e^{-0.24} \approx 0.7866$$

Modello Poiché i nodi sono disposti ad anello, esiste una dipendenza ciclica che impedisce l'uso delle classiche formule serie/parallelo. Usiamo allora il metodo del conditioning su un nodo dell'anello. Usando il conditioning sul nodo 1 possiamo scrivere la Reliability del sistema R_{sys} come somma di due scenari mutuamente esclusivi

1. **Nodo 1 Funzionante (R_m):** Se assumiamo che il nodo 1 funzioni le coppie (8,1) e (1,2) non possono fallire. Segue che non abbiamo più una dipendenza ciclica nella rete e dobbiamo preoccuparci solo dei rimanenti 7 nodi(L_7).

2. **Nodo 1 Guasto ($1 - R_m$):** Affinché il sistema non fallisca subito, i nodi adiacenti (8 e 2) devono funzionare (R_m^2). Secondo un discorso analogo si interrompe la ciclicità di dipendenze e ci preoccupiamo solo della catena dei 5 nodi rimanenti(L_5).

La formula complessiva è dunque:

$$R_{sys} = R_m \cdot L_7 + (1 - R_m) \cdot R_m^2 \cdot L_5 \tag{1}$$

A questo punto dobbiamo calcolare la reliability di una generica catena L_k È fondamentale notare che tale configurazione non va confusa con un sistema in serie. In un sistema in serie, il guasto di un singolo componente causa il fallimento dell'intero sistema ($R_{serie} = R^k$). Qui invece un guasto della catena è dovuto al guasto di due nodi consecutivi.

Per calcolare quindi la reliability usiamo ancora una volta il conditioning su un generico elemento. Si presentano due casistiche:

Caso A: Se il primo nodo funziona (probabilità R_m),il problema si riduce a garantire l'affidabilità della catena rimanente di lunghezza $k - 1$.

$$\text{Contributo} = R_m \cdot L_{k-1}$$

Caso B: Se il primo nodo è guasto (probabilità $1 - R_m$), per evitare il fallimento immediato (due guasti consecutivi), il secondo nodo deve necessariamente funzionare (probabilità R_m). Avendo fissato lo stato dei primi due nodi, il problema si riduce a garantire l'affidabilità della catena rimanente, che ora ha lunghezza $k - 2$.

$$\text{Contributo} = (1 - R_m) \cdot R_m \cdot L_{k-2}$$

Sommando le probabilità dei due eventi disgiunti, otteniamo la legge ricorsiva:

$$L_k = R_m \cdot L_{k-1} + (1 - R_m) R_m \cdot L_{k-2}$$

Calcolo Numerico Ponendo i casi base $L_0 = 1$ e $L_1 = 1$ (è impossibile avere 2 guasti consecutivi in un solo nodo), calcoliamo i valori iterativi con $R_m = 0.7866$:

$$
\begin{aligned}
L_2 &= 0.7866(1) + 0.2134(0.7866)(1) = 0.9545 \\
L_3 &= 0.7866(0.9545) + 0.1679(1) = 0.9186 \\
L_4 &= 0.7866(0.9186) + 0.1679(0.9545) = 0.8826 \\
L_5 &= 0.7866(0.8826) + 0.1679(0.9186) = 0.8483 \\
L_6 &= 0.7866(0.8483) + 0.1679(0.8826) = 0.8153 \\
L_7 &= 0.7866(0.8153) + 0.1679(0.8483) = 0.7836
\end{aligned}
$$

Sostituendo infine i valori di L_7 e L_5 nella formula del sistema circolare (Eq. 1):

$$R_{sys} = (0.7866 \cdot 0.7836) + (0.2134 \cdot 0.7866^2 \cdot 0.8483)$$

$$R_{sys} = 0.6164 + 0.1120 = 0.7284$$

Possiamo concludere che l'affidabilità del sistema al termine del periodo di 48 ore è pari a circa il **72.84%**.

6.4 Esercizio 4

A data center relies on a cooling system to maintain optimal operating temperatures for its servers. The cooling system is composed of the following components:

- Main Cooling Unit (C): Provides primary cooling. MTTFC=10000h
- Backup Cooling Unit (B): Activates only if the Main Cooling Unit fails. MTTFB=5000h
- Power Supply (P): Provides power to the cooling units. MTTFP=7000h
- Control Panel (CP): Monitors and manages the cooling process. MTTFCP=6000h

The cooling system operates as follows. The Main Cooling Unit and the Power Supply must work together in series for primary cooling. If the Main Cooling Unit fails, the Backup Cooling Unit activates, provided the Power Supply is still operational. The Control Panel must always function.

1. **Draw the RBD.** Represent the system as an RBD, showing the series and parallel arrangements, including the redundancy of the cooling units.
2. **Calculate System Reliability.** Compute the overall reliability of the cooling system.
3. **Impact of Redundancy** Discuss how the addition of the Backup Cooling Unit affects the system reliability compared to a simplex system (without redundancy for the cooling units). Calculate the system reliability and visualize how the reliability changes with varying MTTFB values.

La traccia spiega chiaramente la combinazione di parti del nostro sistema che devono funzionare insieme (serie) e quelle che invece permettono invece di essere intercambiabili (parrallelo). La disposizione dei componenti nel diagram block segue qui sotto.

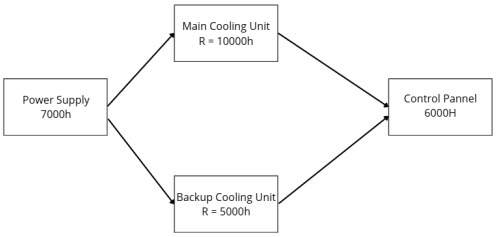


Figura 112: DBG Cooling System

Quindi come si evince dalla figura i due componenti di Main Cooling e Backup Cooling sono in parallelo e il loro parallelo è in serie con tutto il resto.

Per il calcolo della reliability dell'intero sistema possiamo usare le formule relative alle seire e paralleli

$$R_{sis} = R_P * (1 - (1 - R_C) * (1 - R_B)) * R_{CP}$$

Ora dato che la traccia ci mette a disposizione i MTTF dobbiamo da questi ricavare l'affidabilità del nostro sistema. Definiamo il MTTF come:

$$MTTF = \frac{1}{\lambda}$$

E da qui per la reliability:

$$R = e^{-\lambda t}$$

Sostituendo quindi nella formula del nostro sistema avremo

$$R_{sis} = e^{-\lambda_P t} * (1 - (1 - e^{-\lambda_C t}) * (1 - e^{-\lambda_B t})) * e^{-\lambda_{CP} t} =$$
$$= e^{-(\lambda_P + \lambda_{CP})t} * (e^{-\lambda_C t} + e^{-\lambda_B t} - e^{-(\lambda_B + \lambda_C)t}))$$

Se dovessimo rimuovere il Backup intuitivamente la reliability dovrebbe scendere e infatti come mostrato nel grafico possiamo notare che durante tutto il mission time la reliability del sistema 2 è piu bassa con equazione:

$$R_{sis} = e^{-(\lambda_P + \lambda_C + \lambda_{CP})t}$$

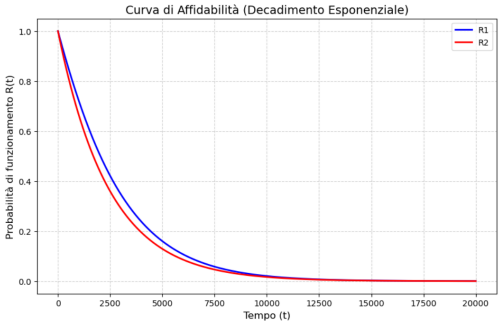


Figura 113: Reliability R1 e R2

Di seguito abbiamo invece osservato come cambia la reliability se andassimo a modificare la MTTF del componente di backup

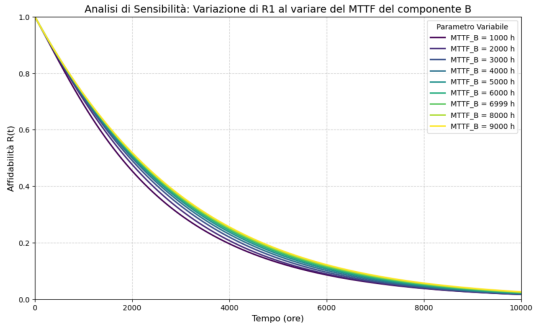
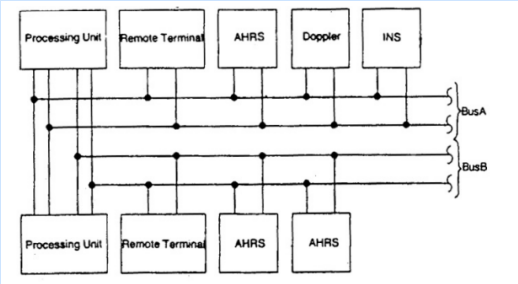


Figura 114: Reliability R1 e R2

Come si puo notare all'aumentare della MTTFB aumenta, sebbene leggermente, anche la reliability complessiva. Tutte queste differenze si notano molto di piu intorno a tempi di missione che vanno dalle 2000 alle 3500 ore circa, andando piano piano a ridursi.

6.5 Esercizio 5

The system shown in the figure below is a processing system for a helicopter. The system has dual-redundant processors and dual-redundant interface units. Two buses are used in the system, and each bus is also dual-redundant. The interesting part of the system is the navigation equipment. The aircraft can be completely navigated using the Inertial Navigation System (INS). If the INS fails, the aircraft can be navigated using the combination of the Doppler and the altitude heading and reference system (AHRS). The system contains three AHRS units, of which only one is needed. This is an example of functional redundancy where the data from the AHRS and the Doppler can be used to replace the INS, if the INS fails. Because of the other sensors and instrumentation, both buses are required for the system to function properly regardless of which navigation mode is being employed.



1. Draw the reliability block diagram of the system.
2. Draw the Fault Tree of the system and analyze the minimal cutsets.
3. Calculate the reliability for a one-hour flight using the MTTF figures given in the table below. Assume that the exponential failure low applies and that the fault coverage is perfect.

Equipment	**MTTF(hr)**
Processing Unit	10000
Remote Terminal	4500
AHRS	2000
INS	2000
Doppler	500
Bus	60000

4. Repeat (2), but this time, incorporate a coverage factor for the fault detection and reconfiguration of the processing units. Using the same failure data, determine the approximate fault coverage value that is required to obtain a reliability (at the end of one hour) of 0.99999.

Di seguito abbiamo i diagrammi a blocchi del sistema e i fault tree.

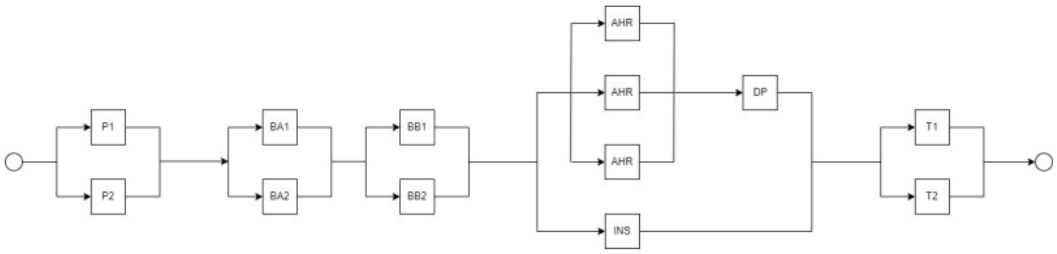


Figura 115: RBD

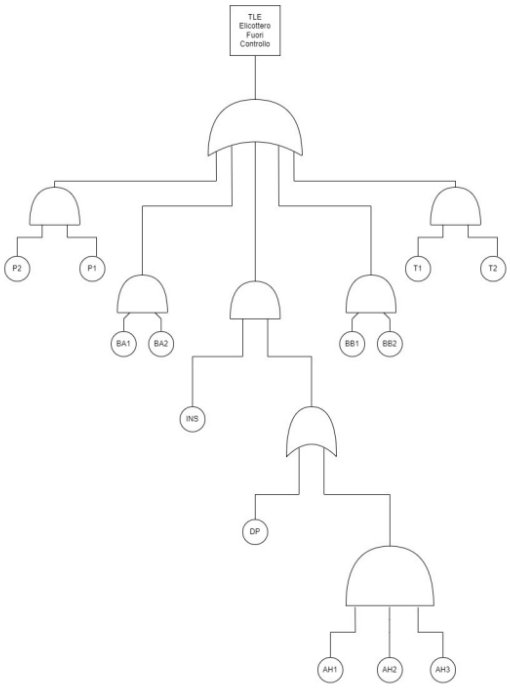


Figura 116: Fault tree

Dal Failure Tree è possibile ricavare la formula che mette in evidenza i minimal cut set

$$\Phi = (P1*P2)+(BA1*BA2)+(BB1*BB2)+(T1*T2)+(INS*(DP+(AH1*AH2*AH3))$$

Da qui i tagli minimi sono evidenti

1. (P1 * P2)
2. (BA1*BA2)
3. (BB1* BB2)
4. (T1 * T2)

5. (INS*(DP+(AH1*AH2*AH3)))

A questo punto possiamo usare il diagramma a blocchi per calcolarci la formula della reliability del sistema.

$$R_{Sys} = (1-(1-R_P)^2) * (1-(1-R_{BA})^2) * (1-(1-R_{BB})^2) \\ * (1-(1-(R_{DP} * (1-(1-R_{AHR})^3)) * (1-R_{INS}))) * (1-(1-R_T)^2$$

Per il calcolo dell'affidabilità dopo 1 ora di servizio basta sostituire

$$R = e^{-\frac{t}{MTTF}}$$

e settare il tempo t (in ore) a 1.

$$R_{Sys}(1h) \approx 0,99999$$

Per quanto riguarda la coverage non ottima sulla processing unit abbiamo

$$R_{Sys} = (R_P + cR_p(1-R_P)) * (1-(1-R_{BA})^2) * (1-(1-R_{BB})^2) \\ * (1-(1-(R_{DP} * (1-(1-R_{AHR})^3)) * (1-R_{INS}))) * (1-(1-R_T)^2$$

Per ottenere quindi una Reliability di 0,99999 è stato stimato un c = 0.91

7 Field Failure Data Analysis

In questo capitolo effettueremo una log-based Field Failure Data Analysis su due sistemi di calcolo ad alte prestazioni: **Mercury** e **BlueGene/L**. L'obiettivo è analizzare i dati per identificare le classi di errore, modellare statisticamente i tempi di fallimento (TTF) e individuare i colli di bottiglia dei sistemi.

7.1 Sistema Mercury

7.1.1 Descrizione del Dataset e Architettura

Mercury è un cluster basato su nodi IBM. Il sistema è organizzato secondo un'architettura gerarchica a tre livelli, gestita da un nodo principale. I log analizzati includono le seguenti tipologie di nodi:

- **Head Node (`tg-master`):** Il nodo di gestione centrale del cluster.
- **Login Nodes (`tg-loginX`):** Nodi dedicati all'accesso degli utenti.
- **Compute Nodes (`tg-cX`):** Nodi dedicati all'esecuzione dei job di calcolo intensivo.
- **Storage Nodes (`tg-sX`):** Nodi dedicati alla gestione dello storage e I/O.

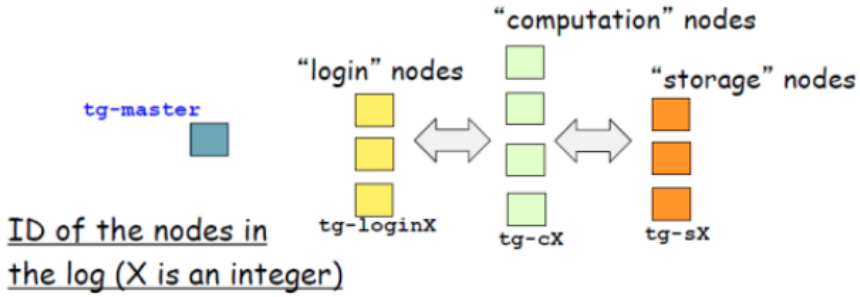


Figura 117: Schema generale dell'architettura del cluster Mercury

Il dataset è costituito da log di sistema che registrano eventi, avvisi e guasti hardware/software provenienti dalle varie componenti del cluster. Ogni singola entry del log segue un formato composto dai seguenti campi:

- **Timestamp:** L'istante temporale in cui l'evento è stato registrato.
- **Originating Node:** L'identificativo del nodo che ha generato l'evento.
- **Originating Subsystem:** Il sottosistema hardware o software specifico da cui proviene la segnalazione.
- **Text-free Message:** La descrizione testuale dell'evento o dell'errore riscontrato.

Dall'analisi preliminare del file `MercuryErrorLog.txt`, gli errori sono stati classificati nelle seguenti categorie principali basandosi sul sottosistema di origine:

- **DEV:** Errori dispositivi (es. PCI).
- **MEM:** Errori di memoria (RAM, ECC).
- **I-O:** Errori di Input/Output su storage.
- **NET:** Errori di rete/connessione.
- **PRO:** Errori relativi ai processori.

7.1.2 Analisi Globale del Sistema

La prima fase di elaborazione dei log ha lo scopo fondamentale di associare le entry grezze ad effettivi eventi di guasto tramite una tecnica di **coalescenza temporale**.

Lo scopo è raggruppare più entry in "tuple" basandosi sulla loro prossimità temporale. La logica utilizzata per il raggruppamento è definita come segue: date due entry consecutive X_i e X_{i+1} con timestamp $t(X)$, esse appartengono alla stessa tupla se:

$$\text{IF } (t(X_{i+1}) - t(X_i) < C_{WIN}) \text{ THEN } X_{i+1} \in \text{Current Tuple}$$

Dove C_{WIN} rappresenta appunto la finestra. Questa tecnica richiede un bilanciamento accurato:

- Una finestra troppo piccola causa errori di truncation (spezza un singolo guasto in due).
- Una finestra troppo grande causa errori di collision (unisce due guasti distinti in uno solo).

Per determinare il valore ottimale, è stata condotta un'analisi di sensitività osservando la variazione del numero di tuple al variare di C_{WIN}. Si applica quindi la regola empirica cercando il punto in cui la curva si stabilizza.

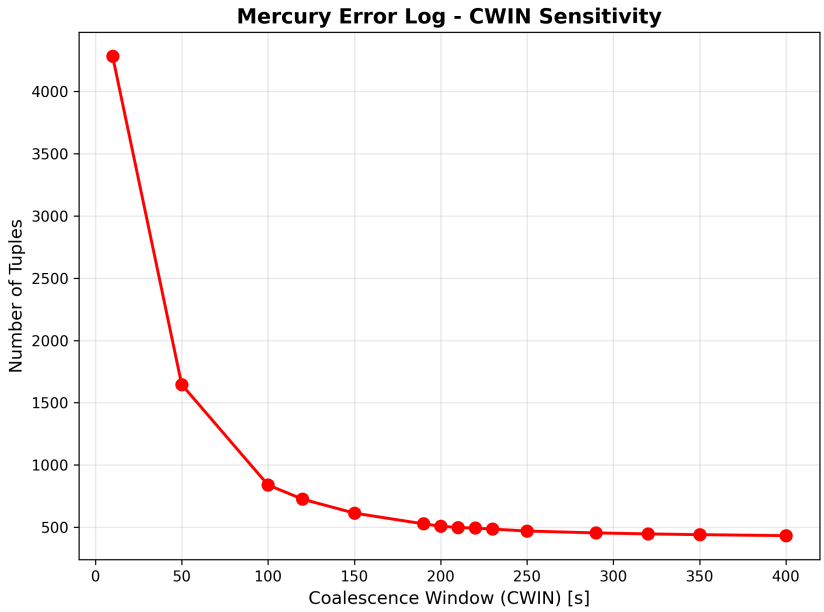


Figura 118: Analisi di Sensitività per la scelta della CWIN

Dall'analisi del grafico in Figura 118, il punto di ginocchio è stato identificato in corrispondenza di:

$$C_{WIN} = 200 \text{ secondi}$$

Modellazione e Fitting della Reliability Applicando la finestra di 200 secondi all'intero dataset, è stato possibile estrarre i tempi di interarrivo e calcolare la funzione di Reliability empirica.

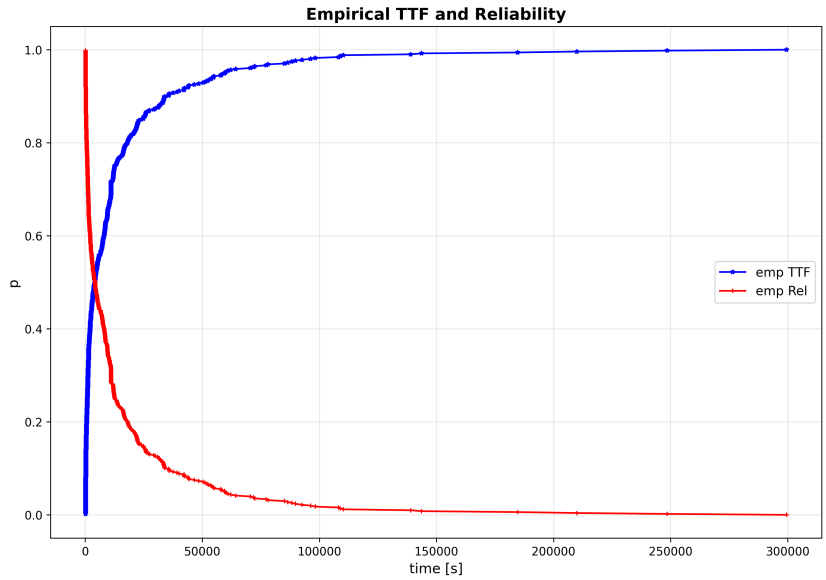


Figura 119: Reliability Empirica e distribuzione dei TTF

Il passo successivo è effettuare il fitting della nostra curva con le seguenti distribuzioni teoriche. Le funzioni utilizzate sono le seguenti:

- **Distribuzione Esponenziale:**

$$f(t) = a \cdot e^{b \cdot t}$$

- **Distribuzione Iper-esponenziale:**

$$f(t) = a \cdot e^{b \cdot t} + c \cdot e^{d \cdot t}$$

- **Distribuzione Weibull:**

$$f(t) = e^{-b \cdot t^a}$$

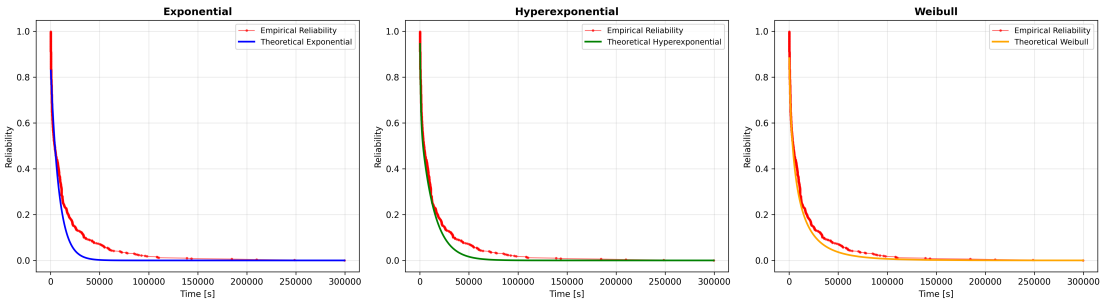


Figura 120: Fitting della Reliability Empirica vs Modelli Teorici.

I risultati numerici del fitting sono riassunti nella Tabella 8. È stato inoltre eseguito il test di Kolmogorov-Smirnov identificare il modello che meglio descrive il comportamento fisico del sistema.

Modello	**SSE**	R^2	**p-value (K-S)**	**Esito Test**
Esponenziale	2.6415	0.9375	$1.0 \cdot 10^{-6}$	FAIL
Iper-esponenziale	0.9005	0.9787	0.0021	FAIL
Weibull	**0.6754**	**0.9840**	**0.2997**	**PASS**

Tabella 8: Confronto bontà del fit per Mercury. La Weibull è l'unica a superare il test K-S ($p > 0.05$) minimizzando l'errore SSE.

Il modello **Weibull** risulta essere il migliore, presentando l'errore quadratico (SSE) minore e l'unico p-value che non rigetta l'ipotesi nulla. I parametri ottimali stimati dal modello di fitting per la distribuzione Weibull ($f(t) = e^{-b \cdot t^a}$) sono i seguenti:

- **Parametro a (shape):** 0.70
- **Parametro b (scale coeff):** 0.0023.

A partire da questi parametri è stato calcolato il Mean Time To Failure del sistema, che risulta essere:

$$MTTF \approx 7200 \text{ secondi} \quad (\approx \textbf{2 ore}).$$

7.1.3 Analisi esplorativa per Categoria e Nodo (Local Analysis)

Per identificare i colli di bottiglia, l'analisi è stata ripetuta filtrando i log per categoria di errore e per nodo fisico.

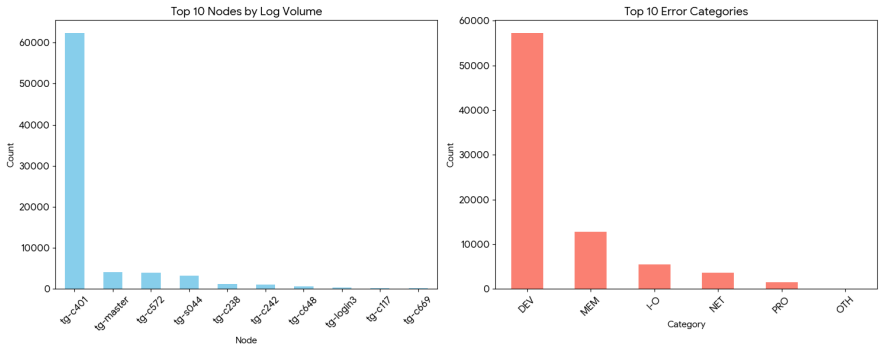


Figura 121: Analisi esplorativa Nodi e Categoria

Analisi per Nodo L'analisi focalizzata sui singoli nodi ha evidenziato che i nodi di calcolo (in particolare `tg-c401`) e il nodo master sono significativamente meno affidabili rispetto ai nodi di login o storage, probabilmente a causa del maggior carico di lavoro (stress factor).

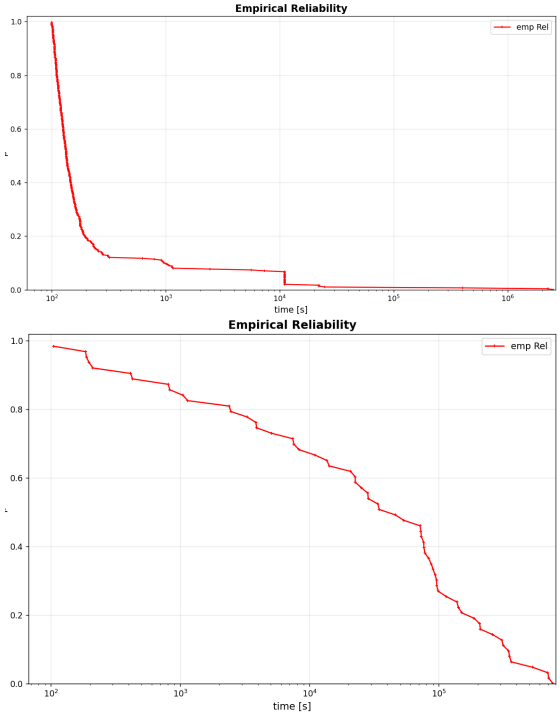


Figura 122: Confronto Reliability per i nodi c401 (Sopra) Master (Sotto) in scala logaritmica)

Da questo possiamo notare come il nodo di calcolo c401 decada molto rapidamente, sebbene la sua reliability può essere considerata non nulla per un tempo

ragionevolmente alto. Il nodo master invece sebbene decada un po' piu lentamente raggiunge lo zero di reliability molto prima.

Analisi per Categoria Una volta che abbiamo individuato i due dei nodi piu critici del sistema si sono prese in esame le diverse categorie confrontando le rispettive curve di affidabilità. È emerso che:

- La categoria DEV e MEM sono quelle più preponderanti nel nodo di calcolo c401.
- NET e I/O sono le categorie principali del tg-master .

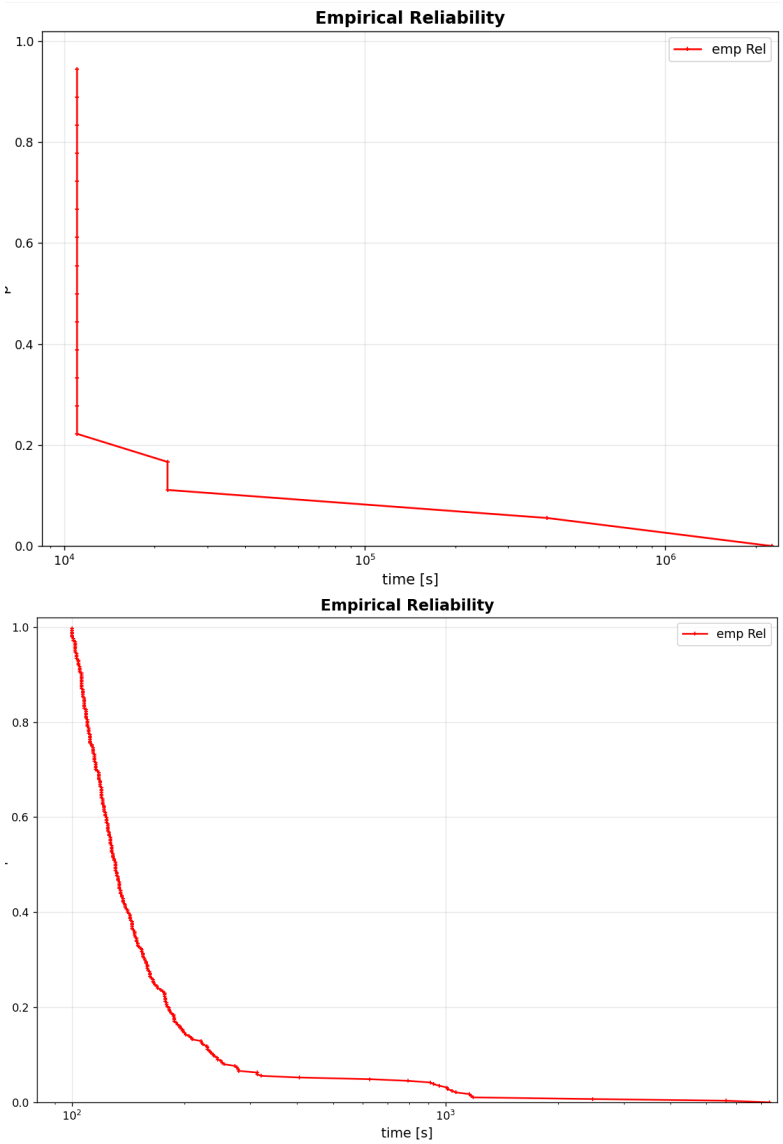


Figura 123: Confronto Reliability per Categoria (DEV sopra MEM sotto) tg-c401

Da una prima analisi è stato riscontrato che piu dell'**80%** degli errori di memoria e piu del **90%** degli errori di DEV appartengono al nodo c401. Questa analisi

potrebbe portare alla conclusione che evidentemente qualche device collegato al nodo c401 potrebbe essere guasto oppure che ci potrebbe essere qualche banco di ram difettoso.

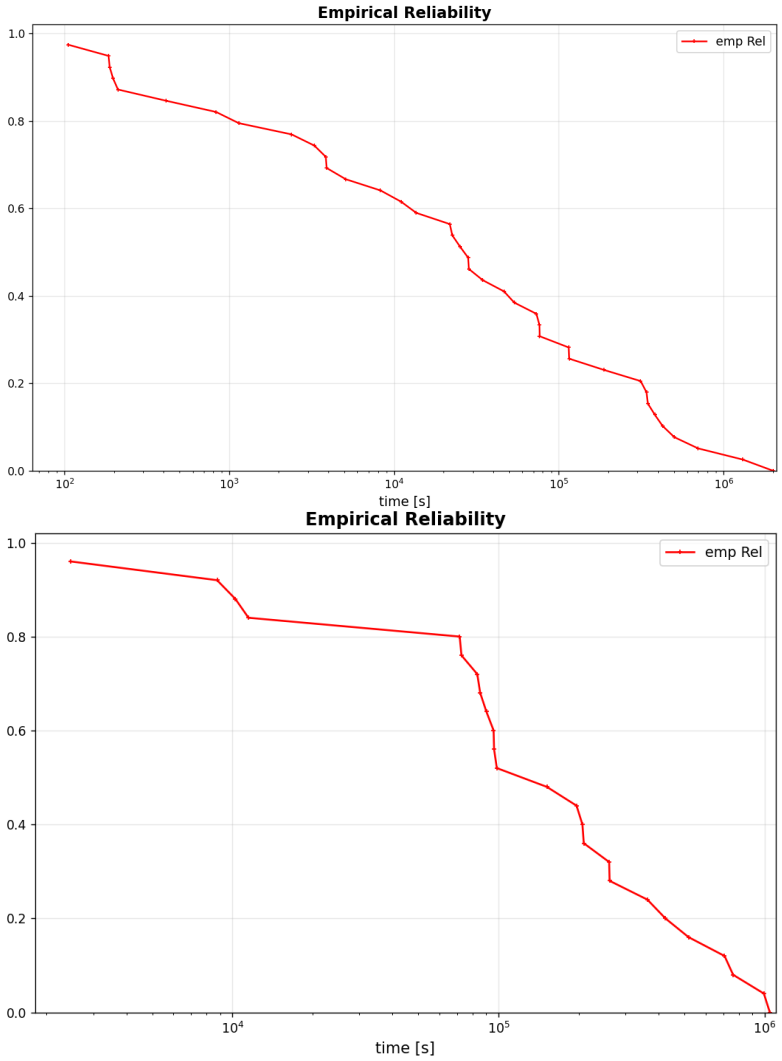


Figura 124: Confronto Reliability per Categoria (NET sopra I-O sotto) tg-master

Per quanto riguarda il nodo master è possibile notare che gli errori principali sono di categoria NET e IO, questo però è in linea con quello che è il ruolo del tg-master, ovvero un nodo dedicato alla gestione dei nodi e quindi ad assicurarsi che ogni nodo sia connesso alla rete. Questi errori tuttavia non sono imputabili al nodo master che infatti si limita solo a segnalare la disconnessione di uno o più nodi. Gli errori di IO invece potrebbero dipendere da errori scaturiti da qualche nodo di login che si sono propagati al nodo master.

7.2 Sistema BlueGene/L

7.2.1 Descrizione del Dataset e Architettura

Il secondo caso di studio riguarda **BlueGene/L**, un supercomputer dall'architettura parallela progettato per raggiungere prestazioni nell'ordine dei petaFLOPS. A differenza del cluster eterogeneo Mercury, BG/L presenta una struttura hardware estremamente regolare e gerarchica, così organizzata:

- **Racks (Cabinets):** Il livello più alto dell'aggregazione fisica.
- **Midplanes:** Ogni rack contiene due midplanes.
- **Node Cards:** Schede che raggruppano i nodi di calcolo.
- **Compute Cards:** Schede contenenti i chip processori e la memoria DRAM.

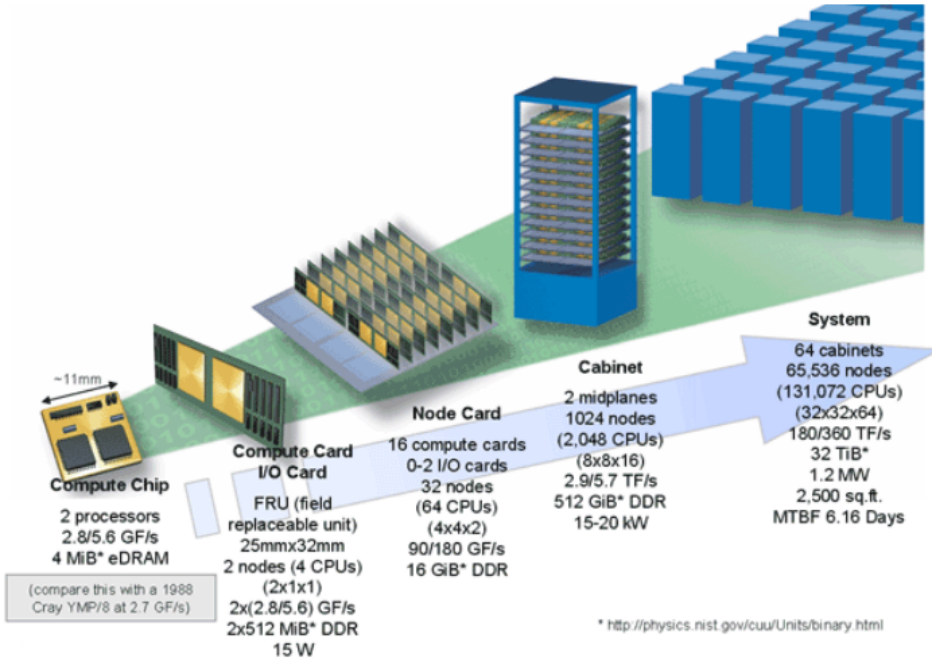


Figura 125: Rappresentazione della gerarchia hardware di BlueGene/L: dai Racks ai singoli Compute Chips.

Un aspetto critico dell'architettura riguarda l'I/O: i nodi identificati come N0, N4, N8 e NC in ogni midplane sono dotati di una scheda di I/O aggiuntiva (identificata come **J18**), assente negli altri nodi. La nomenclatura utilizzata nei log segue lo schema `Rxx-Mx-Nx-C:Jxx-Uxx`, permettendo di localizzare univocamente ogni componente fino al singolo chip.

Il formato di ogni entry include:

- Timestamp dell'evento.
- Nodo di origine.

- Scheda di origine.
- Messaggio di testo libero.

7.2.2 Analisi Globale del Sistema

Anche per BG/L, la metodologia di analisi segue i passi illustrati per Mercury. La fase di **coalescenza temporale** è stata applicata per raggruppare eventi consecutivi riferiti allo stesso guasto fisico.

Utilizzando la medesima logica è stata individuata la finestra temporale ottimale per questo sistema. L'obiettivo è sempre quello di trovare il compromesso ideale tra il raggruppamento corretto degli eventi e l'evitare collisioni improprie.

L'andamento del numero di tuple al variare della finestra temporale è mostrato nel grafico seguente:

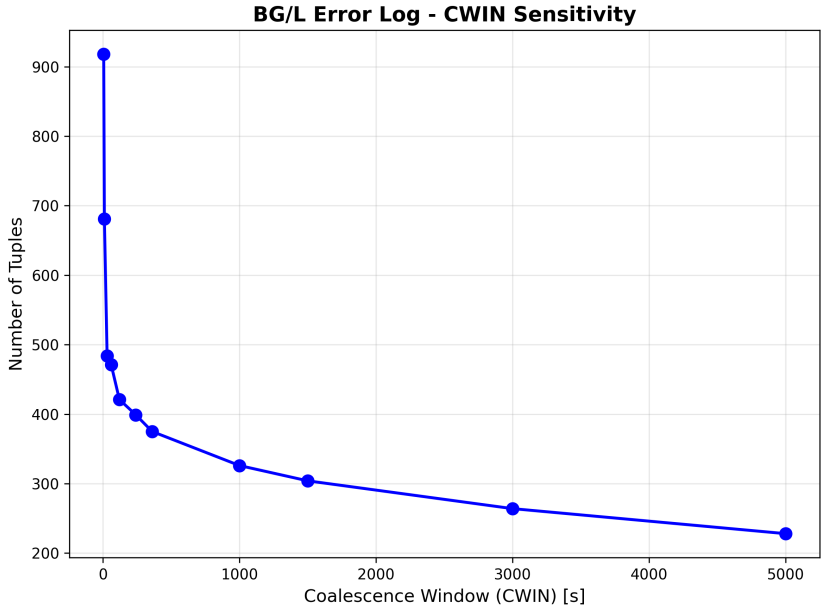


Figura 126: Analisi di Sensitività per la scelta della CWIN.

Dall'analisi del grafico in Figura 126, applicando la regola del ginocchio per stabilizzare il numero di eventi identificati, è stato selezionato un valore di:

$$C_{WIN} = 500 \text{ secondi}$$

Modellazione e Fitting della Reliability Applicando la finestra $C_{WIN} = 500s$ al dataset filtrato, sono stati estratti i TTF e calcolata la curva di affidabilità empirica. Il grafico seguente mostra l'andamento della Reliability (rosso) e della probabilità cumulativa (blu) risultanti dall'applicazione della finestra temporale scelta.

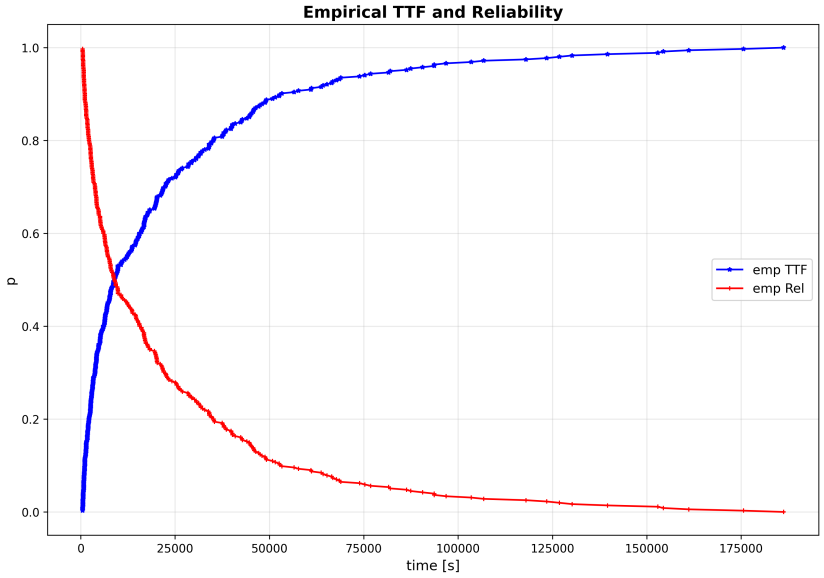


Figura 127: Reliability Empirica e distribuzione dei TTF

Successivamente, è stato eseguito il confronto con i modelli teorici (Esponenziale, Iper-esponenziale, Weibull) per identificare la distribuzione che meglio approssima il comportamento reale del sistema.

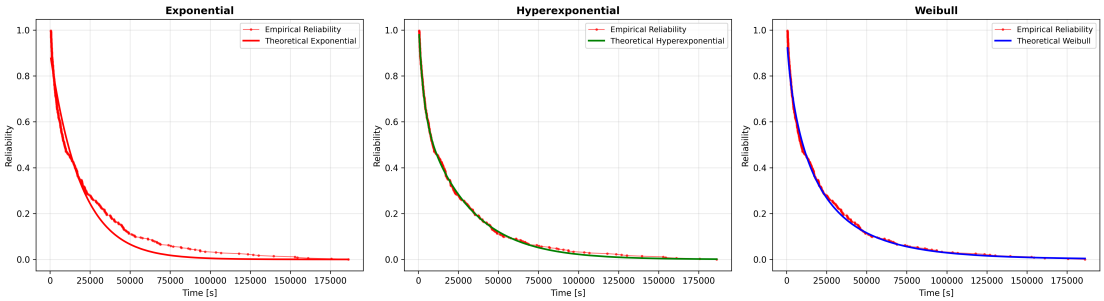


Figura 128: Fitting della Reliability Empirica vs Modelli Teorici (BlueGene/L).

I risultati numerici del fitting, riportati nella Tabella 9, mostrano uno scenario differente rispetto a Mercury.

Modello	**SSE**	R^2	**p-value (K-S)**	**Esito Test**
Esponenziale	1.3828	0.9532	0.0138	FAIL
Iper-esponenziale	**0.3977**	**0.9866**	**0.4460**	**PASS**
Weibull	0.5980	0.9798	0.1585	PASS

Tabella 9: Confronto bontà del fit per BG/L. Sia l'Iper-esponenziale che la Weibull superano il test, ma la prima minimizza l'errore.

In questo caso, il modello Iper-esponenziale risulta essere il migliore in assoluto, offrendo il valore di SSE più basso (0.3977) e il p-value più alto (0.446), indicando

un adattamento eccellente ai dati. Anche la distribuzione Weibull supera il test di Kolmogorov-Smirnov ($p > 0.05$), ma con una precisione leggermente inferiore.

I parametri stimati per il modello (Iper-esponenziale: $f(t) = a \cdot e^{bt} + c \cdot e^{dt}$) sono:

- $a = 0.4334$
- $b = -0.00032$
- $c = 0.6168$
- $d = -3.609 \cdot 10^{-5}$

Utilizzando questi parametri, è stato calcolato il Mean Time To Failure del sistema, dato dalla somma degli integrali delle due componenti esponenziali ($-\frac{a}{b} - \frac{c}{d}$):

$$MTTF \approx 18444 \text{ secondi} \quad (\approx \textbf{5.12 ore}).$$

7.2.3 Analisi Esplorativa per Computational card e Nodo

Anche qui si è cercato di isolare tramite filtraggio i singoli nodi e le singole Computational Card.

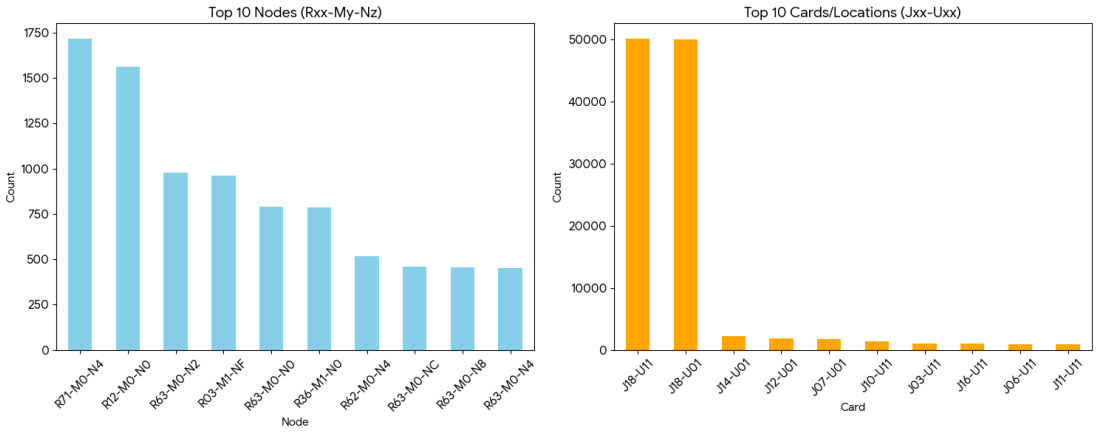


Figura 129: Analisi Esplorativa Nodi e Computational Card

L'analisi mette subito in evidenza come per i nodi non ci sia una grande differenza di occorrenza di failure che sembrano ben spalmati su tutti i nodi. Analizzando invece le Computational card è molto facile vedere dei picchi concentrati sulla Card J18 che è proprio la componente aggiunta al sistema.

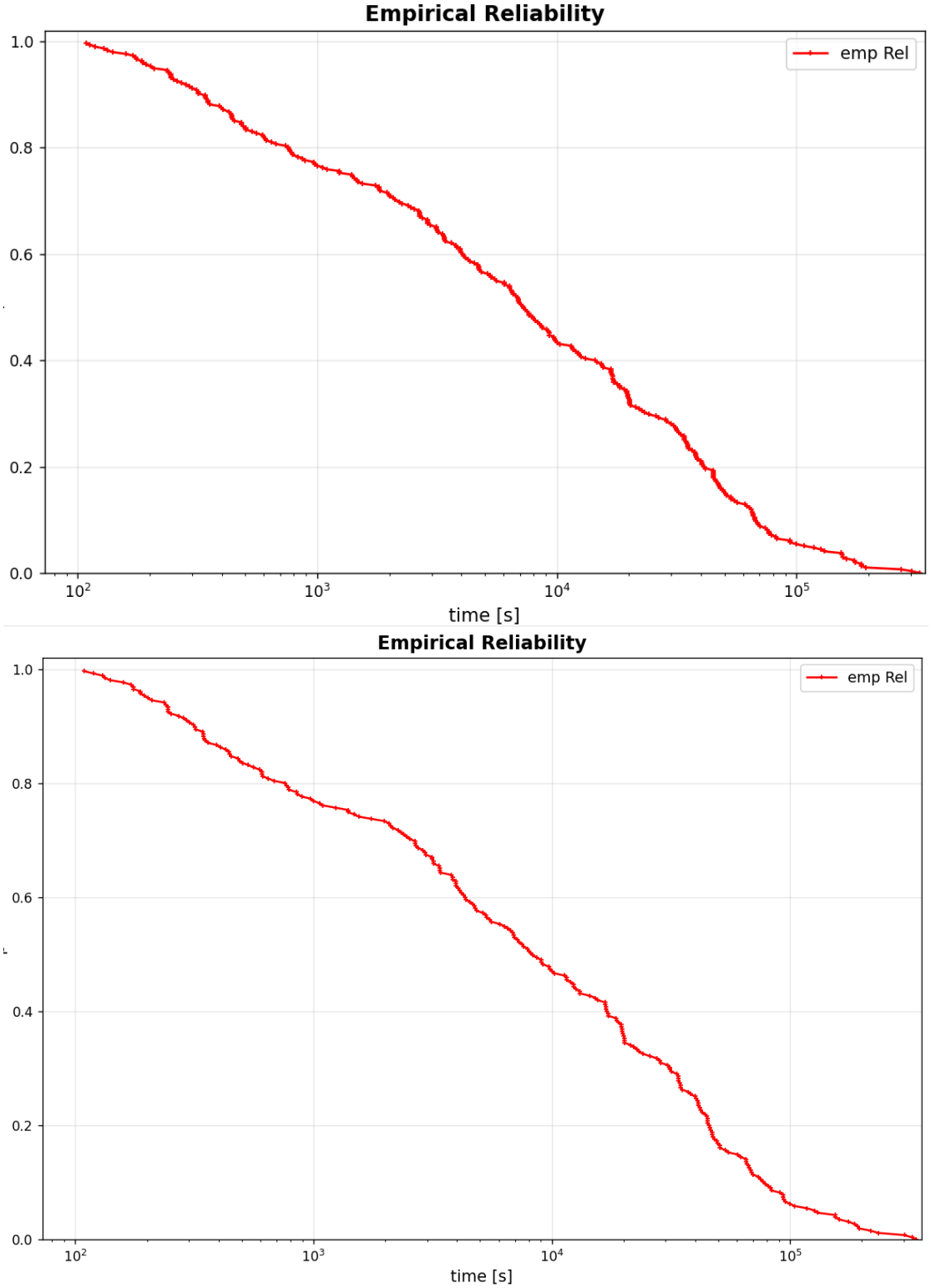


Figura 130: Analisi reliability nodo J18-U11(Sopra) J18-U01(Sotto)

Osservando le due funzioni di reliability è possibile notare la loro somiglianza. Ricordiamo che queste sono le cards I/O aggiuntive che non tutti i nodi possiedono e da tale analisi si evidenzia la loro criticità nel sistema.

7.3 Conclusioni e Confronto

L'analisi FFDA ha permesso di caratterizzare il comportamento dei guasti nei due sistemi:

- **Modello di Guasto:** In entrambi i casi, la distribuzione Weibull offre un fit significativo , sebbene per il sistema BG/L il fit migliore sia la distribuzione iper esponenziale, suggerendo che i guasti non sono puramente casuali (Esponenziale) ma seguono dinamiche complesse.
- **MTTF:** Il cluster BG/L si dimostra complessivamente più affidabile rispetto a Mercury. Sebbene il calcolo esatto dipenda dai parametri di scala, le curve di affidabilità mostrano un decadimento più lento per BG/L.
- **Colli di Bottiglia:** In Mercury le criticità sono diffuse sui dispositivi generici probabilmente legati al nodo c401 (DEV), mentre in BG/L sono fortemente localizzate sulla schede aggiuntiva di I/O (J18), permettendo strategie di manutenzione più mirate.

8 Benchmarking

Il seguente esercizio ha lo scopo di valutare le prestazioni in termini di tempi di esecuzione di due processori differenti. Il workload utilizzato per testare è il benchmark N-body, ovvero un algoritmo di simulazione di un sistema gravitazionale con N pianeti.

Le caratteristiche dei processori sono le seguenti.

- 11th Gen intel(R) Core(TM) i7-1165G7 2.80GHz (Laptop1)
- 13th Gen intel(R) Core(TM) i7-13700H 2.40GHz (Laptop2)

8.1 Raccolta Dati e calcolo dei campioni minimi

Il benchmarking è stato strutturato in modo tale da testare i due dispositivi a 3 diversi livelli di carico 10.000, 100.000 e 1.000.000 numeri di corpi. Fissato n, è stato inzialmente preso un pre-campione di 30 osservazioni, eseguendo di volta in volta un reboot del sistema per cercare di mantenere l'indipendenza. Ogni osservazione è stata mediata eseguendo lo script 5 volte.

Sulla base di questo pre-campione, è stato possibile determinare la dimensione campionaria necessaria affinché la media osservata ricadesse all'interno di un prefissato margine di errore (E) rispetto alla media reale, con un dato livello di confidenza. Attraverso un'analisi di trade-off tra precisione e numero di esecuzioni, il margine di errore ottimale è stato fissato al **10%** della media campionaria

Per il calcolo del numero di campioni necessari è stata quindi applicata la seguente formula, dove con S indichiamo la varianza campionaria.

$$n = \left(\frac{z_{\frac{\alpha}{2}} * S}{E} \right)^2$$

Le 30 osservazioni sono necessarie per rientrare nelle assunzioni del Teorema del Limite Centrale il quale garantisce che la distribuzione delle medie campionarie tenda alla normalità indipendentemente dalla distribuzione della popolazione di partenza. Ciò consente di utilizzare il coefficiente z della distribuzione normale standardizzata come una valida approssimazione per il calcolo della *sample size*.

Applicando la formula queste sono le conclusioni che siamo riusciti ad ottenere.

	N10000P1	**N10000P2**	**N100000P1**	**N100000P2**	**N1000000P1**	**N1000000P2**
Mean	2.285,5 ms	2.021,93	5.272,26	3.929,4	37.123,47	23.855,8
S	595,1 ms	513,91	543,50	447,18	2.684,16	316,68
n	30 Samples	28 Samples	5 Samples	6 Samples	3 Samples	1 Sample

Tabella 10: Statistiche riassuntive sui sample

I risultati confermano che la scelta di 30 campioni rispetti pienamente i vincoli stabiliti.

Di seguito abbiamo analizzato le distribuzioni delle osservazioni per ogni N ed ogni processore.

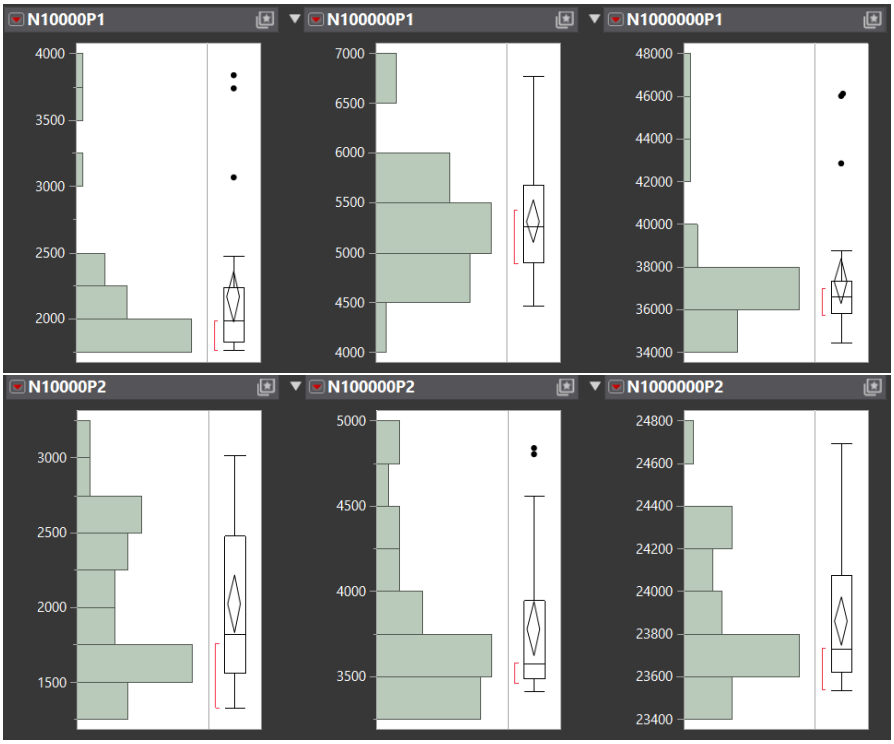


Figura 131: Distribuzione delle osservazioni

Dai grafici possiamo vedere come le nostre osservazioni si distribuiscano secondo una distribuzione asimmetrica pendente verso destra. Questo fenomeno accade a causa della natura delle componenti aleatorie tipiche di un calcolatore. Infatti, a differenza di componenti aleatorie tipiche per le misurazioni fisiche, offrono un contributo moltiplicativo e non additivo all'errore della misurazione, generando cosi questa distribuzione asimmetrica che non risulta essere normale, ma log-normale.

8.2 Test di normalità sui residui

Il prossimo passo è quello di capire se i dati sono stati effettivamente raccolti in modo corretto, ovvero in modo indipendente. Per fare questo è stata effettuata una stima lineare sulle osservazioni e sono stati ricavati i residui, i quali, se tutto è stato fatto correttamente, dovrebbero distribuirsi lungo una normale.

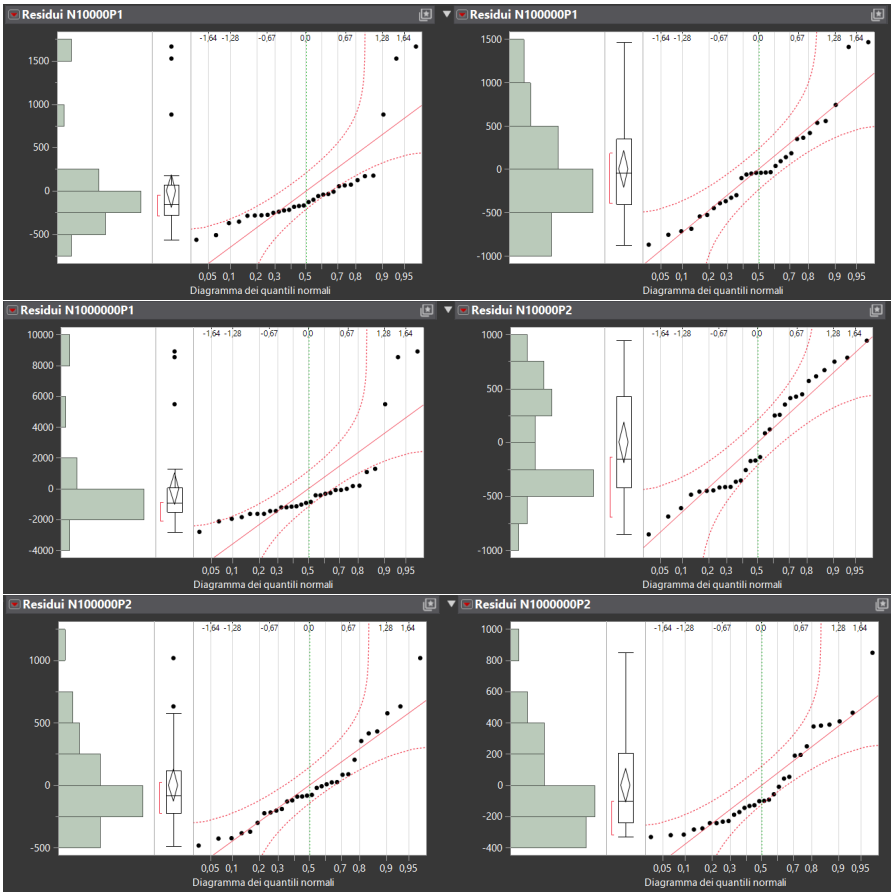


Figura 132: Test visivo di normalità sui residui

Dal grafico tramite un test visivo è possibile notare che non tutti i residui sono distribuiti lungo una normale, in particolare i residui del laptop 1 per 10.000 e 1.000.000 escono molto fuori dal range. Questa non normalità potrebbe dipendere dagli outlier osservati nella distribuzione di partenza e non da osservazioni dipendenti. Per questi casi sono stati scelti quindi dei test non parametrici, essendo piu robusti agli outlier. Il resto delle distribuzioni possono essere considerate normali. Da queste considerazioni iniziali possiamo già iniziare a capire quali test usare per confrontare per ogni N le due distribuzioni.

- N = 10.000 caso residui non normali (causa outlier), inutile fare ulteriori test. Test non parametrico Wilcoxon.
- N = 100.000 caso residui normali applichiamo il t-test appaiato.
- N = 1.000.000 caso residui non normali(causa outlier), inutile fare ulteriori test. Test non parametrico Wilcoxon.

8.3 Test di ipotesi

Quindi come già annunciato abbiamo effettuato i tre test descritti (due non parametrici e uno parametrico), di seguito i risultati.

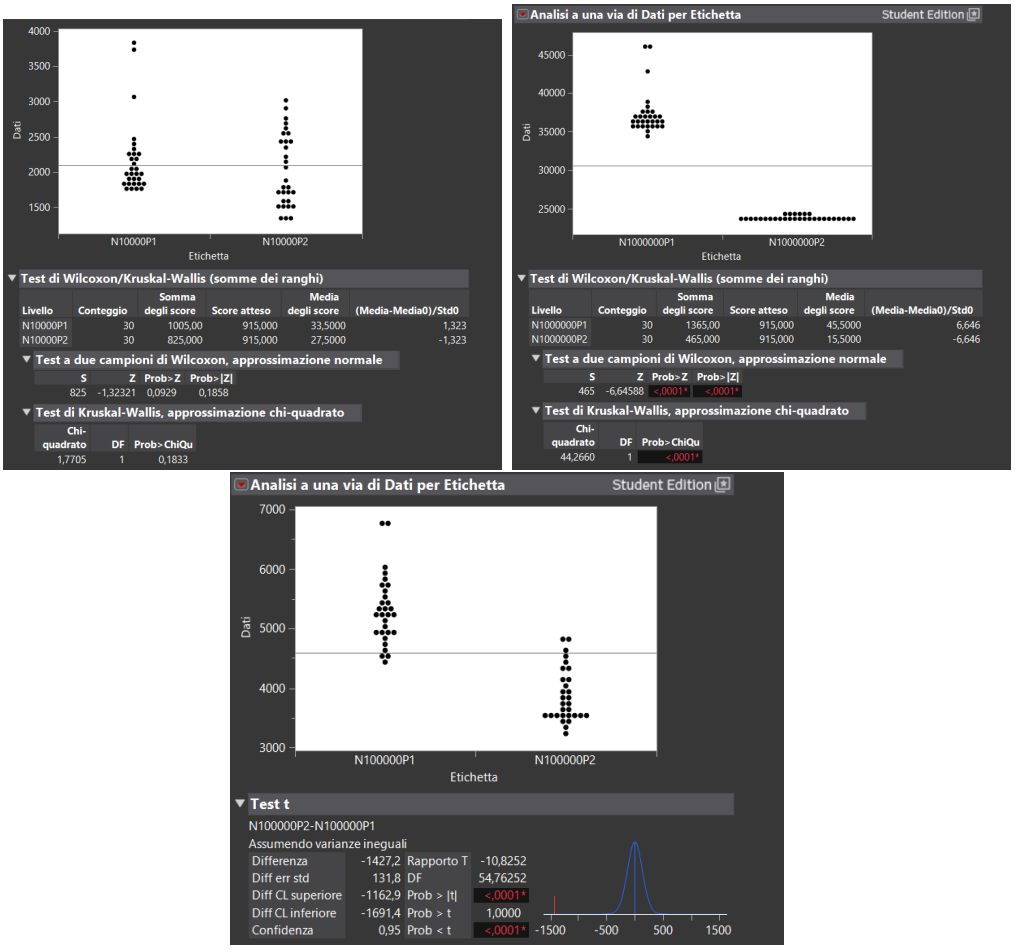


Figura 133: Test di ipotesi sulle distriuzioni

Come ultima analisi delle performance siamo giunti quindi alla conclusione che il processore del laptop 2 è significativamente piu veloce di quello del laptop 1 sia per N = 100.000 che per N = 1.000.000. Per quanto riguarda N = 10.000 non è possibile concludere nulla probabilmente poichè l'errore commesso per N piccolo spiega molta più varianza rispetto a N grandi.